

Hands-On Machine Learning Notes

**Study Notes and Teaching Materials Based on Hands-On Machine Learning with
Scikit-Learn and PyTorch**

Jonathan Florez

Table of contents

Preface	3
1 Introduction	4
I Part I — The Fundamentals of Machine Learning	5
4 Chapter – Classification	8
4.1 MNIST (Modified National Institute of Standards and Technology database)	8
4.2 Training a Binary Classifier	11
4.3 Performance Measures	12
4.3.1 Measuring Accuracy Using Cross-Validation	12
4.3.2 Confusion Matrix	14
4.3.3 Precision and Recall	17
4.3.4 Precision–Recall Trade-off	21
4.3.5 The ROC Curve	33
4.3.6 Multiclass Classification	43
4.3.7 Inspecting the Decision Scores	46
4.3.8 Forcing One-versus-Rest	48
4.3.9 Error Analysis	55
4.3.10 Cross-validated Predictions	55
4.3.11 Focusing Only on Errors	57
4.3.12 Accuracy	61
4.4 A Second Model: Logistic Regression	62
5 Chapter – Training Models	63
5.1 Logistic Regression	63
5.1.1 Estimating Probabilities	63
5.1.2 Training Logistic Regression	65
5.1.3 Decision Boundaries	67
5.1.4 Softmax Regression	76
6 Chapter – Support Vector Machines	85
6.1 Introduction	85
6.2 Linear SVM classification	85
6.2.1 Margin and support vectors	86

6.2.2	A maximum-margin classifier	87
6.3	Why feature scaling matters	89
6.4	Hard-margin classification	92
6.5	Soft-margin classification	92
6.5.1	Visualizing the effect of C	93
6.6	Hinge loss	94
6.7	Nonlinear SVM classification	96
6.7.1	Polynomial features	99
6.8	The kernel trick	101
6.8.1	Linear kernel	102
6.8.2	Polynomial kernel	102
6.8.3	Radial basis function kernel	102
6.9	Polynomial kernel	102
6.10	Similarity features	104
6.11	Gaussian RBF kernel	110
6.11.1	Interpreting γ	111
6.11.2	Interaction between C and γ	111
6.12	Multiclass classification	114
6.13	SVM regression	114
6.13.1	Linear SVR	115
6.13.2	Nonlinear SVR	116
6.14	Computational considerations	118
6.15	Practical modeling workflow	118
6.16	Common mistakes	119
6.17	Chapter summary	119
6.18	Exercises	120
7	Chapter – Decision Trees	121
7.1	Introduction	121
7.2	Training a classification tree	122
7.3	Visualizing the tree with Python	122
7.4	How the tree makes predictions	124
7.5	Samples, values, and predicted classes	124
7.6	Gini impurity	125
7.7	Decision boundaries	126
7.8	Estimating class probabilities	129
7.9	The CART training algorithm	129
7.10	Gini impurity versus entropy	130
7.11	Why Decision Trees overfit	132
7.12	Regularization hyperparameters	135
7.13	Pre-pruning and post-pruning	138
7.13.1	Pre-pruning	138
7.13.2	Post-pruning	138

7.14	Feature importance	140
7.15	Regression trees	141
7.16	Regression splitting criterion	143
7.17	Overfitting in regression trees	144
7.18	Instability of Decision Trees	145
7.19	Decision Trees versus Extra Trees	147
7.20	Computational complexity	150
7.21	Advantages of Decision Trees	150
7.22	Limitations of Decision Trees	151
7.23	Practical modeling workflow	151
7.24	Common mistakes	152
7.25	Chapter summary	152
7.26	Exercises	153
II	Part II — Neural Networks and Deep Learning	154
8	Logistic and Softmax Regression	155
8.1	Logistic Regression	155
8.1.1	Estimating Probabilities	155
8.1.2	Training Logistic Regression	156
8.1.3	Decision Boundaries	158
8.1.4	Softmax Regression	164
III	Additional Material	171
9	Summary	172
	References	173

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Introduction

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

Part I

Part I — The Fundamentals of Machine Learning

2

3

4 Chapter – Classification

4.1 MNIST (Modified National Institute of Standards and Technology database)

Throughout this chapter, we will use the **MNIST** (Modified National Institute of Standards and Technology) dataset, one of the most widely used benchmark datasets for image classification. It contains **70,000 grayscale images** of handwritten digits (0–9), each with a resolution of **28 × 28 pixels**. Every image is labeled with the digit it represents, making MNIST a supervised multiclass classification problem with ten possible classes.

The dataset was created by combining and preprocessing digit images collected from high school students and employees of the United States Census Bureau. Although MNIST is considered a relatively simple benchmark by modern standards, it remains an excellent dataset for introducing the fundamental concepts of classification, model evaluation, error analysis, and multiclass learning.

Since every image contains 28×28 pixels, each instance can be represented as a vector of 784 numerical features. Throughout this chapter, we will use these feature vectors to train, evaluate, and compare different classification algorithms.

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml(
    "mnist_784",
    as_frame=False,
    parser="auto"
)
```

```
mnist.keys()
```

Datasets loaded with Scikit-Learn are typically returned as a dictionary-like object (a Bunch) that stores both the data and its associated metadata. Although the exact contents depend on the dataset, most of them include the following keys:

- **DESCR**: a textual description of the dataset.

- **data**: a two-dimensional array containing the feature values, where each row represents an instance and each column corresponds to a feature.
- **target**: a one-dimensional array containing the class labels or target values associated with each instance.

Some datasets, such as those downloaded from OpenML, also include additional metadata such as feature names, category information, download details, and the dataset source.

Each row of the feature matrix represents a handwritten digit as a one-dimensional vector of 784 pixel intensities. To visualize one of these images, we first select an instance from the dataset and then reshape its feature vector into its original 28×28 pixel format.

```
X, y = mnist.data, mnist.target

print("Feature matrix:", X.shape)
print("Target vector:", y.shape)

import matplotlib.pyplot as plt

some_digit = X[0]
some_digit_image = some_digit.reshape(28, 28)

plt.figure(figsize=(3, 3))
plt.imshow(some_digit_image, cmap="binary")
plt.axis("off")
plt.show()
```

Figure 4.1

The target labels returned by OpenML are stored as strings. Since most machine learning algorithms in Scikit-Learn expect numerical labels, we convert the target vector to 8-bit unsigned integers (`uint8`). This data type is sufficient because the labels range only from 0 to 9, while also requiring very little memory.

```
import numpy as np

y = y.astype(np.uint8)
```

Before training a classifier, we separate the dataset into a training set and a test set. The original MNIST dataset provided by OpenML is already organized so that the first 60,000 images correspond to the training set, while the remaining 10,000 images form the test set.



Figure 4.2: Examples of handwritten digits from the MNIST dataset.

This predefined split has become the standard benchmark used to compare the performance of classification algorithms.

```
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]
```

4.2 Training a Binary Classifier

To introduce the basic concepts of classification, we will begin with a simpler task: identifying a single digit. Specifically, we will train a model to recognize the digit **5** while treating all other digits as a single alternative class.

This type of problem is known as **binary classification**, since the model must decide between only two possible classes: **5** and **not 5**. Despite its simplicity, binary classification introduces many of the core concepts that also apply to more complex multiclass classification problems.

```
y_train_5 = (y_train == 5)
y_test_5 = (y_test == 5)
```

The expressions `y_train == 5` and `y_test == 5` perform an element-wise comparison, producing Boolean arrays. Each element is `True` if the corresponding image represents the digit **5**, and `False` otherwise. These Boolean arrays become the target labels used to train the binary classifier.

To build our first binary classifier, we will use Scikit-Learn's `SGDClassifier`. This classifier implements **Stochastic Gradient Descent (SGD)**, an optimization algorithm that updates the model parameters incrementally by processing one training instance (or a small batch) at a time. Because of its computational efficiency, SGD is particularly well suited for large datasets such as MNIST.

The classifier is initialized with a fixed random seed (`random_state=42`) to ensure that the results are reproducible.

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

After training, we can use the classifier to predict whether a new image represents the digit **5**. As a first example, we classify the digit selected earlier in the chapter (`some_digit`).

```
prediction = sgd_clf.predict([some_digit])

print("Prediction:", prediction[0])
print("Actual label:", y[0] == 5)
```

The `predict()` method returns a Boolean value because this is a binary classification problem. A prediction of **True** indicates that the classifier believes the image represents the digit **5**, whereas **False** indicates that it belongs to the **not 5** class.

In this example, the prediction matches the true label, indicating that the classifier correctly identified the handwritten digit.

4.3 Performance Measures

Training a classifier is only the first step in the machine learning workflow. Equally important is evaluating how well the model performs on unseen data. A model that achieves excellent results on the training set may still generalize poorly if it has learned patterns that do not extend beyond the training data.

4.3.1 Measuring Accuracy Using Cross-Validation

One of the most widely used techniques for estimating a model's predictive performance is **cross-validation**. Rather than relying on a single train-test split, cross-validation repeatedly partitions the training data into different subsets, allowing the model to be trained and evaluated multiple times. This process produces a more reliable estimate of its ability to generalize to new data.

To better understand how cross-validation works, we will implement a simple three-fold stratified cross-validation procedure manually. Although Scikit-Learn provides functions that perform this process automatically, examining the individual steps helps illustrate how the model is repeatedly trained and evaluated on different subsets of the data.

We use **stratified sampling**, which ensures that each fold preserves approximately the same proportion of positive and negative examples as the complete training set. Since the digit **5** represents only a small fraction of the MNIST dataset, stratification produces more representative training and validation subsets.

```
from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold

skfolds = StratifiedKFold(
```

```

    n_splits=3,
    shuffle=True,
    random_state=42
)

for train_index, test_index in skfolds.split(X_train, y_train_5):

    clone_clf = clone(sgd_clf)

    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]

    X_valid_fold = X_train[test_index]
    y_valid_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)

    y_pred = clone_clf.predict(X_valid_fold)

    accuracy = (y_pred == y_valid_fold).mean()

    print(f"Fold accuracy: {accuracy:.4f}")

```

The results obtained so far may appear encouraging, with an accuracy exceeding 90%. However, this value can be misleading. In the MNIST dataset, only about 10% of the images correspond to the digit 5. Consequently, a trivial classifier that always predicts “not 5” would achieve an accuracy close to 90%, despite never correctly identifying a single image of the target class.

This example highlights one of the main limitations of accuracy as an evaluation metric. When the class distribution is unbalanced, a classifier may obtain a high accuracy simply by favoring the majority class, without actually learning to recognize the minority class. Such datasets are known as imbalanced (or skewed) datasets, and they require performance measures that better reflect the classifier’s ability to distinguish between positive and negative instances.

For this reason, the following sections introduce additional evaluation metrics—including the confusion matrix, precision, recall, and the F1-score—which provide a more informative assessment of classifier performance in imbalanced classification problems.

During each iteration, one fold is temporarily held out as the validation set, while the remaining two folds are used to train the classifier. This process is repeated three times so that every observation serves as validation data exactly once.

The reported accuracies correspond to the classifier’s performance on each validation fold. Because the model is retrained from scratch during every iteration, these values provide a more

reliable estimate of its ability to generalize than evaluating it on the training data alone.

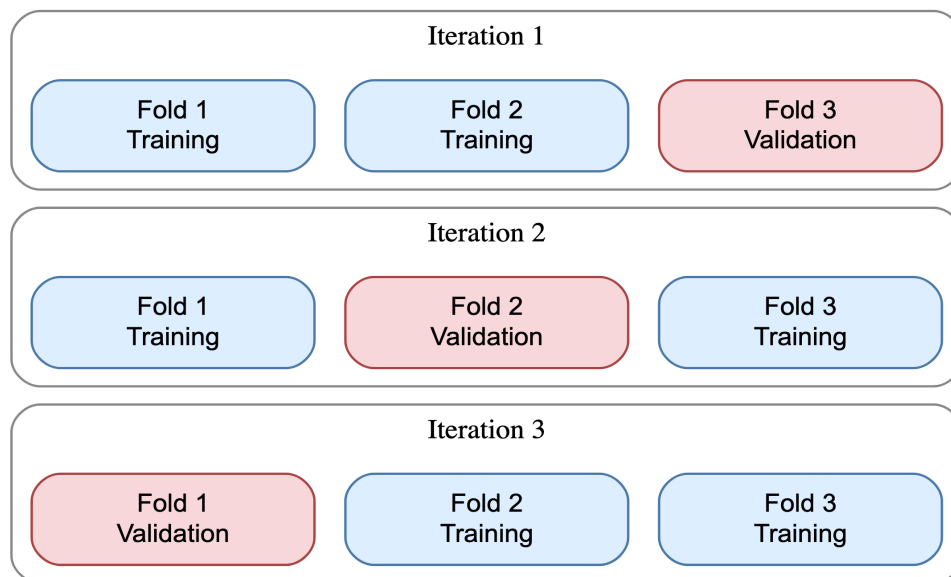


Figure 4.3: Three-fold stratified cross-validation. In each iteration, two folds are used for training and the remaining fold is used for validation.

As shown in Figure 4.3, each fold is used once as the validation set and twice as part of the training set.

4.3.2 Confusion Matrix

In the previous section, we observed that a classifier can achieve an accuracy greater than **90%** while still performing poorly on the class of interest. This example illustrates that **accuracy alone is often insufficient to evaluate a classification model**, particularly when the classes are imbalanced.

A much more informative evaluation tool is the **confusion matrix**, which summarizes how the predicted classes compare with the true classes. Rather than reporting a single overall metric, the confusion matrix reveals **which types of errors** the classifier is making.

For a classification problem with K classes, the confusion matrix is a $K \times K$ table. Each **row** corresponds to the actual class, while each **column** corresponds to the predicted class. Consequently, the element located in row i and column j represents the number of observations that truly belong to class i but were classified as class j .

For example, if we considered the complete MNIST problem, the value located in the fifth row and the third column would indicate the number of images of the digit **5** that were incorrectly classified as **3**. Correct predictions appear along the main diagonal of the matrix, whereas the off-diagonal elements correspond to classification errors.

Before computing the confusion matrix, we need one prediction for every observation in the training set. Although these predictions could be generated using the test set, doing so would compromise its role as an independent dataset for the final evaluation of the model. Instead, we generate **cross-validated predictions** using `cross_val_predict()`.

Unlike `cross_val_score()`, which returns only the evaluation score for each fold, `cross_val_predict()` returns the prediction associated with every observation. During the cross-validation procedure, each observation is predicted by a model that **was not trained using that observation**, providing an unbiased estimate of the classifier's performance on unseen data.

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(
    sgd_clf,
    X_train,
    y_train_5,
    cv=3
)
```

Once these predictions have been generated, the confusion matrix can be obtained by comparing the predicted labels with the true labels.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    y_train_5,
    y_train_pred
)

cm
```

The confusion matrix for a binary classification problem has the following general structure.

As illustrated in Figure 4.4, the rows correspond to the **actual classes**, while the columns correspond to the **predicted classes**. Consequently,

- **True Negatives (TN):** negative observations correctly classified as negative.
- **False Positives (FP):** negative observations incorrectly classified as positive.

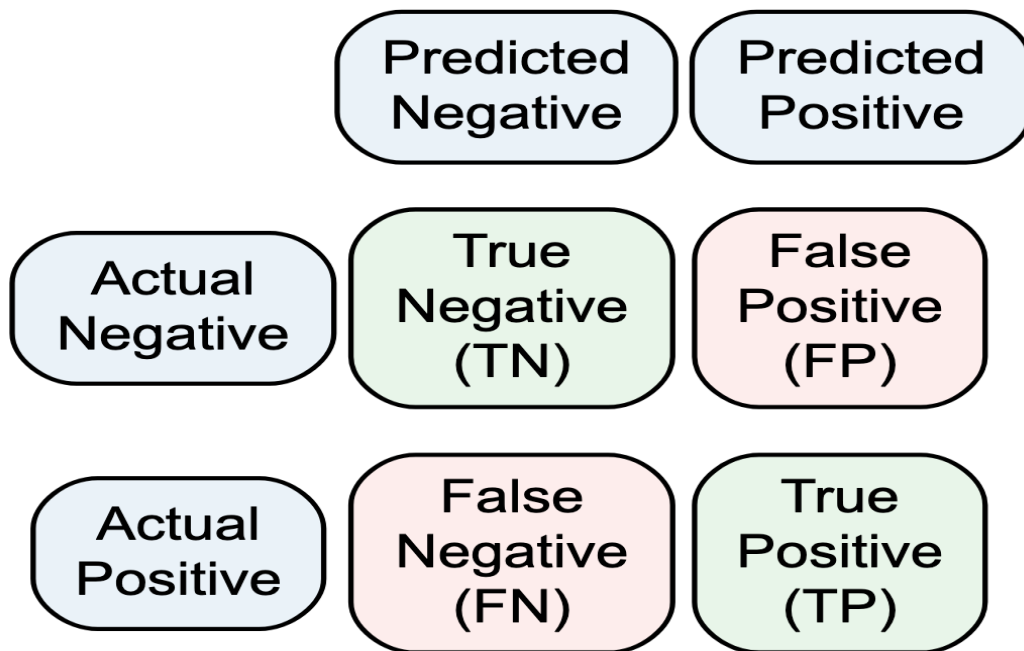


Figure 4.4: General structure of a binary confusion matrix.

- **False Negatives (FN):** positive observations incorrectly classified as negative.
- **True Positives (TP):** positive observations correctly classified as positive.

For our binary classifier, the confusion matrix can be interpreted as follows:

- The **first row** corresponds to images that are **not** the digit **5**. Most of these images were correctly classified as negative (**True Negatives**), while a much smaller number were incorrectly classified as **5** (**False Positives**).
- The **second row** corresponds to images that actually represent the digit **5**. Some of them were incorrectly classified as non-5 (**False Negatives**), whereas the remaining images were correctly identified as **5** (**True Positives**).

The four quantities that summarize the confusion matrix can be extracted directly.

```
tn, fp, fn, tp = cm.ravel()

print(f"True Negatives : {tn}")
print(f"False Positives: {fp}")
print(f"False Negatives: {fn}")
print(f"True Positives : {tp}")
```

A perfect classifier would correctly classify every observation. In that case, the confusion matrix would contain nonzero values only along its main diagonal.

```
y_train_perfect_predictions = y_train_5

confusion_matrix(
    y_train_5,
    y_train_perfect_predictions
)
```

The confusion matrix provides a complete summary of the classifier's predictions. However, comparing several models directly from the four cell counts is often inconvenient. For this reason, a number of evaluation metrics are derived from these quantities. The most commonly used are **precision**, **recall**, and the **F1-score**, which are introduced in the following sections.

4.3.3 Precision and Recall

The confusion matrix provides a detailed summary of a classifier's predictions, but comparing models directly from its four cell counts is often inconvenient. For this reason, several evaluation metrics are derived from the quantities **TP**, **FP**, **TN**, and **FN**. Among the most widely used are **precision** and **recall**, each capturing a different aspect of classification performance.

Although both metrics focus on the positive class, they answer fundamentally different questions. Understanding this distinction is essential because different applications may require optimizing one metric over the other.

4.3.3.1 Precision

Precision measures the reliability of the positive predictions made by a classifier. It answers the following question:

Among all observations predicted as positive, what proportion actually belongs to the positive class?

Mathematically, precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP},$$

where:

- TP is the number of **True Positives**.

- FP is the number of **False Positives**.

A classifier with high precision generates relatively few false alarms. Consequently, precision is particularly important in applications where **false positives are expensive or undesirable**.

Using the predictions generated in the previous section, precision can be computed directly with Scikit-Learn.

```
from sklearn.metrics import precision_score

precision = precision_score(
    y_train_5,
    y_train_pred
)

print(f"Precision: {precision:.4f}")
```

The same quantity can also be verified directly from the confusion matrix.

```
precision_manual = tp / (tp + fp)

print(f"Manual precision: {precision_manual:.4f}")
```

4.3.3.2 Recall

Recall evaluates a different property of the classifier. Instead of measuring how reliable positive predictions are, it measures **how many of the actual positive observations the classifier successfully identifies**.

In other words, recall answers the question:

Among all truly positive observations, what proportion was correctly detected by the classifier?

Recall is defined as

$$\text{Recall} = \frac{TP}{TP + FN},$$

where FN denotes the number of **False Negatives**.

Recall is also known as **Sensitivity** or the **True Positive Rate (TPR)**.

It can be computed as follows.

```
from sklearn.metrics import recall_score

recall = recall_score(
    y_train_5,
    y_train_pred
)

print(f"Recall: {recall:.4f}")
```

Again, the result can be verified directly from the confusion matrix.

```
recall_manual = tp / (tp + fn)

print(f"Manual recall: {recall_manual:.4f}")
```

4.3.3.3 Comparing Precision and Recall

Although both metrics evaluate the positive class, they measure different characteristics of a classifier.

- **Precision** evaluates the reliability of positive predictions.
- **Recall** evaluates the ability to identify all positive observations.

Depending on the application, one metric may be substantially more important than the other.

For example:

- In **email spam detection**, incorrectly classifying a legitimate email as spam can be highly undesirable. In this case, high **precision** is often preferred.
- In **medical screening**, failing to identify a patient with a disease may have serious consequences. Here, maximizing **recall** is usually more important.

Because these objectives often conflict, improving one metric frequently decreases the other. This phenomenon is known as the **precision–recall trade-off**, which we will study in the next section.

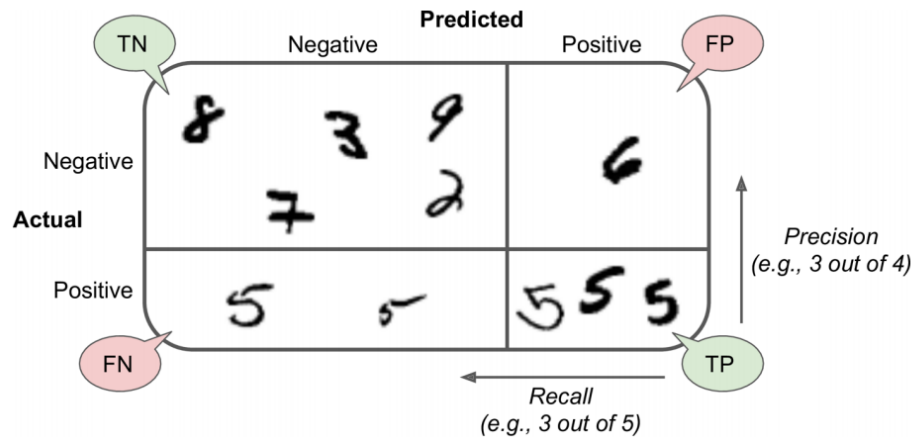


Figure 4.5: An illustrated confusion matrix shows examples of true negatives (top left), false positives (top right), false negatives (lower left), and true positives (lower right)

4.3.3.4 The F1-score

In many situations it is convenient to summarize precision and recall using a single metric. The **F1-score** combines both quantities through their **harmonic mean**.

Unlike the arithmetic mean, the harmonic mean penalizes large differences between its components. Consequently, the F1-score is high only when **both precision and recall are high**.

The F1-score is defined as

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Substituting the definitions of precision and recall yields the equivalent expression

$$F_1 = \frac{2TP}{2TP + FP + FN}.$$

Scikit-Learn computes the F1-score using the `f1_score()` function.

```
from sklearn.metrics import f1_score

f1 = f1_score(
    y_train_5,
    y_train_pred
```

```
)  
  
print(f"F1-score: {f1:.4f}")
```

The F1-score is particularly useful when comparing several classifiers because it summarizes two complementary aspects of performance into a single value. However, it should not replace precision and recall when the application assigns different importance to false positives and false negatives.

i Key takeaway

- **Precision** answers “*When the classifier predicts positive, how often is it correct?*”
- **Recall** answers “*Among all positive observations, how many were detected?*”
- **F1-score** summarizes both metrics and is highest only when precision and recall are simultaneously high.

4.3.4 Precision–Recall Trade-off

In the previous section, we introduced **precision** and **recall** as two complementary measures of classification performance. In practice, however, these metrics are rarely optimized simultaneously. Improving one often comes at the expense of the other, a phenomenon known as the **precision–recall trade-off**.

Understanding this trade-off requires examining how a classifier makes its decisions.

4.3.4.1 Decision Scores and Classification Thresholds

Many classification algorithms do not directly predict a class label. Instead, they first compute a **decision score**, which measures how strongly an observation belongs to the positive class. The final prediction is then obtained by comparing this score with a predefined **decision threshold**.

For the `SGDClassifier`, this process can be summarized as follows:

- If the decision score is **greater than or equal to the threshold**, the observation is classified as **positive**.
- Otherwise, it is classified as **negative**.

By default, the threshold used by `SGDClassifier` is **zero**. However, this value is not necessarily optimal for every application.

Figure 4.6 illustrates this decision process. Observations are ordered according to their decision scores, and the threshold determines which observations are assigned to the positive class.

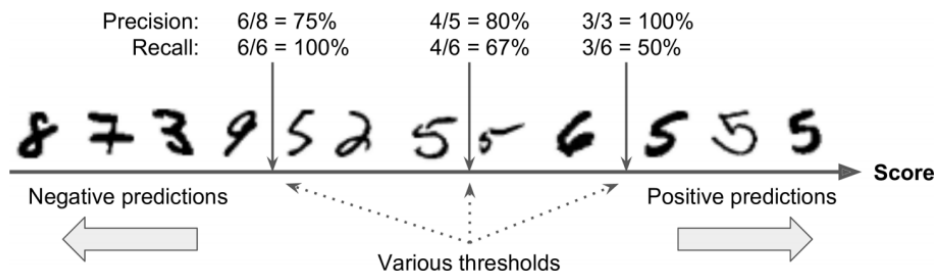


Figure 4.6: Decision scores and the effect of changing the classification threshold.

A **lower threshold** classifies more observations as positive. As a result, the classifier tends to identify a larger proportion of the actual positive observations, increasing **recall**. However, this also increases the number of false positives, which generally reduces **precision**.

Conversely, a **higher threshold** makes the classifier more conservative. Fewer observations are predicted as positive, reducing the number of false positives and typically increasing **precision**. At the same time, more positive observations are missed, causing **recall** to decrease.

This behavior explains why precision and recall usually move in opposite directions. Choosing an appropriate threshold therefore depends on the specific objectives of the application rather than on a universal rule.

Although Scikit-Learn does not allow the classification threshold to be modified directly through the `predict()` method, it provides access to the underlying decision scores through the `decision_function()` method. These scores can then be combined with any threshold selected by the user.

```
y_score = sgd_clf.decision_function([some_digit])

print(f"Decision score: {y_score[0]:.2f}")
```

A prediction equivalent to the default behavior of `predict()` is obtained by comparing the score with a threshold equal to zero.

```
threshold = 0

(y_score >= threshold)
```

Suppose that a more conservative classifier is desired. Increasing the threshold changes the prediction rule.


```
threshold = 8000

(y_score >= threshold)
```

Notice that the same observation may change from a positive prediction to a negative prediction simply because the threshold has changed. The classifier itself has **not** been retrained; only the decision rule has been modified.

This simple example illustrates an important principle in classification: **the prediction depends not only on the model, but also on the decision threshold used to convert scores into class labels.**

In the next section, we will evaluate the classifier over **all possible threshold values** by constructing the **Precision–Recall curve**, which provides a comprehensive view of the trade-off between these two metrics.

4.3.4.2 Selecting a Classification Threshold

Once the classifier’s decision scores are available, the next question is:

Which threshold should be used to convert these scores into class predictions?

There is no universally optimal threshold. The appropriate value depends on the relative importance of **false positives** and **false negatives** in the application.

For example, a medical screening system may prioritize recall because failing to detect a patient with a disease could be especially costly. In contrast, a system that automatically blocks financial transactions may require high precision to avoid incorrectly rejecting legitimate operations.

To analyze the behavior of the classifier across many possible thresholds, we first need decision scores for every observation in the training set.

Instead of generating class predictions with `cross_val_predict()`, we specify `method="decision_function"` so that the function returns the out-of-fold decision score assigned to each observation.

```
from sklearn.model_selection import cross_val_predict

y_scores = cross_val_predict(
    sgd_clf,
    X_train,
    y_train_5,
    cv=3,
```

```
method="decision_function",
n_jobs=-1
)
```

Each score is generated for an observation that was not used to train the corresponding model. This provides a more realistic basis for studying threshold behavior than computing scores directly from a model evaluated on its own training observations.

However, these cross-validated predictions should not be interpreted as a complete estimate of model generalization in every setting. Scikit-Learn notes that results obtained from `cross_val_predict()` may differ from those produced by `cross_validate()` or `cross_val_score()`, particularly when fold sizes differ or the evaluation metric does not decompose over individual observations.

4.3.4.3 Computing Precision and Recall Across Thresholds

The `precision_recall_curve()` function computes precision and recall values for the decision thresholds induced by the classifier scores.

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(
    y_train_5,
    y_scores
)
```

The returned arrays do not have exactly the same length:

```
print(f"Number of precision values: {len(precisions)}")
print(f"Number of recall values: {len(recalls)}")
print(f"Number of thresholds: {len(thresholds)}")
```

Both `precisions` and `recalls` contain one more element than `thresholds`. The final precision and recall values represent the endpoint of the curve and do not correspond to a classification threshold.

Therefore, when plotting precision and recall against thresholds, the final element of each metric array must be excluded.

4.3.4.4 Precision and Recall as Functions of the Threshold

The following function plots both metrics across the available classification thresholds.

```
import matplotlib.pyplot as plt

def plot_precision_recall_vs_threshold(
    precisions,
    recalls,
    thresholds
):
    fig, ax = plt.subplots(figsize=(8, 5))

    ax.plot(
        thresholds,
        precisions[:-1],
        linestyle="--",
        linewidth=2,
        label="Precision"
    )

    ax.plot(
        thresholds,
        recalls[:-1],
        linewidth=2,
        label="Recall"
    )

    ax.set_xlabel("Decision threshold")
    ax.set_ylabel("Score")
    ax.set_ylim(0, 1.05)
    ax.grid(alpha=0.3)
    ax.legend()

    plt.show()

plot_precision_recall_vs_threshold(
    precisions,
    recalls,
    thresholds
)
```

As the threshold increases, the classifier becomes more conservative about assigning observations to the positive class.

In general:

- **Recall decreases** because fewer observations are classified as positive, causing more actual positives to be missed.
- **Precision tends to increase** because the remaining positive predictions are based on stronger decision scores.

Precision is not necessarily perfectly monotonic. Increasing the threshold can occasionally remove a true positive before removing a false positive, producing small downward fluctuations. Recall, in contrast, decreases monotonically as the threshold increases.

The graph allows us to inspect the relationship between the threshold and the resulting metric values. However, selecting a threshold should be based on a performance requirement established by the application, not merely on the visual appearance of the curves.

4.3.4.5 The Precision–Recall Curve

Another useful representation plots precision directly against recall.

This curve removes the threshold from the horizontal axis and displays the combinations of precision and recall that can be achieved by changing the classification threshold.

```
def plot_precision_vs_recall(
    precisions,
    recalls
):
    fig, ax = plt.subplots(figsize=(7, 6))

    ax.plot(
        recalls,
        precisions,
        linewidth=2
    )

    ax.set_xlabel("Recall")
    ax.set_ylabel("Precision")
    ax.set_xlim(0, 1.01)
    ax.set_ylim(0, 1.01)
    ax.grid(alpha=0.3)

    plt.show()
```

```

plot_precision_vs_recall(
    precisions,
    recalls
)

```

Each point on the curve corresponds to a different classification threshold. Moving along the curve illustrates how gaining recall usually requires accepting lower precision, and vice versa.

An ideal classifier would remain close to the upper-right corner, where both precision and recall are high. In practice, the curve helps identify regions where one metric begins to deteriorate rapidly as the other improves.

4.3.4.6 Selecting a Threshold for a Target Precision

Suppose that the application requires the classifier to achieve a precision of at least 90%.

Rather than estimating the threshold visually from the graph, we can identify the first threshold whose associated precision reaches the desired level.

```

import numpy as np

target_precision = 0.90

valid_indices = np.flatnonzero(
    precisions[:-1] >= target_precision
)

if len(valid_indices) == 0:
    raise ValueError(
        "The requested precision cannot be achieved "
        "with the available thresholds."
    )

threshold_index = valid_indices[0]
threshold_90_precision = thresholds[threshold_index]

print(
    "Threshold for at least "
    f"{target_precision:.0%} precision: "
    f"{threshold_90_precision:.2f}"
)

```

Using `precisions[:-1]` is important because the final precision value returned by `precision_recall_curve()` does not have a corresponding threshold.

The selected threshold is the **lowest observed threshold** at which the estimated precision reaches at least 90%. Selecting the lowest qualifying threshold is sensible because, among the thresholds that satisfy the precision requirement, it generally preserves as much recall as possible.

We can inspect the precision and recall associated with this threshold.

```
precision_at_threshold = precisions[threshold_index]
recall_at_threshold = recalls[threshold_index]

print(
    f"Precision at selected threshold: "
    f"{precision_at_threshold:.4f}"
)

print(
    f"Recall at selected threshold: "
    f"{recall_at_threshold:.4f}"
)
```

4.3.4.7 Visualizing the Selected Threshold

The selected operating point can be added to both graphs.

```
def plot_metrics_with_selected_threshold(
    precisions,
    recalls,
    thresholds,
    threshold_index
):
    selected_threshold = thresholds[threshold_index]
    selected_precision = precisions[threshold_index]
    selected_recall = recalls[threshold_index]

    fig, ax = plt.subplots(figsize=(8, 5))

    ax.plot(
        thresholds,
        precisions[:-1],
        linestyle="--",
```

```

        linewidth=2,
        label="Precision"
    )

    ax.plot(
        thresholds,
        recalls[:-1],
        linewidth=2,
        label="Recall"
    )

    ax.axvline(
        selected_threshold,
        linestyle=":",
        linewidth=2,
        label=(
            f"Selected threshold = "
            f"{selected_threshold:.0f}"
        )
    )

    ax.scatter(
        [selected_threshold],
        [selected_precision],
        s=60
    )

    ax.scatter(
        [selected_threshold],
        [selected_recall],
        s=60
    )

    ax.set_xlabel("Decision threshold")
    ax.set_ylabel("Score")
    ax.set_ylim(0, 1.05)
    ax.grid(alpha=0.3)
    ax.legend()

    plt.show()

```

```

plot_metrics_with_selected_threshold(
    precisions,
    recalls,
    thresholds,
    threshold_index
)

```

The corresponding point can also be highlighted on the Precision–Recall curve.

```

def plot_precision_recall_with_selected_point(
    precisions,
    recalls,
    threshold_index
):
    selected_precision = precisions[threshold_index]
    selected_recall = recalls[threshold_index]

    fig, ax = plt.subplots(figsize=(7, 6))

    ax.plot(
        recalls,
        precisions,
        linewidth=2,
        label="Precision-Recall curve"
    )

    ax.scatter(
        selected_recall,
        selected_precision,
        s=80,
        label=(
            f"Precision = {selected_precision:.2f}, "
            f"Recall = {selected_recall:.2f}"
        )
    )

    ax.annotate(
        (
            f"({selected_recall:.2f}, "
            f"{selected_precision:.2f})"
        ),
        xy=(

```



```

        selected_recall,
        selected_precision
    ),
    xytext=(
        max(selected_recall - 0.25, 0.05),
        max(selected_precision - 0.15, 0.05)
    ),
    arrowprops={
        "arrowstyle": "->",
        "linewidth": 1.2
    }
)

ax.set_xlabel("Recall")
ax.set_ylabel("Precision")
ax.set_xlim(0, 1.01)
ax.set_ylim(0, 1.01)
ax.grid(alpha=0.3)
ax.legend()

plt.show()

plot_precision_recall_with_selected_point(
    precisions,
    recalls,
    threshold_index
)

```

4.3.4.8 Generating Predictions With the Selected Threshold

Once the threshold has been selected, class predictions can be obtained by comparing each decision score with that threshold.

```

y_train_pred_90 = (
    y_scores >= threshold_90_precision
)

```

We can then compute the resulting precision and recall.

```

from sklearn.metrics import precision_score, recall_score

precision_90 = precision_score(
    y_train_5,
    y_train_pred_90
)

recall_90 = recall_score(
    y_train_5,
    y_train_pred_90
)

print(f"Precision: {precision_90:.4f}")
print(f"Recall: {recall_90:.4f}")

```

The resulting precision should be at least 90%, subject to the discrete set of scores observed in the cross-validated predictions. The corresponding recall shows the proportion of actual positive observations retained after imposing the stricter classification requirement.

 Do not select and evaluate the threshold on the same data

The preceding analysis is useful for understanding the precision–recall trade-off. However, the reported performance should not be treated as an unbiased estimate of future performance if the same cross-validated scores are used both to select the threshold and to report its final metrics.

A more rigorous workflow is:

1. Use the training or validation data to select the threshold.
2. Fix that threshold.
3. Evaluate the resulting classifier once on an independent test set.

The test set should not influence either model training or threshold selection.

 Threshold selection is a decision problem

A classification threshold should not be selected because it produces a visually attractive graph or because 0.5 or 0 is the software default.

It should reflect:

- the operational cost of false positives;
- the operational cost of false negatives;
- the required minimum precision or recall;

- the prevalence of the positive class;
- the capacity available to review or act on positive predictions.

The threshold is therefore part of the deployed decision system, not merely a technical detail of the classifier.

4.3.5 The ROC Curve

The **Receiver Operating Characteristic**, or **ROC curve**, is another widely used tool for evaluating binary classifiers across different classification thresholds.

Like the Precision–Recall curve, the ROC curve is constructed from the classifier’s decision scores rather than from predictions generated at a single fixed threshold. However, it represents a different trade-off.

The ROC curve plots:

- the **True Positive Rate** on the vertical axis;
- the **False Positive Rate** on the horizontal axis.

4.3.5.1 True Positive Rate

The **True Positive Rate**, abbreviated as **TPR**, is the proportion of actual positive observations that are correctly classified as positive.

It is defined as

$$\text{TPR} = \frac{TP}{TP + FN}.$$

The TPR is equivalent to **recall** and is also known as **sensitivity**.

4.3.5.2 False Positive Rate

The **False Positive Rate**, abbreviated as **FPR**, is the proportion of actual negative observations that are incorrectly classified as positive.

It is defined as

$$\text{FPR} = \frac{FP}{FP + TN}.$$

The complement of the False Positive Rate is the **True Negative Rate**:

$$\text{TNR} = \frac{TN}{TN + FP}.$$

The TNR is also known as **specificity**. Therefore,

$$\text{FPR} = 1 - \text{specificity}.$$

Consequently, the ROC curve can also be described as a plot of

$$\text{Sensitivity} \quad \text{versus} \quad 1 - \text{Specificity}.$$

4.3.5.3 How the ROC Curve Is Constructed

For each possible decision threshold, the classifier generates a different pair of values:

$$(\text{FPR}, \text{TPR}).$$

A very high threshold produces few positive predictions. Both the TPR and FPR will therefore tend to be low.

As the threshold decreases, more observations are classified as positive:

- the classifier detects more true positives, increasing the TPR;
- it also tends to generate more false positives, increasing the FPR.

The ROC curve summarizes all these operating points.

Scikit-Learn computes the corresponding false positive rates, true positive rates, and thresholds using `roc_curve()`.

```
from sklearn.metrics import roc_curve

fpr, tpr, roc_thresholds = roc_curve(
    y_train_5,
    y_scores
)
```

The `y_scores` array may contain either non-thresholded decision values or estimated probabilities for the positive class. In this example, it contains the out-of-fold decision scores generated previously with `cross_val_predict()`. Scikit-Learn's `roc_curve()` is designed for binary classification and returns one FPR and TPR value for each retained decision threshold. [oai_citation:0‡Scikit-learn](#)

4.3.5.4 Plotting the ROC Curve

```
import matplotlib.pyplot as plt

def plot_roc_curve(
    fpr,
    tpr,
    label=None
):
    fig, ax = plt.subplots(figsize=(7, 6))

    ax.plot(
        fpr,
        tpr,
        linewidth=2,
        label=label
    )

    ax.plot(
        [0, 1],
        [0, 1],
        linestyle="--",
        linewidth=1.5,
        label="Random classifier"
    )

    ax.set_xlabel("False Positive Rate")
    ax.set_ylabel("True Positive Rate")
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1.01)
    ax.grid(alpha=0.3)

    if label is not None:
        ax.legend(loc="lower right")

    plt.show()

plot_roc_curve(
    fpr,
    tpr,
```

```
label="SGD classifier"  
)
```

The diagonal line represents the expected behavior of a classifier that ranks observations randomly.

A useful ROC curve rises rapidly toward the upper-left corner. This region represents classifiers that achieve:

- a high True Positive Rate;
- a low False Positive Rate.

The ideal operating point would be

$$(\text{FPR}, \text{TPR}) = (0, 1),$$

meaning that all positive observations are detected without incorrectly classifying any negative observations as positive.

4.3.5.5 Interpreting Points on the ROC Curve

Each point on the ROC curve corresponds to a different classification threshold.

Moving toward the upper-right part of the curve generally means lowering the threshold. This causes the classifier to identify more positive observations, but also to misclassify more negative observations as positive.

Therefore, the ROC curve represents a trade-off between:

- correctly identifying positive observations;
- incorrectly flagging negative observations.

The curve does not determine which threshold should be selected. That choice still depends on the operational consequences of false positives and false negatives.

4.3.5.6 Area Under the ROC Curve

A common way to summarize the ROC curve is through the **Area Under the Curve**, abbreviated as **ROC AUC**.

The ROC AUC produces a single value between 0 and 1:

- an AUC of 1 corresponds to perfect ranking;
- an AUC of approximately 0.5 corresponds to random ranking;
- an AUC below 0.5 indicates that the ranking is systematically worse than random under the specified positive-class orientation.

The ROC AUC for the SGD classifier can be computed as follows.

```
from sklearn.metrics import roc_auc_score

sgd_roc_auc = roc_auc_score(
    y_train_5,
    y_scores
)

print(f"SGD ROC AUC: {sgd_roc_auc:.4f}")
```

For binary classification, `roc_auc_score()` accepts either estimated probabilities for the positive class or non-thresholded decision values, such as those returned by `decision_function()`.
[oai_citation:1](#) [Scikit-learn](#)

4.3.5.7 Probabilistic Interpretation of ROC AUC

The ROC AUC can be interpreted as the probability that the classifier assigns a higher score to a randomly selected positive observation than to a randomly selected negative observation.

Therefore, ROC AUC primarily measures the classifier's **ranking ability**.

This interpretation is important because the ROC AUC does not evaluate the classifier at one specific threshold. Two models can have similar ROC AUC values but behave differently at the operating region that matters for a particular application.

ROC AUC does not select the operating threshold

A high ROC AUC does not guarantee that the classifier will have high precision, recall, or accuracy at the threshold used in practice.

ROC AUC evaluates performance over all thresholds. After comparing models, a threshold must still be selected according to the application's requirements.

4.3.5.8 ROC Curve or Precision–Recall Curve?

The ROC and Precision–Recall curves evaluate different aspects of classifier behavior.

The ROC curve compares:

TPR against FPR.

The Precision–Recall curve compares:

Precision against Recall.

A practical guideline is:

- use the **Precision–Recall curve** when the positive class is rare or when false positives are especially important;
- use the **ROC curve** when both classes are sufficiently represented and the trade-off between sensitivity and specificity is central to the application.

The distinction is especially relevant in imbalanced datasets. The FPR divides the number of false positives by the total number of negative observations:

$$\text{FPR} = \frac{FP}{FP + TN}.$$

When the negative class is very large, a classifier may produce many false positives while still obtaining a relatively small FPR. As a result, the ROC curve can appear optimistic even when the precision of the positive predictions is limited.

In the MNIST binary problem, the positive class consists only of images representing the digit 5, whereas the negative class contains all other digits. Because the negative class is much larger, the Precision–Recall curve provides a particularly informative view of the model’s performance on the positive class.

Practical recommendation

Do not choose between ROC and Precision–Recall curves mechanically.

Use the curve that best reflects the decision problem:

- **ROC:** How much sensitivity can be achieved for a tolerated false positive rate?
- **Precision–Recall:** How reliable are the positive predictions while maintaining an acceptable detection rate?

In strongly imbalanced problems, it is often useful to report both curves, while giving greater attention to the Precision–Recall curve.

4.3.5.9 Comparing the SGD Classifier With a Random Forest

We can now train a `RandomForestClassifier` and compare its ROC curve and ROC AUC with those of the SGD classifier.

Unlike `SGDClassifier`, the random forest provides class probabilities through `predict_proba()`.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_predict

forest_clf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

y_probas_forest = cross_val_predict(
    forest_clf,
    X_train,
    y_train_5,
    cv=3,
    method="predict_proba",
    n_jobs=-1
)
```

For binary classification, `predict_proba()` returns one probability for each class.

```
print(y_probas_forest.shape)
print(y_probas_forest[:,5])
```

Assuming that the positive class is stored in the second column, its estimated probabilities can be extracted as follows:

```
y_scores_forest = y_probas_forest[:, 1]
```

This convention corresponds to the probability of `classes_[1]`, which is the class treated as positive by binary ROC AUC calculations. [oai_citation:2†Scikit-learn](#)

For a reusable workflow, it is preferable to verify the order of the classes after fitting the estimator rather than assuming it silently.

The ROC curve for the random forest can now be computed.

```
fpr_forest, tpr_forest, roc_thresholds_forest = roc_curve(
    y_train_5,
    y_scores_forest
)
```

4.3.5.10 Comparing Both ROC Curves

```
fig, ax = plt.subplots(figsize=(7, 6))

ax.plot(
    fpr,
    tpr,
    linestyle=":",
    linewidth=2,
    label="SGD classifier"
)

ax.plot(
    fpr_forest,
    tpr_forest,
    linewidth=2,
    label="Random Forest"
)

ax.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    linewidth=1.5,
    label="Random classifier"
)

ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_xlim(0, 1)
ax.set_ylim(0, 1.01)
ax.grid(alpha=0.3)
ax.legend(loc="lower right")

plt.show()
```

A curve that lies closer to the upper-left corner generally indicates better ranking performance.

However, the comparison should not rely only on the visual curves. We can compute the ROC AUC of both models.

```
forest_roc_auc = roc_auc_score(
    y_train_5,
    y_scores_forest
)

print(f"SGD ROC AUC: {sgd_roc_auc:.4f}")
print(f"Random Forest ROC AUC: {forest_roc_auc:.4f}")
```

If the Random Forest obtains a higher ROC AUC, this indicates that it ranks positive observations above negative observations more effectively across the complete range of thresholds.

Nevertheless, this does not yet prove that it is the best model for the intended application. The final comparison should also consider:

- the Precision–Recall curve;
- precision and recall at the selected threshold;
- computational cost;
- model interpretability;
- performance on an independent test set.

4.3.5.11 Comparing Precision–Recall Curves

Because the positive class is relatively rare, it is useful to compare the models using the Precision–Recall curve as well.

```
from sklearn.metrics import precision_recall_curve

precisions_forest, recalls_forest, pr_thresholds_forest = (
    precision_recall_curve(
        y_train_5,
        y_scores_forest
    )
)
```

```

fig, ax = plt.subplots(figsize=(7, 6))

ax.plot(
    recalls,
    precisions,
    linestyle=":",
    linewidth=2,
    label="SGD classifier"
)

ax.plot(
    recalls_forest,
    precisions_forest,
    linewidth=2,
    label="Random Forest"
)

ax.set_xlabel("Recall")
ax.set_ylabel("Precision")
ax.set_xlim(0, 1.01)
ax.set_ylim(0, 1.01)
ax.grid(alpha=0.3)
ax.legend(loc="lower left")

plt.show()

```

This second comparison may reveal differences that are less evident in ROC space, particularly in the region where the classifier must maintain high precision while preserving acceptable recall.

i Key takeaway

- The **ROC curve** plots the True Positive Rate against the False Positive Rate.
- The **ROC AUC** summarizes the classifier's ranking ability across all thresholds.
- The **Precision–Recall curve** is often more informative when the positive class is rare.
- Neither ROC AUC nor the visual curve determines the final classification threshold.
- Model comparison should be completed on an independent test set after both model and threshold selection have been finalized.

4.3.6 Multiclass Classification

Binary classification distinguishes between two possible classes. In contrast, **multiclass classification** assigns each observation to exactly one class among three or more alternatives.

In the MNIST problem, for example, the objective is to assign each image to one of ten classes:

$$\{0, 1, 2, \dots, 9\}.$$

This differs from the binary problem studied earlier, where the target only indicated whether an image represented the digit 5.

4.3.6.1 Native Multiclass Models and Reduction Strategies

Classification algorithms do not all approach multiclass problems in the same way.

Some estimators can directly produce predictions among several classes. Examples include:

- decision trees;
- Random Forests;
- naive Bayes classifiers;
- multinomial logistic regression.

Other estimators are fundamentally based on binary decision problems. These models can still be applied to multiclass classification by decomposing the original task into several binary classification problems.

Two of the most common decomposition strategies are:

- **One-versus-Rest**, abbreviated as **OvR**;
- **One-versus-One**, abbreviated as **OvO**.

4.3.6.2 One-versus-Rest

The **One-versus-Rest** strategy trains one binary classifier for each class.

For a problem with K classes, OvR fits

$$K$$

binary classifiers.

Each classifier learns to distinguish one class from all the remaining classes.

For MNIST, the ten binary classification problems are conceptually:

- digit 0 versus all other digits;
- digit 1 versus all other digits;
- digit 2 versus all other digits;
- ...
- digit 9 versus all other digits.

At prediction time, the new observation is evaluated by every binary classifier. The class associated with the highest decision score is selected.

Input image

0 versus rest classifier	score for class 0
1 versus rest classifier	score for class 1
2 versus rest classifier	score for class 2
...	
9 versus rest classifier	score for class 9

Class with highest score

OvR is computationally attractive because it requires only one classifier per class. It is also relatively interpretable because each fitted estimator corresponds to a specific class.

4.3.6.3 One-versus-One

The **One-versus-One** strategy trains one binary classifier for every possible pair of classes.

For K classes, the number of binary classifiers is

$$\frac{K(K-1)}{2}.$$

For MNIST:

$$\frac{10(10-1)}{2} = 45.$$

Therefore, the system must train 45 pairwise classifiers:

- 0 versus 1;
- 0 versus 2;

- 0 versus 3;
- ...
- 8 versus 9.

Each binary classifier is trained only with observations belonging to its two corresponding classes.

At prediction time, all pairwise classifiers vote. The class receiving the greatest support is selected.

Pairwise classifiers		
0 vs 1		
0 vs 2		
0 vs 3		
...	Votes and confidence values	Predicted class
7 vs 9		
8 vs 9		

The main advantage of OvO is that each binary classifier is trained using only a subset of the complete training data. This may be useful for algorithms whose training cost increases rapidly with the number of observations.

Scikit-Learn's `OneVsOneClassifier` fits one estimator per pair of classes and selects the final class from the pairwise votes, complemented by confidence values to resolve ties. [oai_citation:1¶Scikit-learn](#)

i OvR versus OvO

For K classes:

$$\text{OvR classifiers} = K,$$

whereas

$$\text{OvO classifiers} = \frac{K(K-1)}{2}.$$

OvR trains fewer models, but each model generally uses the full training set.

OvO trains more models, but each model uses observations from only two classes.

4.3.6.4 Multiclass Classification With an SVC

Scikit-Learn's `SVC` supports multiclass classification through an internal **One-versus-One** strategy.

For ten MNIST classes, it internally constructs 45 pairwise classification problems. However, by default, its `decision_function()` output is transformed into one score per class to provide an interface consistent with other multiclass classifiers. Internally, the training strategy remains OvO. [oai_citation:2†Scikit-learn](#)

```
from sklearn.svm import SVC

svm_clf = SVC()

svm_clf.fit(
    X_train,
    y_train
)
```

Notice that the target is now `y_train`, which contains the original digit labels from 0 to 9, rather than the binary target `y_train_5`.

The classifier can then predict the class of the previously selected image.

```
svm_prediction = svm_clf.predict(
    [some_digit]
)

svm_prediction
```

4.3.7 Inspecting the Decision Scores

The `decision_function()` method returns one score for each class under the default output configuration.

```
some_digit_scores = svm_clf.decision_function(
    [some_digit]
)

some_digit_scores
```

The output has shape

```
some_digit_scores.shape
```

For a single observation and ten classes, the expected shape is


```
(1, 10)
```

The class associated with the largest score can be found using `argmax()`.

```
import numpy as np

highest_score_index = np.argmax(
    some_digit_scores
)

highest_score_index
```

However, the index of the maximum score is not necessarily the class label itself. The correct class must be retrieved from the classifier's `classes_` attribute.

```
predicted_class = svm_clf.classes_[
    highest_score_index
]

predicted_class
```

The full order of the classes can be inspected directly.

```
svm_clf.classes_
```

Score indices are not class labels

The position of a score in the output of `decision_function()` corresponds to a position in `classes_`, not necessarily to the numerical value of the class.

For MNIST, the coincidence is convenient because the classes are ordered from 0 to 9. In another problem, the labels could be strings or nonconsecutive numbers.

Always map score positions through:

```
classifier.classes_[score_index]
```

4.3.7.1 Inspecting the Original OvO Decision Function

By default, `SVC` returns one transformed score per class. To inspect the original pairwise OvO decision values, the estimator can be configured with:

```
svm_ovo_clf = SVC(
    decision_function_shape="ovo"
)

svm_ovo_clf.fit(
    X_train,
    y_train
)
```

```
ovo_scores = svm_ovo_clf.decision_function(
    [some_digit]
)

ovo_scores.shape
```

With ten classes, the output contains 45 pairwise decision values:

$$\frac{10(10 - 1)}{2} = 45.$$

This distinction is important:

- `decision_function_shape="ovr"` returns one transformed score per class;
- `decision_function_shape="ovo"` returns one value per pair of classes;
- the internal training strategy of `SVC` remains One-versus-One in both cases. [oai_citation:3‡Scikit-learn](#)

4.3.7.2 Explicitly Choosing a Multiclass Strategy

Scikit-Learn provides wrapper estimators that allow a multiclass decomposition strategy to be selected explicitly.

4.3.8 Forcing One-versus-Rest

The following model uses an `SVC` as its base estimator but forces an OvR decomposition.

```

from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(
    SVC()
)

ovr_clf.fit(
    X_train,
    y_train
)

```

```

ovr_clf.predict(
    [some_digit]
)

```

The number of fitted binary estimators can be inspected as follows:

```
len(ovr_clf.estimators_)
```

For ten classes, the result should be:

```
10
```

`OneVsRestClassifier` fits one classifier per class and uses `decision_function()` when available, falling back to `predict_proba()` otherwise. [oai_citation:4†Scikit-learn](#)

4.3.8.1 Forcing One-versus-One

Similarly, an explicit OvO strategy can be created with `OneVsOneClassifier`.

```

from sklearn.multiclass import OneVsOneClassifier

ovo_clf = OneVsOneClassifier(
    SVC()
)

ovo_clf.fit(
    X_train,
    y_train
)

```

```
len(ovo_clf.estimators_)
```

For MNIST, this produces 45 estimators.

💡 Do not wrap estimators unnecessarily

Multiclass wrapper classes are most useful when:

- the base estimator does not directly support the desired strategy;
- the strategy must be controlled explicitly;
- you want to compare OvR and OvO under the same base estimator.

If an estimator already supports the desired multiclass behavior, adding a wrapper may increase computational cost without providing a practical benefit.

4.3.8.2 Multiclass Classification With SGD

An `SGDClassifier` can also be trained using the ten original digit classes.

```
from sklearn.linear_model import SGDClassifier

sgd_multiclass_clf = SGDClassifier(
    random_state=42
)

sgd_multiclass_clf.fit(
    X_train,
    y_train
)
```

```
sgd_multiclass_clf.predict(
    [some_digit]
)
```

For multiclass problems, `SGDClassifier` uses a One-versus-Rest decomposition and fits one binary classifier for each class. Its `decision_function()` returns one score per class. [oai_citation:5†Scikit-learn](#)

```
sgd_digit_scores = (
    sgd_multiclass_clf.decision_function(
        [some_digit]
    )
)

sgd_digit_scores
```

The predicted class should again be obtained through `classes_`.

```
sgd_score_index = np.argmax(
    sgd_digit_scores
)

sgd_predicted_class = (
    sgd_multiclass_clf.classes_[
        sgd_score_index
    ]
)

sgd_predicted_class
```

4.3.8.3 Evaluating Multiclass Accuracy

A first estimate of model performance can be obtained with cross-validation.

```
from sklearn.model_selection import cross_val_score

sgd_accuracy_scores = cross_val_score(
    sgd_multiclass_clf,
    X_train,
    y_train,
    cv=3,
    scoring="accuracy",
    n_jobs=-1
)

sgd_accuracy_scores
```

The mean cross-validated accuracy can be reported as follows.

```
print(
    "Mean cross-validated accuracy: "
    f"{sgd_accuracy_scores.mean():.4f}"
)
```

Because MNIST contains ten classes, a uniform random prediction mechanism would have an expected accuracy close to

$$\frac{1}{10} = 0.10.$$

However, this comparison should only be interpreted as a basic reference. A stronger baseline would account for the actual distribution of the classes, for example by always predicting the most frequent class.

Accuracy is also insufficient for understanding which digits the classifier confuses. For that reason, the analysis will later include a multiclass confusion matrix and class-specific metrics.

4.3.8.4 Scaling the Features Correctly

Linear models trained with stochastic gradient descent are sensitive to differences in feature scale. Standardizing the pixel variables may therefore improve optimization and classification performance.

However, scaling must be performed **inside each cross-validation fold**. Fitting a scaler once on the complete training set before cross-validation introduces information from the validation folds into the preprocessing step.

For this reason, the scaler and classifier should be combined in a **Pipeline**.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

scaled_sgd_clf = make_pipeline(
    StandardScaler(),
    SGDClassifier(
        random_state=42
    )
)
```

The complete pipeline is then evaluated with cross-validation.

```
scaled_sgd_accuracy_scores = cross_val_score(
    scaled_sgd_clf,
    X_train.astype("float64"),
    y_train,
    cv=3,
    scoring="accuracy",
    n_jobs=-1
)

scaled_sgd_accuracy_scores
```

```
print(
    "Mean accuracy without scaling: "
    f"{sgd_accuracy_scores.mean():.4f}"
)

print(
    "Mean accuracy with scaling: "
    f"{scaled_sgd_accuracy_scores.mean():.4f}"
)
```

! Preprocessing belongs inside the pipeline

This approach is preferable to:

```
X_train_scaled = scaler.fit_transform(X_train)
cross_val_score(model, X_train_scaled, y_train, cv=3)
```

because that code fits the scaler using all training observations before constructing the validation folds.

A pipeline ensures that, within each fold:

1. the scaler is fitted only on the training portion;
2. the same fitted transformation is applied to the validation portion;
3. the classifier is fitted using only properly transformed training data.

4.3.8.5 Evaluating Multiclass Predictions

The overall accuracy provides a useful summary of classifier performance, but it does not indicate **which classes are being confused with one another**.

For multiclass problems, one of the most informative evaluation tools is the **confusion matrix**. Unlike the binary case, where the matrix contains only four possible outcomes, a multiclass confusion matrix has one row and one column for each class.

For the MNIST dataset, the confusion matrix has size 10×10 , where:

- each **row** represents the actual digit;
- each **column** represents the predicted digit;
- the **main diagonal** contains the correctly classified images;
- the **off-diagonal entries** represent classification errors.

To construct the confusion matrix, we first generate cross-validated predictions for every observation in the training set.

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(
    scaled_sgd_clf,
    X_train.astype(np.float64),
    y_train,
    cv=3,
    n_jobs=-1
)
```

Next, we compute and visualize the confusion matrix.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

conf_mx = confusion_matrix(y_train, y_train_pred)

fig, ax = plt.subplots(figsize=(8, 7))

ConfusionMatrixDisplay(
    confusion_matrix=conf_mx
).plot(
    cmap="Blues",
    colorbar=True,
    ax=ax
)

ax.set_title("Multiclass Confusion Matrix")

plt.show()
```


The confusion matrix immediately reveals whether the classifier performs uniformly across all classes or systematically confuses certain digits. However, because the diagonal entries are usually much larger than the off-diagonal ones, it can be difficult to identify the most important error patterns.

In the next section, we normalize the confusion matrix and focus specifically on the classifier's errors.

4.3.9 Error Analysis

A single accuracy value summarizes the overall proportion of correct predictions, but it does not reveal which classes the model handles well or which classes it systematically confuses.

For multiclass problems, a useful error-analysis workflow includes:

1. generating out-of-fold predictions;
2. constructing the confusion matrix;
3. normalizing the matrix by the actual class;
4. examining the largest off-diagonal values;
5. reviewing precision, recall, and F1-score for each class;
6. inspecting representative misclassified observations.

4.3.10 Cross-validated Predictions

To construct the confusion matrix without evaluating the model on observations used to fit the corresponding estimator, we generate out-of-fold predictions with `cross_val_predict()`.

The preprocessing pipeline must be passed as a whole so that scaling is fitted separately inside each fold.

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(
    scaled_sgd_clf,
    X_train.astype("float64"),
    y_train,
    cv=3,
    method="predict",
    n_jobs=-1
)
```

Each prediction is produced by a model that did not use that observation during fitting.

4.3.10.1 Multiclass Confusion Matrix

```
from sklearn.metrics import confusion_matrix

conf_mx = confusion_matrix(
    y_train,
    y_train_pred,
    labels=sgd_multiclass_clf.classes_
)

conf_mx
```

In a multiclass confusion matrix:

- rows represent the actual classes;
- columns represent the predicted classes;
- diagonal cells contain correct classifications;
- off-diagonal cells contain errors.

A raw confusion matrix can be visualized using Scikit-Learn's display utility.

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

fig, ax = plt.subplots(figsize=(8, 7))

ConfusionMatrixDisplay.from_predictions(
    y_train,
    y_train_pred,
    labels=np.arange(10),
    cmap="gray_r",
    colorbar=True,
    ax=ax
)

ax.set_title(
    "Confusion Matrix: SGD Classifier"
)

plt.show()
```

The diagonal values are usually much larger than the off-diagonal values because they represent correct predictions. However, this can make smaller but important error patterns difficult to see.

4.3.10.2 Normalized Confusion Matrix

To compare error proportions across classes, the confusion matrix can be normalized by the actual class.

```
fig, ax = plt.subplots(figsize=(8, 7))

ConfusionMatrixDisplay.from_predictions(
    y_train,
    y_train_pred,
    labels=np.arange(10),
    normalize="true",
    values_format=".2f",
    cmap="gray_r",
    colorbar=True,
    ax=ax
)

ax.set_title(
    "Normalized Confusion Matrix"
)

plt.show()
```

With `normalize="true"`, every row sums approximately to one. Each cell therefore represents the proportion of observations from an actual class assigned to a predicted class.

For example, the cell in row 5 and column 3 represents:

the proportion of actual digit 5 images classified as digit 3.

This normalized matrix is more informative than manually dividing the matrix rows and then suppressing its diagonal.

4.3.11 Focusing Only on Errors

If the objective is to emphasize misclassification patterns, the diagonal can be set to zero after normalization.

```

normalized_conf_mx = confusion_matrix(
    y_train,
    y_train_pred,
    labels=np.arange(10),
    normalize="true"
)

error_conf_mx = normalized_conf_mx.copy()

np.fill_diagonal(
    error_conf_mx,
    0
)

```

```

fig, ax = plt.subplots(figsize=(8, 7))

image = ax.imshow(
    error_conf_mx,
    cmap="gray_r"
)

ax.set_xlabel("Predicted class")
ax.set_ylabel("Actual class")
ax.set_xticks(np.arange(10))
ax.set_yticks(np.arange(10))
ax.set_title(
    "Normalized Classification Errors"
)

fig.colorbar(
    image,
    ax=ax
)

plt.show()

```

The brightest off-diagonal cells identify the class pairs that deserve further examination.

i Two different normalization questions

Normalizing by rows answers:

Among observations that truly belong to this class, where does the model send them?

Normalizing by columns answers:

Among observations predicted as this class, where did they actually come from?

The first perspective is closely related to class-specific recall. The second is closely related to class-specific precision.

4.3.11.1 Identifying the Most Frequent Confusions

Instead of relying only on visual inspection, the strongest off-diagonal errors can be extracted numerically.

```
confusion_pairs = []

for actual_class in range(10):
    for predicted_class in range(10):
        if actual_class != predicted_class:
            confusion_pairs.append(
                (
                    actual_class,
                    predicted_class,
                    error_conf_mx[
                        actual_class,
                        predicted_class
                    ]
                )
            )

confusion_pairs = sorted(
    confusion_pairs,
    key=lambda item: item[2],
    reverse=True
)
```

```
confusion_pairs[:10]
```

A more readable representation can be created as a table.

```
import pandas as pd

largest_confusions = pd.DataFrame(
    confusion_pairs[:10],
    columns=[
        "Actual class",
        "Predicted class",
        "Error proportion"
    ]
)

largest_confusions
```

This table provides a direct starting point for inspecting the classes that the classifier has the greatest difficulty separating.

4.3.11.2 Class-specific Evaluation Metrics

A multiclass classification report provides precision, recall, and F1-score for every class.

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_train,
        y_train_pred,
        digits=4
    )
)
```

For each digit:

- **precision** measures how reliable the predictions of that digit are;
- **recall** measures how many actual images of that digit were detected;
- **F1-score** balances precision and recall;
- **support** reports the number of actual observations in that class.

The report also includes several averages.

4.3.11.3 Macro Average

The macro average computes the metric independently for each class and then takes the unweighted mean:

$$\text{MacroMetric} = \frac{1}{K} \sum_{k=1}^K \text{Metric}_k .$$

Each class contributes equally, regardless of its frequency.

4.3.11.4 Weighted Average

The weighted average weights each class-specific metric according to the number of observations in that class:

$$\text{WeightedMetric} = \sum_{k=1}^K \frac{n_k}{n} \text{Metric}_k .$$

Classes with more observations contribute more heavily.

4.3.12 Accuracy

Accuracy measures the overall fraction of correct predictions:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} .$$

In a single-label multiclass problem, micro-averaged precision, recall, and F1 coincide with accuracy, which is why the standard classification report does not always display a separate micro-average row.

Accuracy can conceal class-specific weaknesses

Two classifiers can have similar accuracy while making very different types of errors. Always inspect:

- the normalized confusion matrix;
- class-specific precision and recall;
- macro-averaged metrics;
- the observations associated with the most important confusions.

4.4 A Second Model: Logistic Regression

A multinomial logistic regression model can also be evaluated using the same cross-validation workflow.

```
from sklearn.linear_model import LogisticRegression

logistic_clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        solver="lbfgs",
        max_iter=1000
    )
)
```

```
y_pred_logistic = cross_val_predict(
    logistic_clf,
    X_train.astype("float64"),
    y_train,
    cv=3,
    method="predict",
    n_jobs=-1
)
```

```
print(
    classification_report(
        y_train,
        y_pred_logistic,
        digits=4
    )
)
```

The comparison should use the same folds, preprocessing logic, and evaluation metrics for both models. A higher overall accuracy does not automatically imply superior performance for every digit, so the corresponding confusion matrix should also be reviewed.

5 Chapter – Training Models

5.1 Logistic Regression

Logistic Regression, also called *Logit Regression*, is commonly used to estimate the probability that an instance belongs to a particular class. For example, it can estimate the probability that an email is spam.

If the estimated probability is greater than or equal to 50%, the model predicts that the instance belongs to the **positive class**, usually labeled 1. Otherwise, it predicts that the instance belongs to the **negative class**, usually labeled 0.

Despite its name, Logistic Regression is therefore primarily used as a **binary classification algorithm**.

5.1.1 Estimating Probabilities

Logistic Regression begins by computing a weighted sum of the input features, including a bias term, just as Linear Regression does. However, instead of returning this score directly, it transforms it using the **logistic function**:

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \theta).$$

The logistic function, denoted by $\sigma(\cdot)$, is a sigmoid-shaped function defined as

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

It transforms any real-valued score into a number between 0 and 1, allowing the output to be interpreted as a probability.

Once the model estimates the probability

$$\hat{p} = h_{\theta}(\mathbf{x})$$

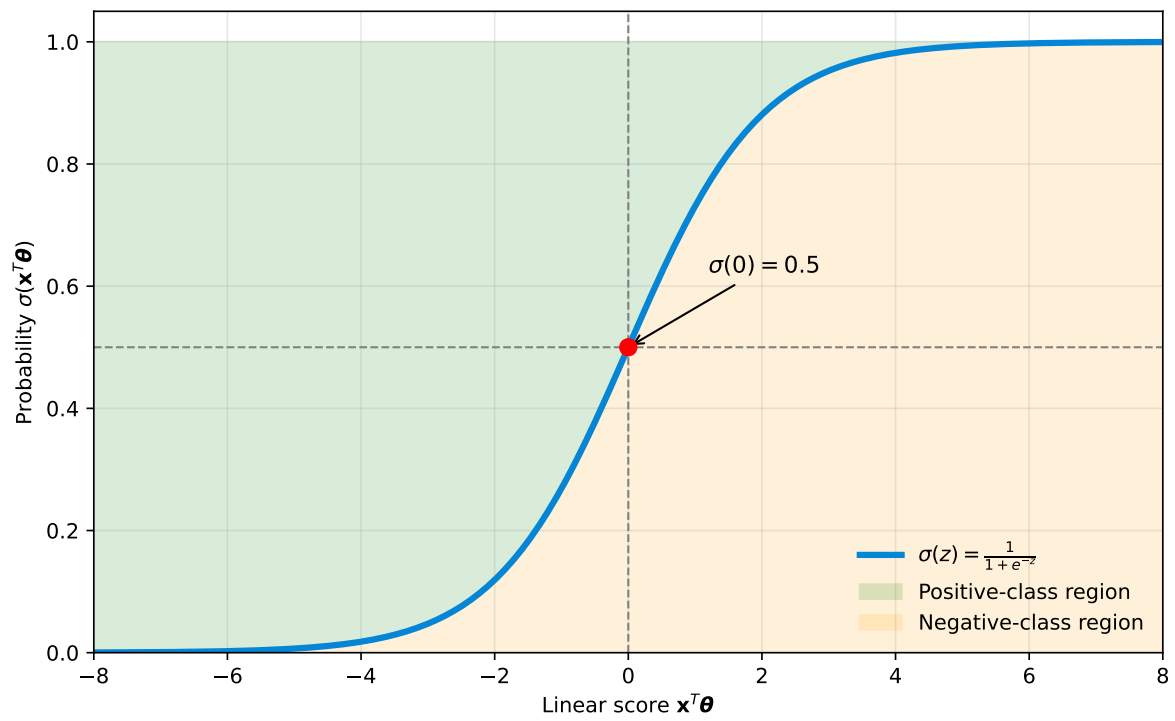


Figure 5.1: The logistic, or sigmoid, function.

that an instance belongs to the positive class, it converts that probability into a class prediction using a threshold:

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5, \\ 1, & \text{if } \hat{p} \geq 0.5. \end{cases}$$

The value 0.5 is the default classification threshold, although a different threshold may be selected depending on the objectives and consequences of the classification problem.

5.1.2 Training Logistic Regression

Training Logistic Regression consists of finding the parameter vector θ that produces appropriate probabilities for the observed classes.

The objective is:

- For positive instances, where $y = 1$, the predicted probability $\hat{p}$ should be close to 1.
- For negative instances, where $y = 0$, the predicted probability $\hat{p}$ should be close to 0.

To achieve this objective, Logistic Regression minimizes a cost function known as **log loss** or **binary cross-entropy**.

Cost Function for a Single Training Instance

For a single observation, the cost is

$$c(\theta) = \begin{cases} -\log(\hat{p}), & \text{if } y = 1, \\ -\log(1 - \hat{p}), & \text{if } y = 0. \end{cases}$$

This definition has an intuitive interpretation:

- Correct and confident predictions produce a cost close to 0.
- Incorrect predictions receive a larger penalty.
- Incorrect predictions made with high confidence receive an especially large penalty.

Why Use the Logarithm?

The logarithmic function offers two important advantages:

- It strongly penalizes confident errors.
- It contributes to a convex objective function that can be optimized efficiently using gradient-based methods.

Cost Function for the Entire Training Set

For a dataset containing m training instances, the average cost is

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})].$$

Training therefore consists of solving the optimization problem

$$\min_{\theta} J(\theta).$$

The Logistic Regression cost function is convex. Consequently, gradient-based optimization does not become trapped in suboptimal local minima. Under appropriate optimization conditions, the algorithm converges toward a global minimizer.

The minimizer is not necessarily unique in every possible dataset—for example, redundant or perfectly collinear predictors may produce equivalent parameterizations—but the convexity of the objective remains an important computational advantage.

Gradient of the Cost Function

The partial derivative of the cost function with respect to parameter θ_j is

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m [\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}] x_j^{(i)}.$$

For each training instance, the optimization algorithm:

1. Computes the predicted probability.
2. Calculates the difference between the prediction and the target.
3. Multiplies this difference by the corresponding feature value.
4. Averages the result across the training instances.

The resulting gradient indicates how the parameters should change to reduce the cost.

Batch Gradient Descent uses all training observations at each optimization step. Stochastic Gradient Descent uses one observation at a time, while Mini-batch Gradient Descent uses a small group of observations.

Key Takeaways

- Logistic Regression estimates class probabilities before producing class labels.
- The logistic function maps a linear score to the interval $(0, 1)$.
- Log loss strongly penalizes confident incorrect predictions.

- The cost function is convex, so optimization is not affected by suboptimal local minima.
- Gradient-based algorithms can be used to estimate the model parameters efficiently.

5.1.3 Decision Boundaries

The Iris dataset provides a simple example of Logistic Regression. It contains sepal and petal measurements for 150 flowers belonging to three species:

- *Iris setosa*
- *Iris versicolor*
- *Iris virginica*



Figure 5.2: Flowers from the three Iris species.

The following block imports the required tools and loads the dataset as a pandas DataFrame. Loading it with `as_frame=True` allows the predictors to be selected using their descriptive column names.

```
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

iris = datasets.load_iris(as_frame=True)

list(iris.keys())
```

The `load_iris()` function returns a dictionary-like Bunch object containing:

Listing 5.1

```
['data',  
 'target',  
 'frame',  
 'target_names',  
 'DESCR',  
 'feature_names',  
 'filename',  
 'data_module']
```

- the predictor matrix;
- the target variable;
- feature names;
- class names;
- metadata describing the dataset.

5.1.3.1 Binary Classification Using One Feature

Suppose the objective is to identify *Iris virginica* flowers using only petal width.

The original three-class target is transformed into a binary target:

- **False**: the flower is not *Iris virginica*;
- **True**: the flower is *Iris virginica*.

The data is divided into training and test sets before fitting the model. The training set is used to estimate the parameters, while the test set is kept separate for later model evaluation.

Stratification preserves approximately the same class proportions in both subsets.

The Logistic Regression model is then fitted exclusively on the training data.

```
log_reg = LogisticRegression(random_state=42)  
log_reg.fit(X_train, y_train)
```

Once trained, the model can estimate class probabilities for any possible petal width.

The following code creates a sequence of petal-width values, obtains the corresponding probabilities, and identifies the point at which the probability of *Iris virginica* reaches 0.5.

Listing 5.2

```
X = iris.data[["petal width (cm)"]].values
y = iris.target == 2

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

Listing 5.3

```
LogisticRegression(random_state=42)
```

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)

y_proba = log_reg.predict_proba(X_new)

decision_boundary = X_new[
    y_proba[:, 1] >= 0.5
][0, 0]

plt.figure(figsize=(7, 3.5))

plt.plot(
    X_new,
    y_proba[:, 0],
    "b--",
    linewidth=2,
    label="Not Iris virginica"
)

plt.plot(
    X_new,
    y_proba[:, 1],
    "g-",
    linewidth=2,
```

```

    label="Iris virginica"
)

plt.plot(
    [decision_boundary, decision_boundary],
    [0, 1],
    "k:",
    linewidth=2,
    label="Decision boundary"
)

plt.arrow(
    x=decision_boundary,
    y=0.08,
    dx=-0.3,
    dy=0,
    head_width=0.05,
    head_length=0.1,
    fc="b",
    ec="b"
)

plt.arrow(
    x=decision_boundary,
    y=0.92,
    dx=0.3,
    dy=0,
    head_width=0.05,
    head_length=0.1,
    fc="g",
    ec="g"
)

plt.plot(
    X_train[y_train == 0],
    y_train[y_train == 0],
    "bs",
    label="Training: not virginica"
)

plt.plot(
    X_train[y_train == 1],

```



```

y_train[y_train == 1],
"g^",
label="Training: virginica"
)

plt.xlabel("Petal width (cm)")
plt.ylabel("Estimated probability")
plt.legend(loc="center left")
plt.axis([0, 3, -0.02, 1.02])
plt.grid(alpha=0.3)
plt.tight_layout()

plt.show()

```

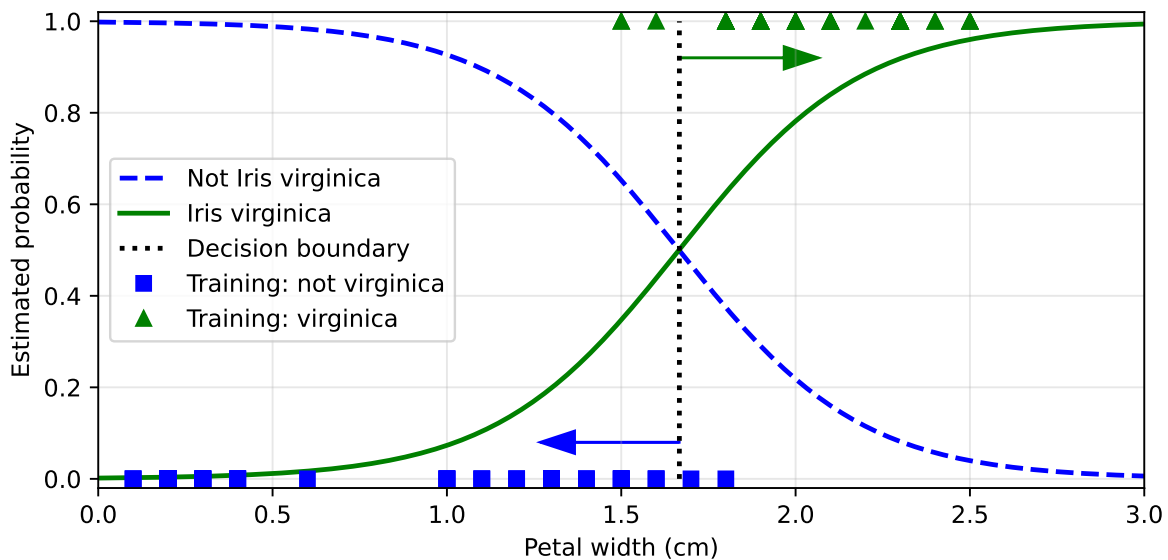


Figure 5.3: Estimated probabilities and decision boundary for *Iris virginica* classification.

The two curves represent the estimated probabilities of the two classes. For every observation, these probabilities sum to 1.

As petal width increases, the estimated probability of *Iris virginica* also increases. The vertical dotted line marks the decision boundary, where both probabilities are approximately 0.5.

Therefore:

- Petal widths below the boundary are classified as **Not Iris virginica**.

- Petal widths above the boundary are classified as **Iris virginica**.

The squares and triangles represent the training observations used to estimate the model.

The numerical value of the decision boundary can be inspected directly:

```
decision_boundary
```

Listing 5.4

```
np.float64(1.6666666666666667)
```

The exact value may vary slightly depending on the training subset. This is expected because the model is fitted using only the observations assigned to the training set.

Once the model has been trained, new flowers can be classified using the `predict()` method.

```
log_reg.predict([[1.7], [1.5]])
```

Listing 5.5

```
array([ True, False])
```

The first observation has a petal width of 1.7 cm, while the second has a petal width of 1.5 cm.

Their predictions depend on which side of the learned decision boundary they fall. Instead of assuming a fixed boundary value in the explanation, the predictions should be interpreted relative to the boundary estimated in the preceding block.

The underlying probabilities can be obtained using `predict_proba()`:

```
log_reg.predict_proba([[1.7], [1.5]])
```

Listing 5.6

```
array([[0.46703281, 0.53296719],  
       [0.65270529, 0.34729471]])
```

Each row contains the probabilities assigned to the two classes:

- The first column corresponds to **Not Iris virginica**.
- The second column corresponds to **Iris virginica**.

The `predict()` method returns the class with the highest estimated probability.

5.1.3.2 Logistic Regression Using Two Features

The previous model uses only petal width. Although this simplifies visualization, it ignores potentially useful information contained in other predictors.

The following example uses both petal length and petal width. In a two-dimensional feature space, the decision boundary is no longer a single threshold. Instead, it becomes a line separating two regions of the plane.

A new training-test split is created because the predictor matrix has changed.

Listing 5.7

```
X = iris.data[
    ["petal length (cm)", "petal width (cm)"]
].values

y = iris.target == 2

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

The model is then fitted using the two predictors.

```
log_reg_2d = LogisticRegression(
    C=2,
    random_state=42
)

log_reg_2d.fit(X_train, y_train)
```

The parameter `C` controls the inverse strength of regularization. Larger values of `C` correspond to weaker regularization, while smaller values impose stronger regularization.

Listing 5.8

```
LogisticRegression(C=2, random_state=42)
```

```
x0, x1 = np.meshgrid(
    np.linspace(2.9, 7, 500),
    np.linspace(0.8, 2.7, 200)
)

X_new = np.c_[x0.ravel(), x1.ravel()]

y_proba = log_reg_2d.predict_proba(X_new)
zz = y_proba[:, 1].reshape(x0.shape)

left_right = np.array([2.9, 7])

boundary = -(
    log_reg_2d.coef_[0, 0] * left_right
    + log_reg_2d.intercept_[0]
) / log_reg_2d.coef_[0, 1]

plt.figure(figsize=(7, 4))

plt.plot(
    X_train[y_train == 0, 0],
    X_train[y_train == 0, 1],
    "bs",
    label="Not Iris virginica"
)

plt.plot(
    X_train[y_train == 1, 0],
    X_train[y_train == 1, 1],
    "g^",
    label="Iris virginica"
)

contour = plt.contour(
    x0,
    x1,
    zz,
```

```

    cmap=plt.cm.brg
)

plt.clabel(
    contour,
    inline=True,
    fontsize=8
)

plt.plot(
    left_right,
    boundary,
    "k--",
    linewidth=2,
    label="Decision boundary"
)

plt.text(
    3.5,
    1.27,
    "Not Iris virginica",
    color="navy",
    ha="center"
)

plt.text(
    6.4,
    2.3,
    "Iris virginica",
    color="darkgreen",
    ha="center"
)

plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.axis([2.9, 7, 0.8, 2.7])
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()

plt.show()

```

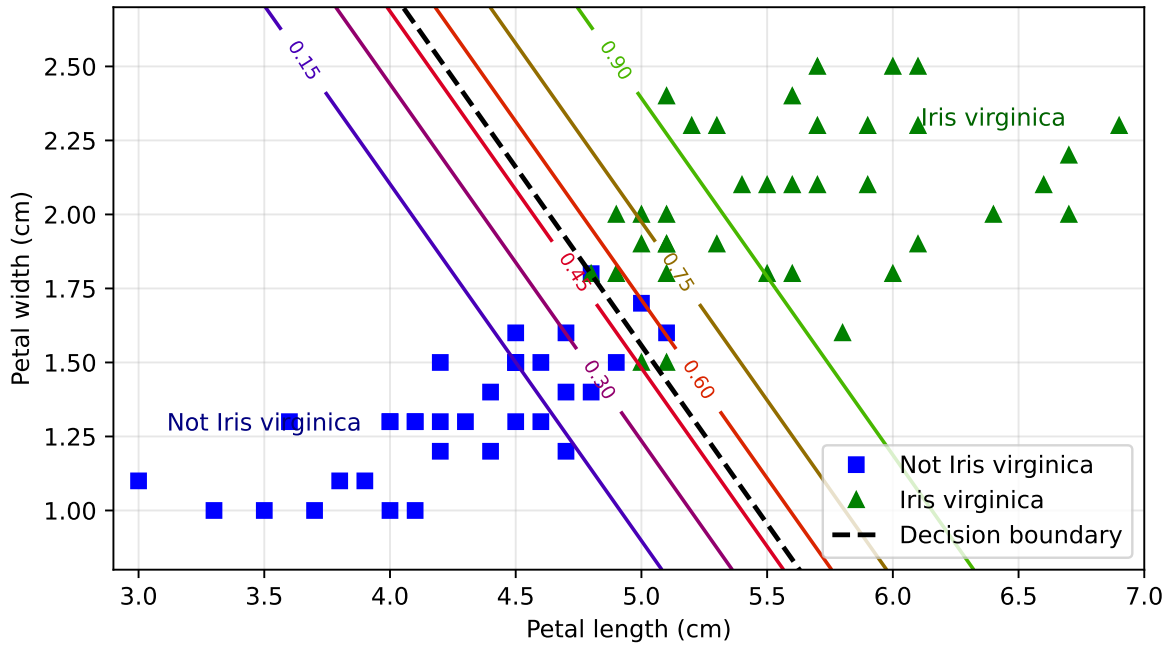


Figure 5.4: Decision boundary and estimated probabilities using petal length and petal width.

As shown in Figure 8.3, each point represents a training observation.

The dashed line is the linear decision boundary. It separates the feature space into two regions:

- Flowers on one side are classified as **Not Iris virginica**.
- Flowers on the other side are classified as **Iris virginica**.

The contour lines represent different estimated probabilities of belonging to the *Iris virginica* class.

Near the decision boundary, the predicted probability is close to 0.5, which indicates greater uncertainty. As an observation moves farther from the boundary, the model assigns increasingly extreme probabilities to one of the two classes.

5.1.4 Softmax Regression

Logistic Regression can be generalized to support more than two classes directly. This generalization is known as **Softmax Regression** or **Multinomial Logistic Regression**.

Given an instance $\mathbf{x}$, the model first calculates a score for each class k :

$$s_k(\mathbf{x}) = \mathbf{x}^T \theta^{(k)}.$$

Each class has its own parameter vector $\theta^{(k)}$. These class-specific vectors can be stored together in a parameter matrix .

Once the score for each class has been computed, the Softmax function transforms the scores into class probabilities:

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}.$$

In this expression:

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is the vector containing all class scores.
- $\hat{p}_k$ is the estimated probability that instance $\mathbf{x}$ belongs to class k .

The Softmax probabilities satisfy

$$\sum_{k=1}^K \hat{p}_k = 1.$$

The predicted class is the one with the highest estimated probability:

$$\hat{y} = \arg \max_k \hat{p}_k = \arg \max_k s_k(\mathbf{x}) = \arg \max_k \left[\left(\theta^{(k)} \right)^T \mathbf{x} \right].$$

The arg max operator returns the value of k that maximizes the corresponding expression. In this case, it returns the class with the highest score and therefore the highest estimated probability.

💡 Softmax Regression Predicts One Class

Softmax Regression predicts **one class per observation**. It is a multiclass classifier, not a multilabel or multioutput classifier.

It is therefore appropriate when the classes are mutually exclusive, such as different plant species or handwritten digits.

For example, a Softmax model can classify a flower as *Iris setosa*, *Iris versicolor*, or *Iris virginica*, but it cannot assign several of these species to the same flower.

Similarly, it is not directly suitable for a problem in which multiple labels may be correct simultaneously, such as identifying several people present in the same photograph.

5.1.4.1 Training Softmax Regression

Training Softmax Regression consists of finding a parameter matrix that assigns high probability to the correct class and low probabilities to the remaining classes.

The model minimizes the **categorical cross-entropy** cost function:

$$J() = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)}).$$

In this equation:

- m is the number of training instances.
- K is the number of classes.
- $y_k^{(i)} = 1$ when instance i belongs to class k .
- $y_k^{(i)} = 0$ when instance i does not belong to class k .
- $\hat{p}_k^{(i)}$ is the probability assigned by the model to class k for instance i .

Because the target classes are represented using one-hot encoding, only the logarithm corresponding to the correct class contributes to the cost for each observation.

When there are only two classes, this expression reduces to the binary cross-entropy cost used by binary Logistic Regression.

The gradient with respect to the parameter vector of class k is

$$\nabla_{\theta^{(k)}} J() = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}.$$

This gradient indicates how the parameters associated with each class should change to reduce the total cost.

An optimization algorithm computes the gradient for every class and iteratively updates the complete parameter matrix .

Why Cross-Entropy?

Cross-entropy penalizes a classifier when it assigns a low probability to the correct class. The penalty grows rapidly as the probability assigned to the correct class approaches zero. Consequently, the optimization process encourages the model to assign increasingly high probabilities to the observed target classes.

5.1.4.2 Multiclass Classification of Iris Flowers

The following example trains a Softmax Regression model to classify all three Iris species using petal length and petal width.

As in the binary examples, the observations are divided into training and test sets before fitting the model.

Listing 5.9

```
X = iris.data[
    ["petal length (cm)", "petal width (cm)"]
].values

y = iris.target.values

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

A compatible solver such as "lbfgs" can optimize the multinomial Logistic Regression objective directly.

In current Scikit-Learn versions, it is not necessary to specify the deprecated `multi_class="multinomial"` argument when using a compatible solver for a multiclass target.

```
softmax_reg = LogisticRegression(
    solver="lbfgs",
    C=10,
    random_state=42
)

softmax_reg.fit(X_train, y_train)
```

Listing 5.10

```
LogisticRegression(C=10, random_state=42)
```

Unlike binary Logistic Regression, Softmax Regression learns one set of parameters for each class.

The learned intercepts and coefficients can be inspected directly:

Listing 5.11

```
print("Intercepts:")
print(softmax_reg.intercept_)

print("\nCoefficients:")
print(softmax_reg.coef_)
```

```
Intercepts:
[ 18.56219292   6.93248241 -25.49467533]

Coefficients:
[[-4.41240489 -2.19019064]
 [-0.15153592 -1.72265407]
 [ 4.5639408   3.9128447  ]]
```

The coefficient matrix contains one row per class and one column per predictor.

In this example:

- each row corresponds to one Iris species;
- the first column corresponds to petal length;
- the second column corresponds to petal width.

The model can then estimate the probabilities of all three species for a new observation:

```
softmax_reg.predict_proba([[5.0, 2.0]])
```

Listing 5.12

```
array([[2.01272527e-06, 8.15683946e-02, 9.18429593e-01]])
```

The resulting row contains one probability for each class, following the order stored in

```
softmax_reg.classes_
```

The final class prediction is the class with the highest estimated probability:

Listing 5.13

```
array([0, 1, 2])
```

```
print(softmax_reg.predict_proba([[5.0, 2.0]]))
prediction = softmax_reg.predict([[5.0, 2.0]])

iris.target_names[prediction]
```

```
[[2.01272527e-06 8.15683946e-02 9.18429593e-01]]
```

Listing 5.14

```
array(['virginica'], dtype='<U10')
```

Although `X_test` and `y_test` are not used in this section, they remain isolated from the training process. They can therefore be used later to study model generalization without introducing information from the test data during parameter estimation.

```
from matplotlib.colors import ListedColormap

custom_cmap = ListedColormap(
    ["#fafab0", "#9898ff", "#a0faa0"]
)

x0, x1 = np.meshgrid(
    np.linspace(0, 8, 500),
    np.linspace(0, 3.5, 200)
)

X_new = np.c_[x0.ravel(), x1.ravel()]

y_proba = softmax_reg.predict_proba(X_new)
y_predict = softmax_reg.predict(X_new)

zz = y_predict.reshape(x0.shape)
zz1 = y_proba[:, 1].reshape(x0.shape)

plt.figure(figsize=(8,4))
```

```

plt.plot(
    X_train[y_train == 2,0],
    X_train[y_train == 2,1],
    "g^",
    label="Iris virginica"
)

plt.plot(
    X_train[y_train == 1,0],
    X_train[y_train == 1,1],
    "bs",
    label="Iris versicolor"
)

plt.plot(
    X_train[y_train == 0,0],
    X_train[y_train == 0,1],
    "yo",
    label="Iris setosa"
)

plt.contourf(
    x0,
    x1,
    zz,
    cmap=custom_cmap,
    alpha=0.35
)

contour = plt.contour(
    x0,
    x1,
    zz1,
    cmap="hot"
)

plt.clabel(
    contour,
    inline=True,
    fontsize=8
)

```

```
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")

plt.legend(loc="upper left")

plt.axis([0.5,7,0,3.5])

plt.grid(alpha=0.3)

plt.tight_layout()

plt.show()
```

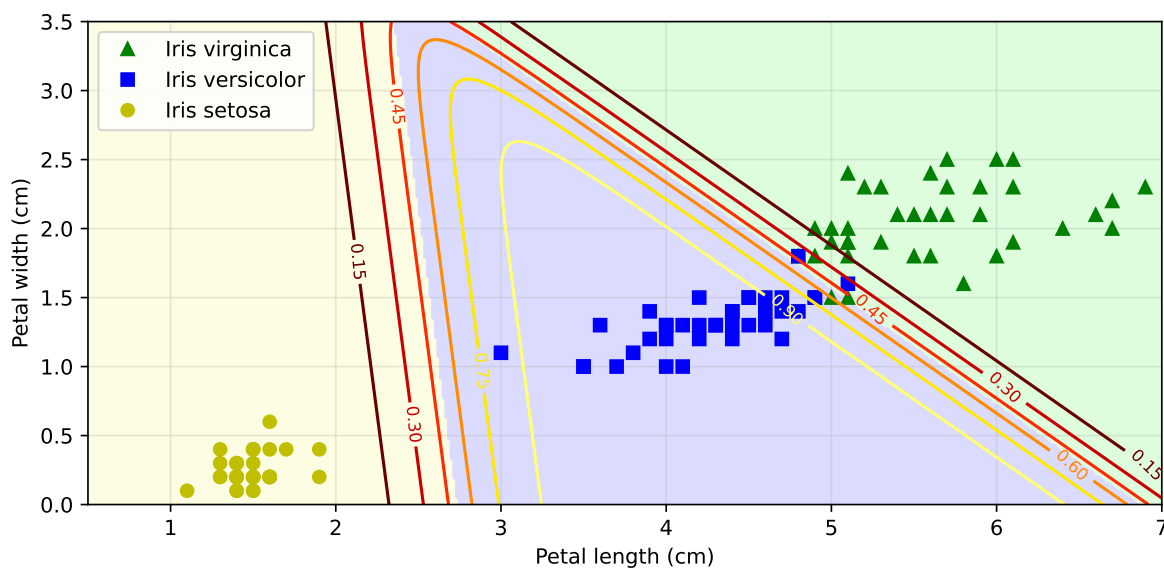


Figure 5.5: Decision regions and estimated probabilities obtained with Softmax Regression.

Figure 8.4 illustrates how Softmax Regression partitions the feature space into three decision regions, one for each Iris species.

Each background color represents the class predicted by the classifier, while the contour lines indicate the estimated probability of the Iris versicolor class. Every point in the feature space belongs to exactly one decision region, and the predicted class is the one with the highest estimated probability.

Notice that the decision boundaries are still linear, but together they divide the predictor space into multiple regions corresponding to the three flower species. This is the multiclass extension of the linear decision boundary introduced earlier for binary Logistic Regression.

Logistic Regression and Softmax Regression are among the most widely used classification algorithms because they combine a solid probabilistic foundation with efficient optimization. Despite their simplicity, they often provide highly competitive performance and serve as the basis for many more advanced classification methods introduced in later chapters.

6 Chapter – Support Vector Machines

6.1 Introduction

Support Vector Machines (SVMs) are supervised learning models that can be used for classification, regression, and outlier detection. Their central idea is geometric: instead of merely finding a decision boundary that separates classes, an SVM seeks a boundary with the **largest possible margin** between the classes.

SVMs are especially effective for small- and medium-sized datasets with complex decision boundaries. They can model both linear and nonlinear relationships, but they are sensitive to feature scaling and their performance depends strongly on the choice of hyperparameters.

Learning objectives

After completing this chapter, you should be able to:

- explain the concepts of separating hyperplane, margin, and support vectors;
- distinguish hard-margin and soft-margin classification;
- interpret the hyperparameter C ;
- explain why feature scaling is essential for SVMs;
- use linear, polynomial, and radial basis function kernels;
- interpret the role of γ in an RBF kernel;
- fit SVM classifiers and regressors with Scikit-Learn;
- diagnose underfitting and overfitting in an SVM model.

6.2 Linear SVM classification

Consider a binary classification problem with a feature vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}.$$

A linear classifier uses a decision function of the form

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b,$$

where $\mathbf{w}$ contains the model coefficients and b is the intercept.

The decision boundary is the hyperplane

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

For binary labels $y \in \{-1, +1\}$, the predicted class is determined by the sign of the decision function:

$$\hat{y} = \begin{cases} +1, & f(\mathbf{x}) \geq 0, \\ -1, & f(\mathbf{x}) < 0. \end{cases}$$

In two dimensions, the decision boundary is a line. In three dimensions, it is a plane. In higher-dimensional spaces, it is called a hyperplane.

6.2.1 Margin and support vectors

Many different hyperplanes may separate a linearly separable dataset. An SVM selects the one that maximizes the distance between the decision boundary and the closest observations from each class.

The two margin boundaries are

$$\mathbf{w}^T \mathbf{x} + b = 1$$

and

$$\mathbf{w}^T \mathbf{x} + b = -1.$$

The distance between them is

$$\frac{2}{\|\mathbf{w}\|}.$$

Therefore, maximizing the margin is equivalent to minimizing $\|\mathbf{w}\|$.

The observations that lie on or inside the margin are called **support vectors**. These observations determine the position of the decision boundary. Observations located far from the margin generally do not affect the fitted hyperplane.

6.2.2 A maximum-margin classifier

The following example uses two Iris features to illustrate a linear SVM. The figure is generated entirely with Python.

```
iris = load_iris(as_frame=True)

X = iris.data.loc[
    iris.target.isin([0, 1]),
    ["petal length (cm)", "petal width (cm)"]
].to_numpy()

y = iris.target[iris.target.isin([0, 1])].to_numpy()

linear_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="linear", C=1_000)
)

linear_svm.fit(X, y)

fig, ax = plt.subplots(figsize=(7, 5))

DecisionBoundaryDisplay.from_estimator(
    linear_svm,
    X,
    response_method="decision_function",
    plot_method="contour",
    levels=[-1, 0, 1],
    linestyles=["--", "-", "--"],
    ax=ax
)

ax.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    marker="o",
    label="Iris setosa"
```

```

)

ax.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    marker="s",
    label="Iris versicolor"
)

support_vectors_scaled = linear_svm.named_steps["svc"].support_vectors_
support_vectors = linear_svm.named_steps["standardscaler"].inverse_transform(
    support_vectors_scaled
)

ax.scatter(
    support_vectors[:, 0],
    support_vectors[:, 1],
    s=180,
    facecolors="none",
    linewidths=1.5,
    label="Support vectors"
)

ax.set_xlabel("Petal length (cm)")
ax.set_ylabel("Petal width (cm)")
ax.legend()
plt.show()

```

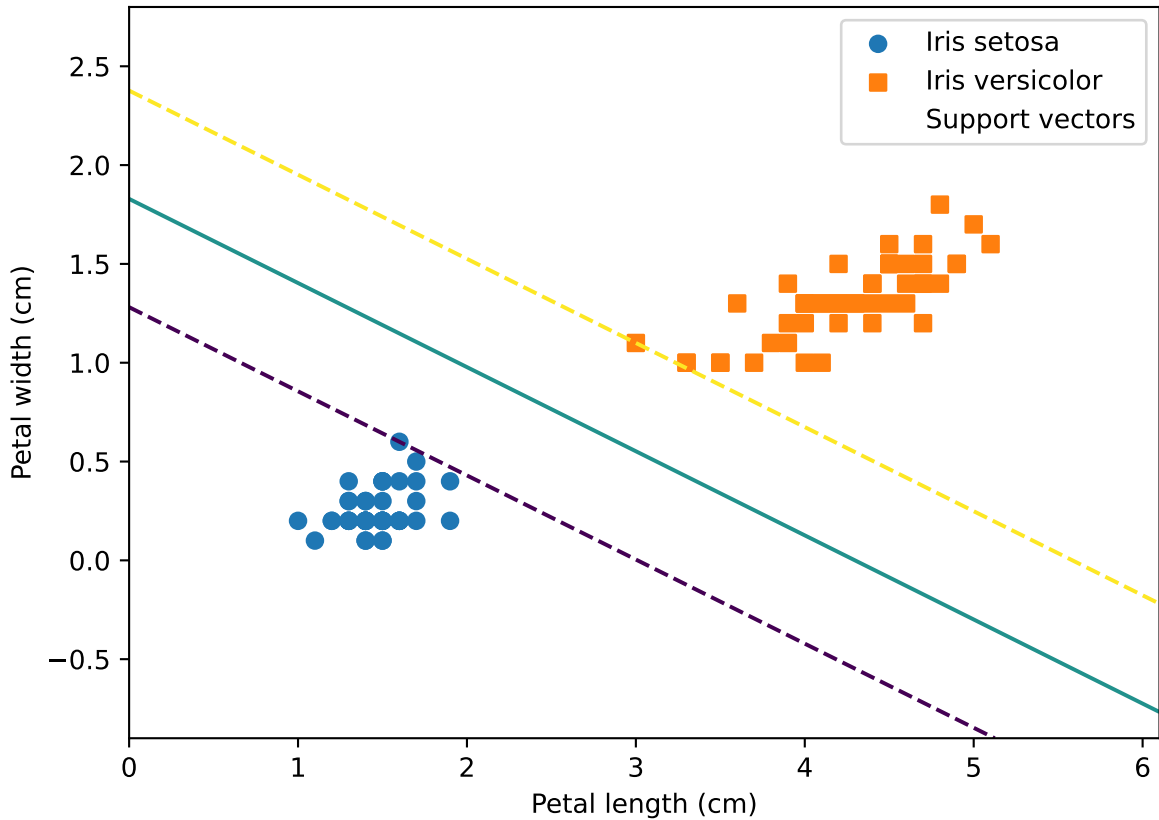


Figure 6.1: Linear SVM classification and its maximum-margin decision boundary.

The solid line represents the decision boundary. The dashed lines represent the margins, and the circled observations are the support vectors.

6.3 Why feature scaling matters

SVMs are sensitive to differences in feature scales because the margin depends on geometric distances. A feature measured over a wide numerical range may dominate another feature whose values occupy a much narrower range.

The next example compares the decision boundary before and after standardization.

```
X_scale = np.array([
    [1.0, 50],
    [5.0, 20],
    [3.0, 80],
```

```

    [5.0, 60]
])

y_scale = np.array([0, 0, 1, 1])

unscaled_model = SVC(kernel="linear", C=100)
unscaled_model.fit(X_scale, y_scale)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_scale)

scaled_model = SVC(kernel="linear", C=100)
scaled_model.fit(X_scaled, y_scale)

fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))

DecisionBoundaryDisplay.from_estimator(
    unscaled_model,
    X_scale,
    response_method="decision_function",
    plot_method="contour",
    levels=[-1, 0, 1],
    linestyles=["--", "-", "--"],
    ax=axes[0]
)

axes[0].scatter(X_scale[:, 0], X_scale[:, 1], c=y_scale)
axes[0].set_title("Without scaling")
axes[0].set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$")

DecisionBoundaryDisplay.from_estimator(
    scaled_model,
    X_scaled,
    response_method="decision_function",
    plot_method="contour",
    levels=[-1, 0, 1],
    linestyles=["--", "-", "--"],
    ax=axes[1]
)

axes[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_scale)

```

```

axes[1].set_title("After standardization")
axes[1].set_xlabel("Standardized  $x_1$ ")
axes[1].set_ylabel("Standardized  $x_2$ ")

plt.tight_layout()
plt.show()

```

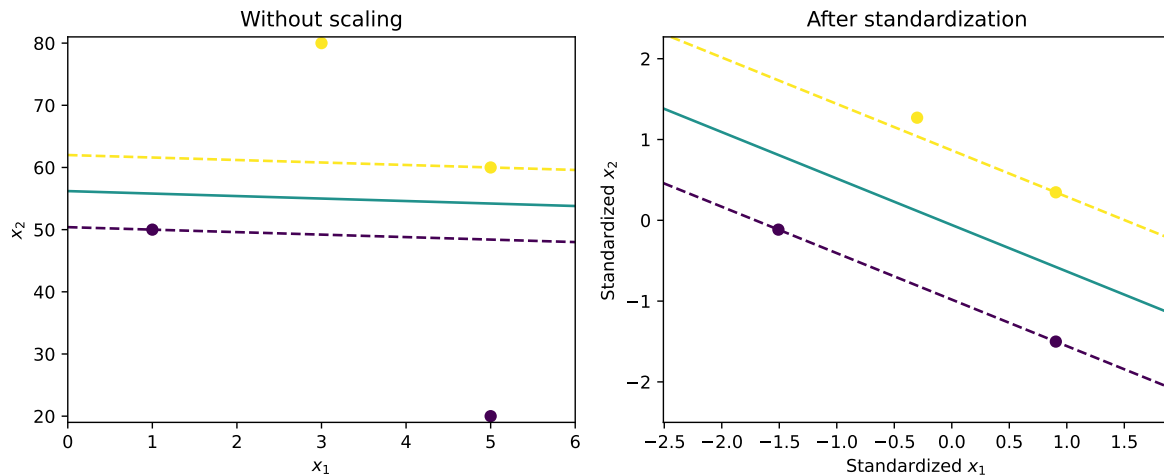


Figure 6.2: Effect of feature scaling on a linear SVM.

⚠ Scale the features

For SVMs, scaling should normally be part of the modeling pipeline. The scaler must be fitted only on the training data to prevent data leakage.

A robust Scikit-Learn workflow is:

```

svm_clf = make_pipeline(
    StandardScaler(),
    LinearSVC(C=1, loss="hinge", dual=True, random_state=42)
)

svm_clf.fit(X, y)

```

```

Pipeline(steps=[('standardscaler', StandardScaler()),
                 ('linearsvc',
                  LinearSVC(C=1, dual=True, loss='hinge', random_state=42))])

```

6.4 Hard-margin classification

A hard-margin classifier requires every training observation to be correctly classified and to remain outside the margin:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

The optimization problem is

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Hard-margin classification has two important limitations:

1. it works only when the data are linearly separable;
2. it is highly sensitive to outliers.

A single unusual observation may substantially alter the separating hyperplane or make perfect separation impossible.

6.5 Soft-margin classification

Soft-margin classification allows some observations to fall inside the margin or even on the wrong side of the decision boundary. It introduces slack variables $\xi_i \geq 0$:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i.$$

The corresponding objective is

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i.$$

The hyperparameter C controls the trade-off between a wide margin and margin violations:

- **large C** penalizes violations strongly and usually produces a narrower margin;
- **small C** tolerates more violations and usually produces a wider, more regularized margin.

6.5.1 Visualizing the effect of C

```
iris = load_iris()
X_c = iris.data[:, (2, 3)]
y_c = (iris.target == 2).astype(int)

models = [
    make_pipeline(StandardScaler(), SVC(kernel="linear", C=0.1)),
    make_pipeline(StandardScaler(), SVC(kernel="linear", C=100))
]

fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))

for model, ax, c_value in zip(models, axes, [0.1, 100]):
    model.fit(X_c, y_c)

    DecisionBoundaryDisplay.from_estimator(
        model,
        X_c,
        response_method="decision_function",
        plot_method="contour",
        levels=[-1, 0, 1],
        linestyles=["--", "-", "--"],
        ax=ax
    )

    ax.scatter(X_c[y_c == 0, 0], X_c[y_c == 0, 1], marker="o")
    ax.scatter(X_c[y_c == 1, 0], X_c[y_c == 1, 1], marker="s")
    ax.set_title(f"C = {c_value}")
    ax.set_xlabel("Petal length (cm)")
    ax.set_ylabel("Petal width (cm)")

plt.tight_layout()
plt.show()
```

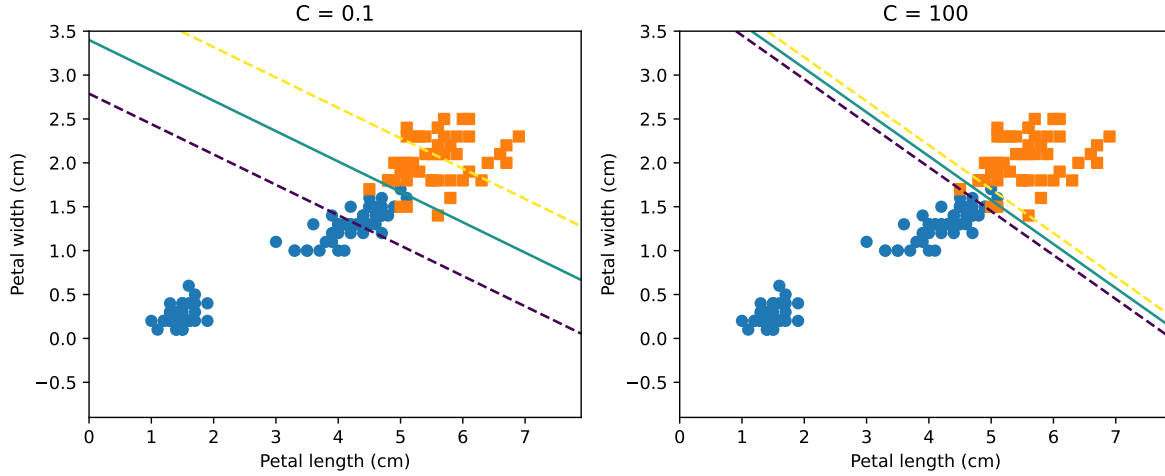


Figure 6.3: Effect of the regularization parameter C on the SVM margin.

A very large C may reduce training errors but create a model with high variance. A smaller C generally increases regularization and may improve generalization.

6.6 Hinge loss

The hinge loss for observation i is

$$L_i = \max(0, 1 - y_i f(\mathbf{x}_i)),$$

where

$$f(\mathbf{x}_i) = \mathbf{w}^\top \mathbf{x}_i + b.$$

The value

$$y_i f(\mathbf{x}_i)$$

is the signed functional margin.

Three cases are possible:

- if $y_i f(\mathbf{x}_i) \geq 1$, the observation is correctly classified and outside the margin, so the loss is zero;
- if $0 < y_i f(\mathbf{x}_i) < 1$, it is correctly classified but inside the margin;

- if $y_i f(\mathbf{x}_i) \leq 0$, it is misclassified.

```
signed_margin = np.linspace(-2, 3, 500)
hinge_loss = np.maximum(0, 1 - signed_margin)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(signed_margin, hinge_loss)
ax.axvline(0, linestyle="--")
ax.axvline(1, linestyle="--")
ax.set_xlabel(r"Signed margin  $y f(\mathbf{x})$ ")
ax.set_ylabel("Hinge loss")
ax.set_title("Hinge loss")
plt.show()
```

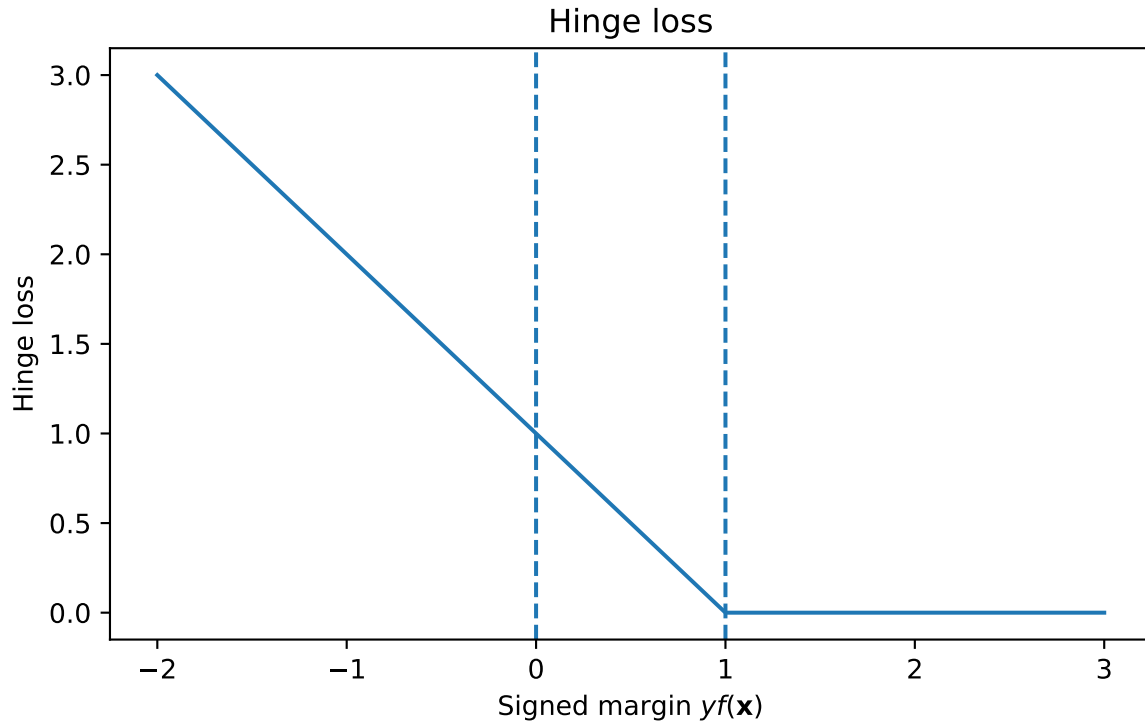


Figure 6.4: Hinge loss as a function of the signed margin.

! SVM scores are not probabilities

By default, SVM classifiers produce decision scores rather than class probabilities. `SVC(probability=True)` can estimate probabilities using an additional calibration procedure, but this increases training time.

6.7 Nonlinear SVM classification

A linear boundary is insufficient when the classes are not linearly separable in the original feature space. SVMs address this problem in two main ways:

1. explicitly create nonlinear features;
2. use a kernel that computes inner products in an implicit feature space.

```
# Original one-dimensional observations
X_original = np.linspace(-4, 4, 9).reshape(-1, 1)

# Feature mapping
X_transformed = np.c_[
    X_original,
    X_original ** 2
]

# Binary classes
y = np.array([0, 0, 1, 1, 1, 1, 1, 0, 0])

fig, axes = plt.subplots(
    1,
    2,
    figsize=(9.5, 4)
)

# -----
# Original feature space
# -----

ax = axes[0]

ax.grid(alpha=0.3)
ax.axhline(0, linewidth=1)
```

```

ax.plot(
    X_original[y == 0],
    np.zeros(np.sum(y == 0)),
    "s",
    label="Class 0"
)

ax.plot(
    X_original[y == 1],
    np.zeros(np.sum(y == 1)),
    "^",
    label="Class 1"
)

ax.set_xlabel(r"$x_1$")
ax.set_xlim(-4.5, 4.5)
ax.set_ylim(-0.2, 0.2)
ax.set_yticks([])
ax.set_title("Original feature space")

# -----
# Higher-dimensional feature space
# -----

ax = axes[1]

ax.grid(alpha=0.3)

ax.axhline(0, linewidth=1)
ax.axvline(0, linewidth=1)

ax.plot(
    X_transformed[y == 0, 0],
    X_transformed[y == 0, 1],
    "s",
    label="Class 0"
)

ax.plot(
    X_transformed[y == 1, 0],
    X_transformed[y == 1, 1],
    "^",

```

```

    label="Class 1"
)

# Linear separator

ax.plot(
    [-4.5, 4.5],
    [6.5, 6.5],
    "--",
    linewidth=2.5,
    label="Linear separator"
)

ax.set_xlabel(r"$x_1$")
ax.set_ylabel(r"$x_2=x_1^2$")
ax.set_xlim(-4.5, 4.5)
ax.set_ylim(-1, 17)

ax.set_title("Transformed feature space")

plt.tight_layout()
plt.show()

```

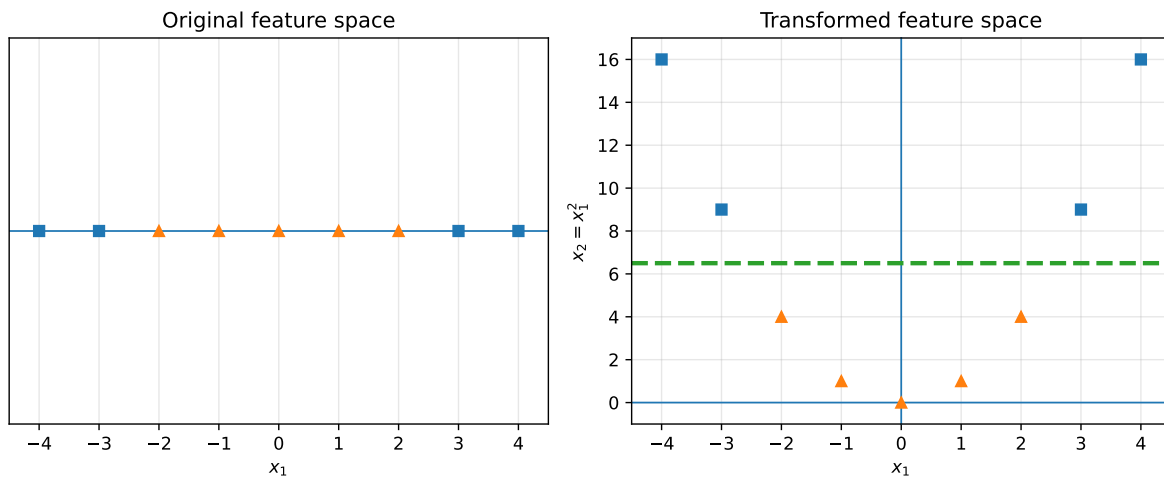


Figure 6.5: A nonlinear classification problem becomes linearly separable after mapping the observations to a higher-dimensional feature space.

Consider the one-dimensional dataset shown in the left panel of Figure Figure 6.5. No

linear classifier can perfectly separate the two classes because every possible threshold leaves observations from both classes on the same side.

Now suppose that we create a new feature,

$$x_2 = x_1^2.$$

The observations are then represented in the two-dimensional feature space (x_1, x_2) shown in the right panel. In this transformed space, the classes become linearly separable.

This simple example illustrates a fundamental idea in machine learning: a problem that is nonlinear in the original feature space may become linear after an appropriate feature transformation. Explicitly constructing these transformations is feasible for simple mappings such as x_1^2 , but quickly becomes impractical when thousands or millions of transformed features are required. Support Vector Machines overcome this limitation through the **kernel trick**, which computes inner products in the transformed feature space without explicitly generating all transformed features.

6.7.1 Polynomial features

A simple strategy is to transform the original inputs into polynomial features and then fit a linear SVM.

```
X_moons, y_moons = make_moons(
    n_samples=200,
    noise=0.15,
    random_state=42
)

polynomial_svm = make_pipeline(
    PolynomialFeatures(degree=3, include_bias=False),
    StandardScaler(),
    LinearSVC(C=10, loss="hinge", max_iter=20_000, random_state=42)
)

polynomial_svm.fit(X_moons, y_moons)
```

```
Pipeline(steps=[('polynomialfeatures',
                  PolynomialFeatures(degree=3, include_bias=False)),
                ('standardscaler', StandardScaler()),
                ('linearsvc',
```

```
LinearSVC(C=10, loss='hinge', max_iter=20000,  
          random_state=42))])
```

```
fig, ax = plt.subplots(figsize=(7, 5))  
  
DecisionBoundaryDisplay.from_estimator(  
    polynomial_svm,  
    X_moons,  
    response_method="decision_function",  
    plot_method="contourf",  
    alpha=0.25,  
    ax=ax  
)  
  
ax.scatter(  
    X_moons[y_moons == 0, 0],  
    X_moons[y_moons == 0, 1],  
    marker="o",  
    label="Class 0"  
)  
  
ax.scatter(  
    X_moons[y_moons == 1, 0],  
    X_moons[y_moons == 1, 1],  
    marker="s",  
    label="Class 1"  
)  
  
ax.set_xlabel("$x_1$")  
ax.set_ylabel("$x_2$")  
ax.legend()  
plt.show()
```

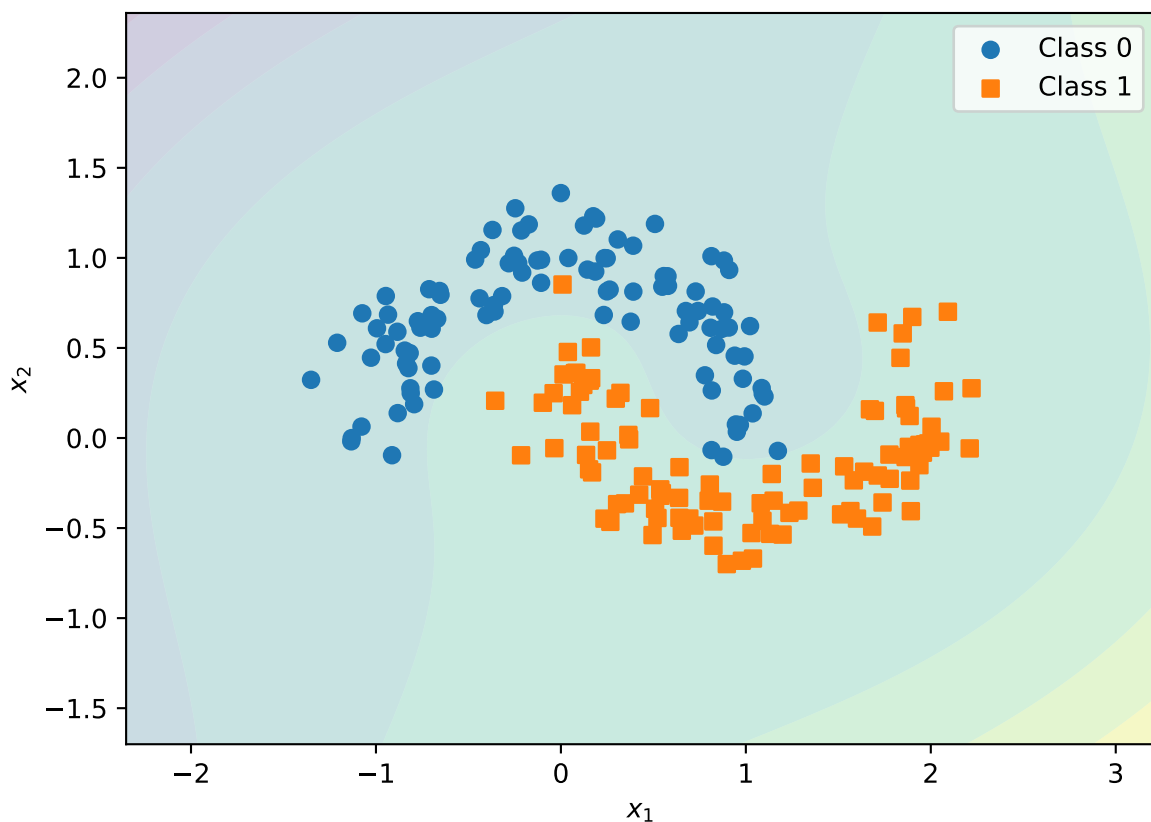


Figure 6.6: Linear SVM fitted after a third-degree polynomial feature transformation.

Explicit polynomial expansion is intuitive, but the number of generated features can increase quickly with the degree and the original number of predictors.

6.8 The kernel trick

A kernel computes a similarity measure between two observations:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}'),$$

where $\phi(\cdot)$ represents a potentially high-dimensional feature transformation.

The kernel trick makes it possible to work with the inner products in that transformed space without explicitly computing all transformed features.

Common kernels include:

6.8.1 Linear kernel

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'.$$

6.8.2 Polynomial kernel

$$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d,$$

where:

- d is the polynomial degree;
- γ controls the influence of the inner product;
- r , represented by `coef0` in Scikit-Learn, controls the contribution of lower- and higher-order terms.

6.8.3 Radial basis function kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2).$$

The RBF kernel is one of the most commonly used nonlinear kernels.

6.9 Polynomial kernel

```
poly_kernel_svm = make_pipeline(  
    StandardScaler(),  
    SVC(  
        kernel="poly",  
        degree=3,  
        coef0=1,  
        C=5  
    )  
)  
  
poly_kernel_svm.fit(X_moons, y_moons)
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),  
                 ('svc', SVC(C=5, coef0=1, kernel='poly'))])
```



```

fig, ax = plt.subplots(figsize=(7, 5))

DecisionBoundaryDisplay.from_estimator(
    poly_kernel_svm,
    X_moons,
    response_method="decision_function",
    plot_method="contourf",
    alpha=0.25,
    ax=ax
)

ax.scatter(X_moons[y_moons == 0, 0], X_moons[y_moons == 0, 1], marker="o")
ax.scatter(X_moons[y_moons == 1, 0], X_moons[y_moons == 1, 1], marker="s")

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
plt.show()

```

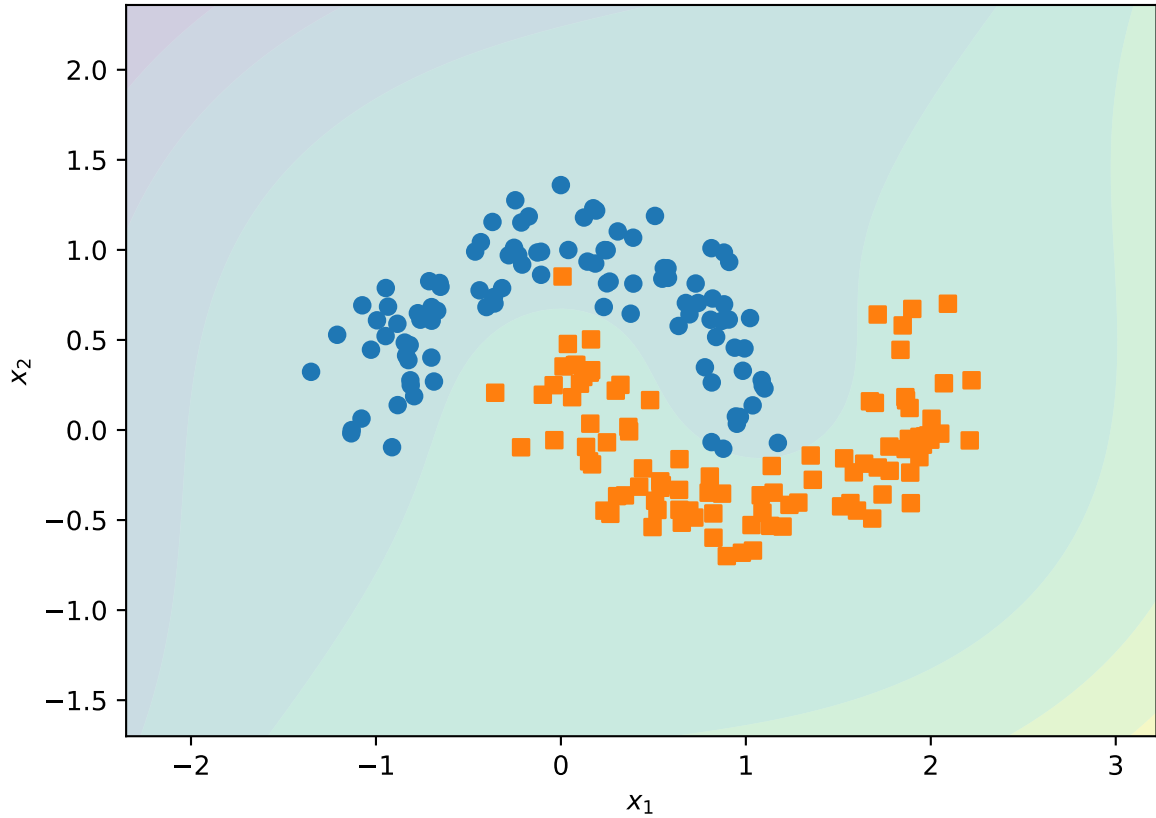


Figure 6.7: Decision boundary produced by a third-degree polynomial kernel.

If the model underfits, possible adjustments include increasing **degree**, increasing **C**, or changing **coef0**. If it overfits, the reverse adjustments may help.

6.10 Similarity features

A nonlinear transformation can also be constructed by measuring the similarity between an observation and selected landmarks.

Using a Gaussian radial basis function centered at landmark ℓ ,

$$\phi_{\ell}(\mathbf{x}) = \exp(-\gamma\|\mathbf{x} - \ell\|^2).$$

A small distance from the landmark produces a value close to 1, whereas a large distance produces a value close to 0.

```

def gaussian_rbf(X, landmark, gamma):
    """
    Compute the Gaussian radial basis function similarity
    between each observation in X and a landmark.
    """
    return np.exp(
        -gamma * np.linalg.norm(X - landmark, axis=1) ** 2
    )

# One-dimensional observations
X_1d = np.linspace(-4, 4, 9).reshape(-1, 1)

# Binary class labels
y_rbf = np.array([0, 0, 1, 1, 1, 1, 1, 0, 0])

# Landmarks and RBF parameter
landmark_1 = -2
landmark_2 = 1
gamma = 0.3

# Continuous grid used to draw the similarity functions
x_grid = np.linspace(-4.5, 4.5, 500).reshape(-1, 1)

similarity_landmark_1 = gaussian_rbf(
    x_grid,
    landmark_1,
    gamma
)

similarity_landmark_2 = gaussian_rbf(
    x_grid,
    landmark_2,
    gamma
)

# Transform the original observations into two similarity features
X_transformed = np.c_[
    gaussian_rbf(X_1d, landmark_1, gamma),
    gaussian_rbf(X_1d, landmark_2, gamma)
]

```

```

fig, axes = plt.subplots(
    1,
    2,
    figsize=(9.5, 4)
)

# -----
# Left panel: original one-dimensional feature space
# -----

ax = axes[0]

ax.axhline(
    y=0,
    linewidth=1
)

ax.scatter(
    [landmark_1, landmark_2],
    [0, 0],
    s=150,
    alpha=0.5,
    label="Landmarks"
)

ax.plot(
    X_1d[y_rbf == 0, 0],
    np.zeros(np.sum(y_rbf == 0)),
    "s",
    label="Class 0"
)

ax.plot(
    X_1d[y_rbf == 1, 0],
    np.zeros(np.sum(y_rbf == 1)),
    "^",
    label="Class 1"
)

ax.plot(
    x_grid[:, 0],
    similarity_landmark_1,

```

```

    "--",
    label=fr"Similarity to $\ell_1={landmark_1}$"
)

ax.plot(
    x_grid[:, 0],
    similarity_landmark_2,
    ":",
    label=fr"Similarity to $\ell_2={landmark_2}$"
)

ax.annotate(
    r"$\mathbf{x}$",
    xy=(X_1d[3, 0], 0),
    xytext=(-0.5, 0.22),
    ha="center",
    arrowprops={
        "arrowstyle": "->"
    },
    fontsize=14
)

ax.text(
    landmark_1,
    0.90,
    r"$\ell_1$",
    ha="center",
    fontsize=14
)

ax.text(
    landmark_2,
    0.90,
    r"$\ell_2$",
    ha="center",
    fontsize=14
)

ax.set_xlabel(r"Original feature $x_1$")
ax.set_ylabel("Similarity")
ax.set_xlim(-4.5, 4.5)
ax.set_ylim(-0.1, 1.1)

```

```

ax.set_yticks([0, 0.25, 0.5, 0.75, 1])
ax.grid(alpha=0.3)
ax.legend(fontsize=8, loc="upper right")

# -----
# Right panel: transformed similarity-feature space
# -----

ax = axes[1]

ax.axhline(
    y=0,
    linewidth=1
)

ax.axvline(
    x=0,
    linewidth=1
)

ax.plot(
    X_transformed[y_rbf == 0, 0],
    X_transformed[y_rbf == 0, 1],
    "s",
    label="Class 0"
)

ax.plot(
    X_transformed[y_rbf == 1, 0],
    X_transformed[y_rbf == 1, 1],
    "^",
    label="Class 1"
)

ax.annotate(
    r"$\phi(\mathbf{x})$",
    xy=(
        X_transformed[3, 0],
        X_transformed[3, 1]
    ),
    xytext=(0.68, 0.52),

```

```

        ha="center",
        arrowprops={
            "arrowstyle": "->"
        },
        fontsize=14
    )

# Illustrative linear decision boundary
x_boundary = np.array([-0.1, 1.1])
y_boundary = 0.51 - 0.61 * x_boundary

ax.plot(
    x_boundary,
    y_boundary,
    "--",
    linewidth=2.5,
    label="Linear decision boundary"
)

ax.set_xlabel(
    r"$x_2=\phi_{\ell_1}(\mathbf{x})$"
)

ax.set_ylabel(
    r"$x_3=\phi_{\ell_2}(\mathbf{x})$"
)

ax.set_xlim(-0.1, 1.1)
ax.set_ylim(-0.1, 1.1)
ax.grid(alpha=0.3)
ax.legend(fontsize=8, loc="upper right")

plt.tight_layout()
plt.show()

```

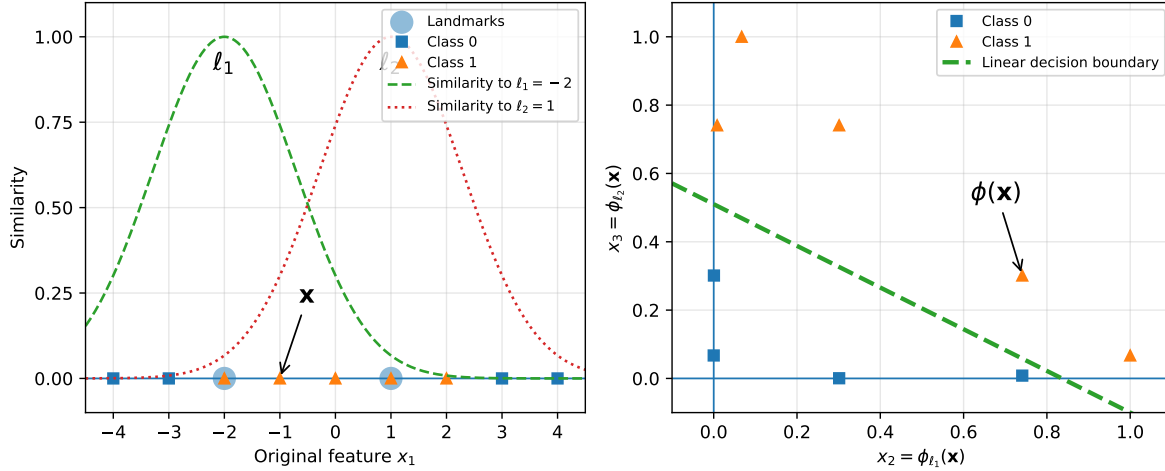


Figure 6.8: Gaussian RBF similarity features. Left: observations in the original one-dimensional space and their similarity to two landmarks. Right: observations in the transformed feature space.

The left panel shows the original one-dimensional observations and their Gaussian RBF similarity to two landmarks located at $\ell_1 = -2$ and $\ell_2 = 1$.

Each original observation is transformed into two new features:

$$x_2 = \phi_{\ell_1}(\mathbf{x}) = \exp(-\gamma\|\mathbf{x} - \ell_1\|^2),$$

and

$$x_3 = \phi_{\ell_2}(\mathbf{x}) = \exp(-\gamma\|\mathbf{x} - \ell_2\|^2).$$

The right panel represents the observations in the transformed feature space (x_2, x_3) . Although the classes are not linearly separable in the original one-dimensional space, the RBF transformation makes it possible to separate them using a linear decision boundary.

Increasing **gamma** makes each similarity curve narrower. Decreasing it makes the curve wider.

6.11 Gaussian RBF kernel

An RBF SVM can learn complex nonlinear boundaries without explicitly creating one feature per landmark.


```
rbf_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", gamma=5, C=1)
)

rbf_svm.fit(X_moons, y_moons)
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                  ('svc', SVC(C=1, gamma=5))])
```

6.11.1 Interpreting gamma

The hyperparameter **gamma** controls how rapidly the similarity decreases with distance:

- **large gamma** gives each training observation a narrow region of influence and can produce an irregular boundary;
- **small gamma** gives observations a broader region of influence and produces a smoother boundary.

Thus, **gamma** behaves as an inverse measure of the radius of influence.

6.11.2 Interaction between C and gamma

The following figure compares several combinations.

```
parameter_combinations = [
    (0.1, 0.1),
    (100, 0.1),
    (0.1, 5),
    (100, 5)
]

fig, axes = plt.subplots(2, 2, figsize=(9.5, 8))

for ax, (c_value, gamma_value) in zip(
    axes.ravel(),
    parameter_combinations
):
    model = make_pipeline(
        StandardScaler(),
```

```

        SVC(
            kernel="rbf",
            C=c_value,
            gamma=gamma_value
        )
    )

model.fit(X_moons, y_moons)

DecisionBoundaryDisplay.from_estimator(
    model,
    X_moons,
    response_method="decision_function",
    plot_method="contourf",
    alpha=0.25,
    ax=ax
)

ax.scatter(
    X_moons[y_moons == 0, 0],
    X_moons[y_moons == 0, 1],
    marker="o"
)

ax.scatter(
    X_moons[y_moons == 1, 0],
    X_moons[y_moons == 1, 1],
    marker="s"
)

ax.set_title(
    fr"$C={c_value}$, $\gamma$={gamma_value}$"
)

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")

plt.tight_layout()
plt.show()

```

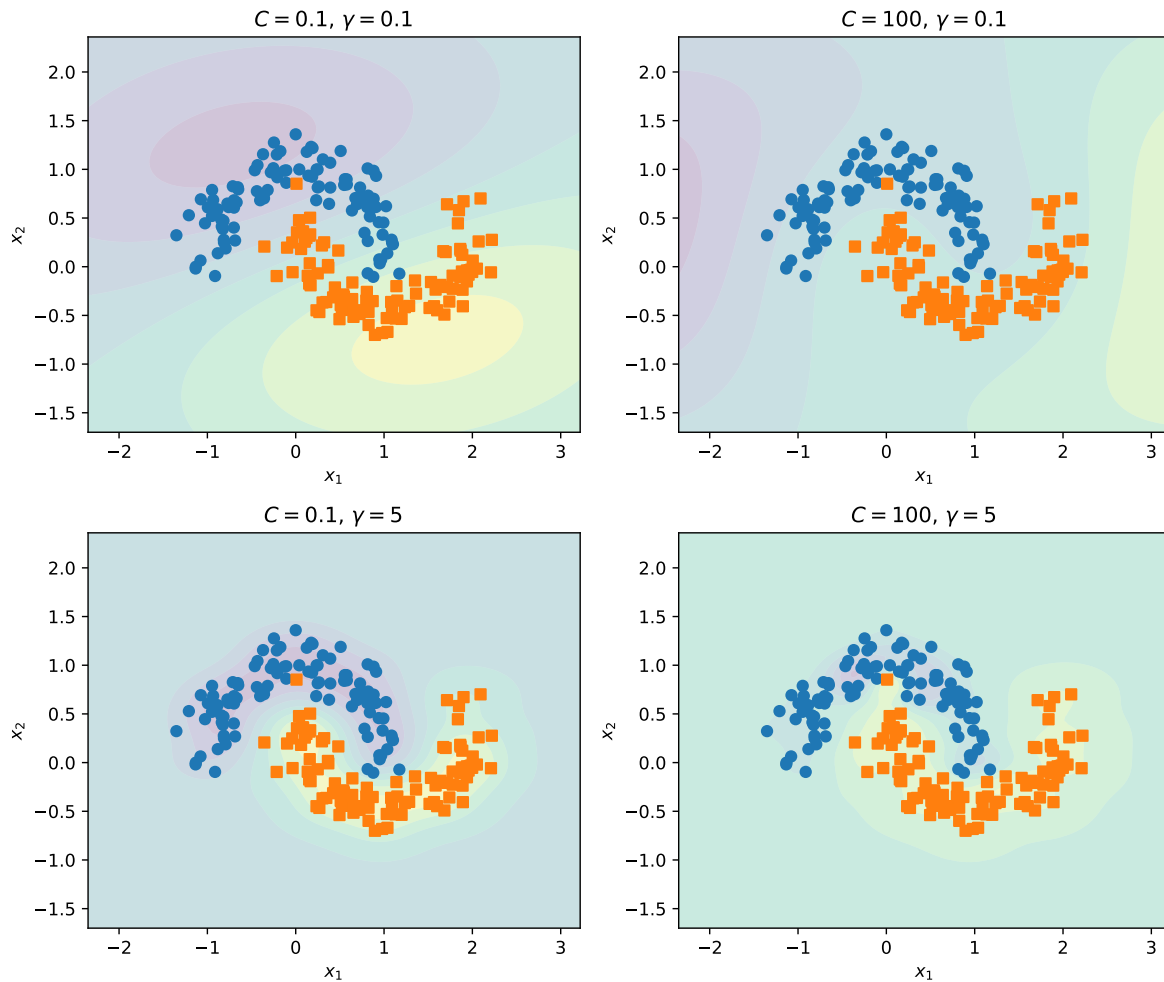


Figure 6.9: Interaction between C and γ for an RBF SVM.

A large C combined with a large **gamma** can fit highly localized patterns and may overfit. A small C combined with a small **gamma** produces a smoother and more regularized model.

💡 Practical tuning strategy

For an RBF SVM:

1. standardize all numerical predictors;
2. begin with logarithmic grids for C and **gamma**;
3. evaluate combinations with cross-validation;
4. inspect both predictive performance and the gap between training and validation scores.

6.12 Multiclass classification

SVMs are fundamentally binary classifiers, but Scikit-Learn extends them to multiclass problems.

SVC uses a one-versus-one strategy internally. For K classes, it trains

$$\frac{K(K-1)}{2}$$

binary classifiers.

`LinearSVC` uses a one-versus-rest strategy by default, fitting one classifier for each class.

```
iris = load_iris()

multiclass_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1, gamma="scale")
)

multiclass_svm.fit(iris.data, iris.target)

prediction = multiclass_svm.predict(
    iris.data[[0]]
)

prediction
```

```
array([0])
```

6.13 SVM regression

SVMs can also be used for regression. The objective is reversed relative to classification: the model attempts to place as many observations as possible inside an ε -wide tube around the regression function.

For prediction function

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b,$$

the ε -insensitive loss is

$$L_\varepsilon = \max(0, |y - f(\mathbf{x})| - \varepsilon).$$

Errors smaller than ε are ignored.

6.13.1 Linear SVR

```
rng = np.random.default_rng(42)

X_reg = 2 * rng.random((80, 1))
y_reg = 4 + 3 * X_reg[:, 0] + rng.normal(0, 0.8, 80)

epsilon_values = [0.2, 1.0]

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

x_plot = np.linspace(0, 2, 300).reshape(-1, 1)

for epsilon, ax in zip(epsilon_values, axes):
    model = make_pipeline(
        StandardScaler(),
        LinearSVR(
            epsilon=epsilon,
            C=1,
            dual="auto",
            max_iter=20_000,
            random_state=42
        )
    )

    model.fit(X_reg, y_reg)
    y_plot = model.predict(x_plot)

    ax.scatter(X_reg[:, 0], y_reg)
    ax.plot(x_plot[:, 0], y_plot)
    ax.plot(x_plot[:, 0], y_plot + epsilon, linestyle="--")
    ax.plot(x_plot[:, 0], y_plot - epsilon, linestyle="--")
    ax.set_title(fr"$\varepsilon$={epsilon}")
    ax.set_xlabel("$x$")
```

```
ax.set_ylabel("$y$")

plt.tight_layout()
plt.show()
```

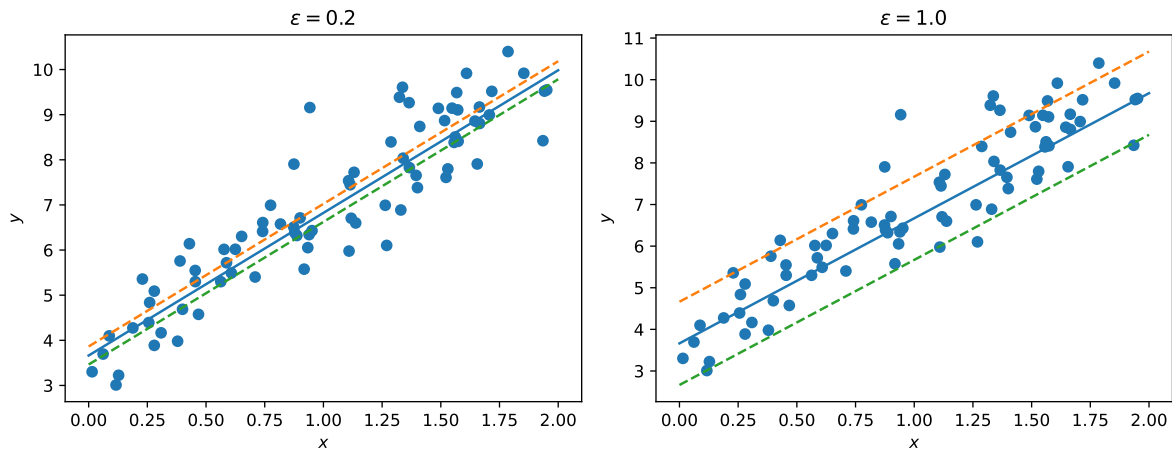


Figure 6.10: Linear support vector regression with two values of epsilon.

The model is called ε -insensitive because movements of training observations within the tube do not affect the loss.

6.13.2 Nonlinear SVR

A kernelized SVR can model nonlinear relationships.

```
rng = np.random.default_rng(42)

X_quad = 2 * rng.random((100, 1)) - 1
y_quad = (
    0.2
    + 0.1 * X_quad[:, 0]
    + 0.5 * X_quad[:, 0] ** 2
    + rng.normal(0, 0.08, 100)
)

models = [
    SVR(kernel="poly", degree=2, C=100, epsilon=0.1, coef0=1),
```

```

SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1, coef0=1)
]

x_plot = np.linspace(-1, 1, 300).reshape(-1, 1)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for model, ax, c_value in zip(models, axes, [100, 0.01]):
    model.fit(X_quad, y_quad)
    y_plot = model.predict(x_plot)

    ax.scatter(X_quad[:, 0], y_quad)
    ax.plot(x_plot[:, 0], y_plot)
    ax.plot(x_plot[:, 0], y_plot + model.epsilon, linestyle="--")
    ax.plot(x_plot[:, 0], y_plot - model.epsilon, linestyle="--")
    ax.set_title(fr"Polynomial SVR, $C={c_value}$")
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")

plt.tight_layout()
plt.show()

```

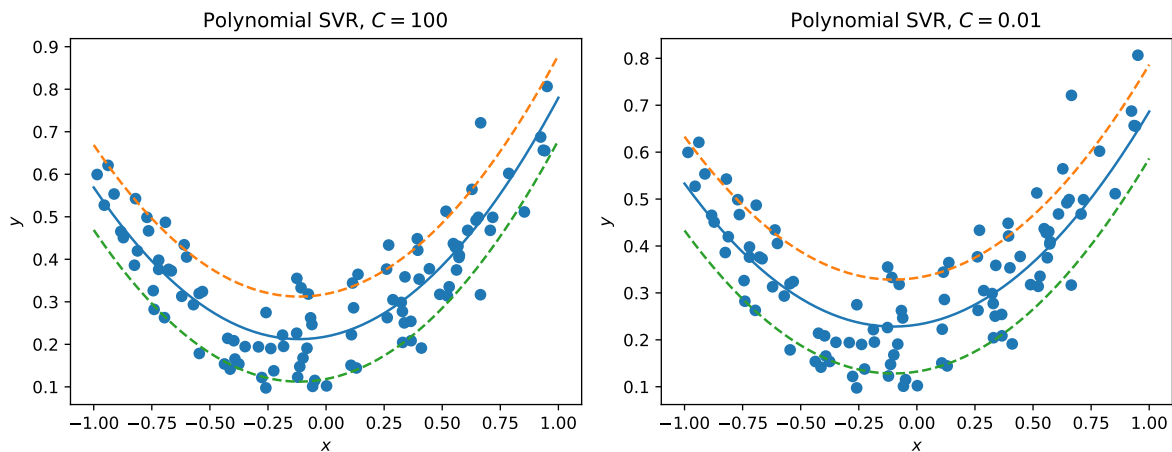


Figure 6.11: Polynomial-kernel SVR with different values of C .

A larger C reduces regularization and attempts to fit the training observations more closely. A smaller C produces a smoother model.

6.14 Computational considerations

The main Scikit-Learn SVM estimators have different computational characteristics.

Estimator	Main use	Kernel support	Typical behavior
<code>LinearSVC</code>	Linear classification	No	Scales comparatively well to large datasets
<code>SVC(kernel="linear")</code>	Linear classification	Yes	Useful when support vectors are required
<code>SVC(kernel="rbf")</code>	Nonlinear classification	Yes	Powerful but expensive for large datasets
<code>LinearSVR</code>	Linear regression	No	Suitable for larger datasets
<code>SVR</code>	Nonlinear regression	Yes	Can become slow as sample size increases

Kernel SVMs usually require substantial computation as the number of observations grows. Their training time can increase rapidly because the algorithm works with pairwise relationships between observations.

6.15 Practical modeling workflow

A sound SVM workflow includes:

1. split the data into training and test sets;
2. place preprocessing and SVM estimation in one pipeline;
3. standardize numerical variables;
4. select a kernel according to the expected boundary complexity;
5. tune `C`, and for RBF models tune `gamma`;
6. use cross-validation for model selection;
7. evaluate the final model once on untouched test data;
8. inspect class imbalance and choose suitable metrics.

Example pipeline:


```
svm_pipeline = make_pipeline(  
    StandardScaler(),  
    SVC(  
        kernel="rbf",  
        C=1,  
        gamma="scale"  
    )  
)
```

6.16 Common mistakes

Data leakage

Do not standardize the complete dataset before splitting it. Fit the scaler only through a pipeline trained on the training data.

Unscaled predictors

Using predictors with very different scales may distort the geometry of the problem and produce a poor decision boundary.

Interpreting C incorrectly

In Scikit-Learn, a larger C means **less regularization**, while a smaller C means **more regularization**.

Tuning on the test set

The test set should not be used to select C, gamma, the kernel, or any preprocessing decision.

6.17 Chapter summary

Support Vector Machines build decision boundaries by maximizing the margin between classes. The closest training observations, called support vectors, determine the boundary. Hard-margin classification requires perfect linear separation, while soft-margin classification allows violations controlled by C.

Nonlinear SVMs use kernels to represent complex boundaries without explicitly generating all transformed features. The polynomial and RBF kernels are common choices. In an RBF model, `gamma` controls the radius of influence of each observation, while `C` controls the penalty for margin violations.

SVM regression uses an ϵ -insensitive tube and ignores prediction errors smaller than ϵ . In both classification and regression, feature scaling and careful hyperparameter tuning are essential.

6.18 Exercises

1. Fit linear SVM classifiers to the Iris dataset using several values of `C`. Compare the number of support vectors and the width of the margin.
2. Generate a moons dataset with a higher noise level. Compare a polynomial-feature SVM, a polynomial-kernel SVM, and an RBF SVM.
3. For an RBF classifier, create a logarithmic grid of values for `C` and `gamma`. Use cross-validation to select the best combination.
4. Explain why increasing `gamma` may lead to overfitting.
5. Compare `LinearSVC` and `SVC(kernel="linear")` on the same standardized dataset. Examine their accuracy, training time, and available attributes.
6. Fit an `SVR` model to a nonlinear regression dataset. Study how the fitted curve changes when `C`, `epsilon`, and `gamma` are varied.

7 Chapter – Decision Trees

7.1 Introduction

Decision Trees are supervised learning models that can perform classification, regression, and multioutput tasks. They are intuitive, flexible, and capable of learning nonlinear relationships and complex interactions between predictors.

A Decision Tree partitions the feature space recursively. At each internal node, the algorithm asks a question of the form

$$x_j \leq t,$$

where x_j is one predictor and t is a threshold. Each answer sends the observation to the left or right branch until it reaches a terminal node, called a **leaf**. The prediction associated with that leaf is then returned.

Decision Trees are also the basic building blocks of ensemble methods such as Random Forests, Extra Trees, Gradient Boosting, XGBoost, and LightGBM.

Learning objectives

After completing this chapter, you should be able to:

- explain how a Decision Tree recursively partitions the feature space;
- interpret the structure and output of a classification tree;
- calculate and interpret Gini impurity and entropy;
- explain the Classification and Regression Tree algorithm;
- interpret class probabilities produced by a tree;
- recognize why unrestricted trees tend to overfit;
- regularize a tree using structural hyperparameters;
- fit and interpret regression trees;
- explain cost-complexity pruning;
- distinguish Decision Trees from Extra Trees;
- identify the main advantages and limitations of tree-based models.

7.2 Training a classification tree

We begin with the Iris dataset. To make the tree easy to visualize, we use only petal length and petal width.

```
iris = load_iris()

X = iris.data[:, 2:]
y = iris.target

tree_clf = DecisionTreeClassifier(
    max_depth=2,
    random_state=42
)

tree_clf.fit(X, y)
```

```
DecisionTreeClassifier(max_depth=2, random_state=42)
```

Unlike many distance-based models, Decision Trees do not require feature scaling or centering. Their splits depend only on the ordering of predictor values and not on their units.

Minimal preprocessing

Decision Trees generally do not require standardization. However, missing values, categorical variables, leakage, and inappropriate train-test splitting still require careful treatment.

7.3 Visualizing the tree with Python

Scikit-Learn can draw the fitted tree directly with `plot_tree()`, so Graphviz is not required.

```
fig, ax = plt.subplots(figsize=(11, 7))

plot_tree(
    tree_clf,
    feature_names=iris.feature_names[2:],
    class_names=iris.target_names,
    rounded=True,
```

```

filled=True,
impurity=True,
proportion=False,
ax=ax
)

plt.show()

```

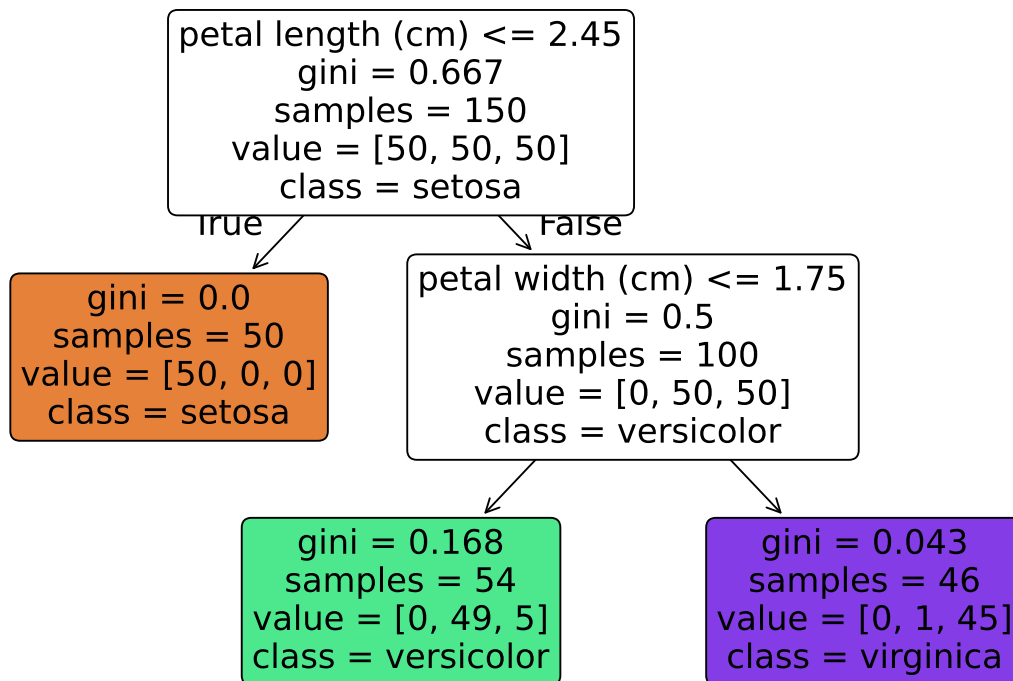


Figure 7.1: Decision Tree trained on petal length and petal width from the Iris dataset.

Each node contains several pieces of information:

- the splitting rule;
- the impurity measure;
- the number of training observations in the node;
- the class counts;
- the predicted class.

The root node is at depth 0. Internal nodes contain splitting rules, while leaf nodes return final predictions.

7.4 How the tree makes predictions

Suppose an Iris flower has a petal length smaller than the root threshold. The observation moves to the left branch. If that branch ends in a leaf dominated entirely by *Iris setosa*, the predicted class is *setosa*.

If the petal length is larger than the root threshold, the observation moves to the right branch. A second question, perhaps involving petal width, then separates *versicolor* from *virginica*.

This prediction process is deterministic. Every observation follows exactly one path from the root to a leaf.

```
new_flowers = np.array([
    [1.5, 0.3],
    [5.0, 1.5],
    [6.0, 2.2]
])

tree_clf.predict(new_flowers)
```

```
array([0, 1, 2])
```

7.5 Samples, values, and predicted classes

The **samples** entry reports how many training observations reach the node.

The **value** entry contains the class counts. For example,

```
value = [0, 49, 5]
```

means that the node contains:

- 0 observations from class 0;
- 49 observations from class 1;
- 5 observations from class 2.

The predicted class is the class with the largest count.

7.6 Gini impurity

Scikit-Learn uses Gini impurity by default for classification trees.

For node i , the Gini impurity is

$$G_i = 1 - \sum_{k=1}^K p_{i,k}^2,$$

where $p_{i,k}$ is the proportion of observations from class k in node i .

A node is pure when all observations belong to the same class. In that case, one class proportion is 1 and all others are 0, so

$$G_i = 0.$$

For a node with counts

$$[0, 49, 5],$$

the impurity is

$$G_i = 1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 \approx 0.168.$$

```
counts = np.array([0, 49, 5])
probabilities = counts / counts.sum()

gini = 1 - np.sum(probabilities ** 2)
gini
```

```
np.float64(0.16803840877914955)
```

A lower impurity indicates that the node contains a more homogeneous set of classes.

7.7 Decision boundaries

A Decision Tree creates axis-aligned partitions because every split involves one predictor at a time.

```
fig, ax = plt.subplots(figsize=(8, 6))

DecisionBoundaryDisplay.from_estimator(
    tree_clf,
    X,
    response_method="predict",
    plot_method="pcolormesh",
    alpha=0.25,
    ax=ax
)

for class_id, marker in zip(np.unique(y), ["o", "s", "^"]):
    ax.scatter(
        X[y == class_id, 0],
        X[y == class_id, 1],
        marker=marker,
        label=iris.target_names[class_id]
    )

ax.set_xlabel("Petal length (cm)")
ax.set_ylabel("Petal width (cm)")
ax.legend()
plt.show()
```

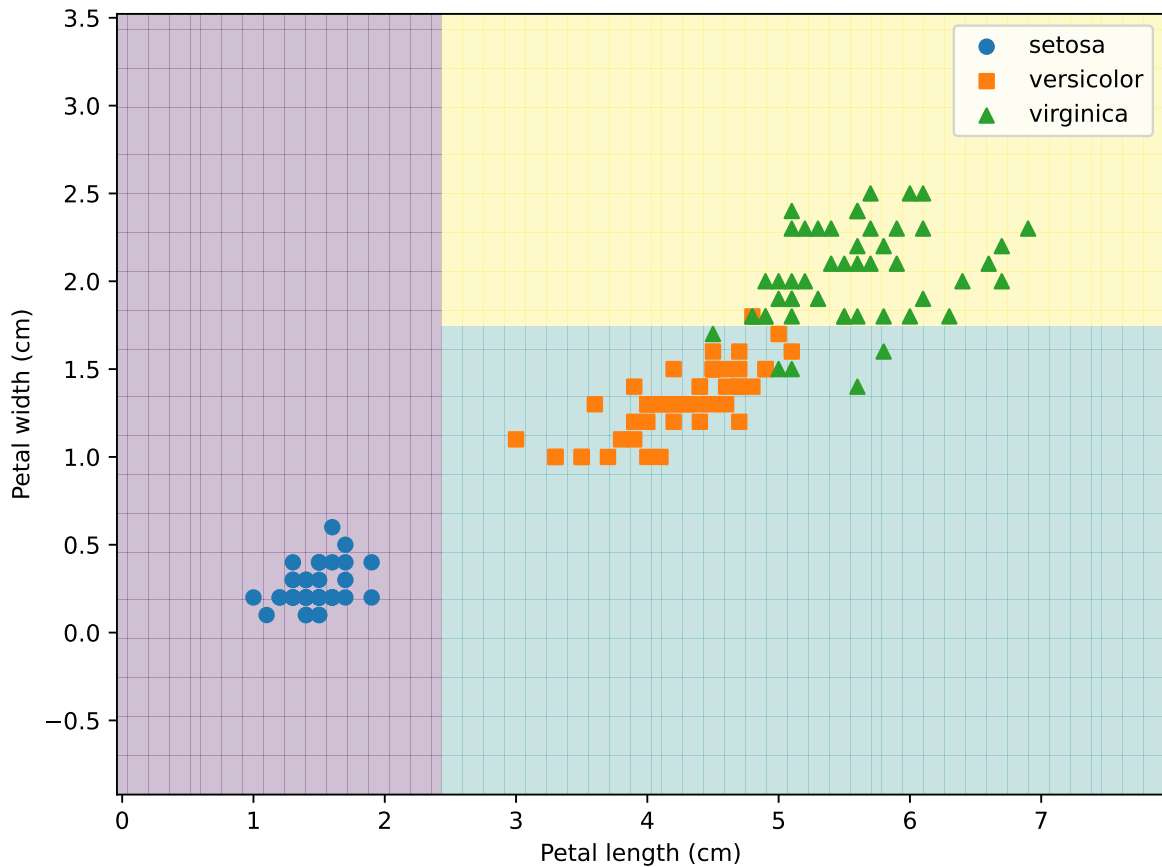



Figure 7.2: Decision boundaries generated by a depth-two Decision Tree.

The first split divides the plane vertically because it uses petal length. A later split divides one region horizontally because it uses petal width.

Increasing `max_depth` allows the tree to create more rectangular regions.

```
depths = [1, 2, 4]

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for depth, ax in zip(depths, axes):
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )
    model.fit(X, y)
```

```

DecisionBoundaryDisplay.from_estimator(
    model,
    X,
    response_method="predict",
    plot_method="pcolormesh",
    alpha=0.25,
    ax=ax
)

for class_id, marker in zip(np.unique(y), ["o", "s", "^"]):
    ax.scatter(
        X[y == class_id, 0],
        X[y == class_id, 1],
        marker=marker
    )

ax.set_title(f"max_depth = {depth}")
ax.set_xlabel("Petal length")
ax.set_ylabel("Petal width")

plt.tight_layout()
plt.show()

```

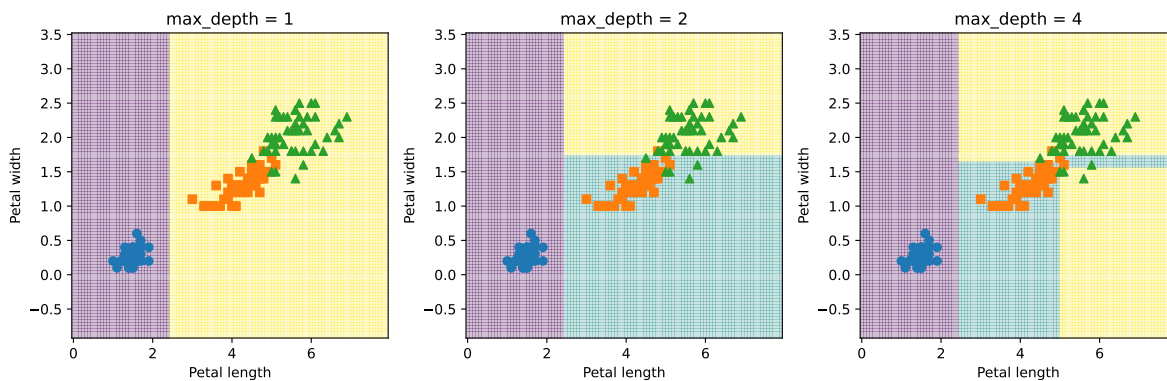


Figure 7.3: Increasing tree depth creates increasingly detailed partitions.

7.8 Estimating class probabilities

A classification tree can estimate class probabilities. It first locates the leaf containing the new observation and then computes the relative frequency of every class in that leaf.

For a flower with petal length 5 cm and petal width 1.5 cm:

```
tree_clf.predict_proba([[5, 1.5]])
```

```
array([[0.          , 0.90740741, 0.09259259]])
```

The corresponding class prediction is:

```
tree_clf.predict([[5, 1.5]])
```

```
array([1])
```

The class with the largest estimated probability becomes the predicted class.

! Piecewise-constant probabilities

All observations that fall in the same leaf receive exactly the same probability vector. Decision Tree probabilities may therefore be poorly calibrated, especially when leaves contain few observations.

7.9 The CART training algorithm

Scikit-Learn uses the Classification and Regression Tree algorithm. CART grows binary trees: every split produces exactly two child nodes.

At a given node, the algorithm searches over candidate features j and thresholds t_j . For classification, it selects the split minimizing

$$J(j, t_j) = \frac{m_{\text{left}}}{m} I_{\text{left}} + \frac{m_{\text{right}}}{m} I_{\text{right}},$$

where:

- m is the number of observations in the current node;

- m_{left} and m_{right} are the child-node sizes;
- I_{left} and I_{right} are their impurities.

The algorithm repeats this process recursively until a stopping condition is reached.

Typical stopping conditions include:

- the maximum depth has been reached;
- the node contains too few observations to split;
- no candidate split provides sufficient improvement;
- a structural restriction prevents further growth.

CART uses a greedy strategy. It chooses the best split available at the current node without considering whether another split might produce a globally better tree later.

i Greedy optimization

The fitted tree is not guaranteed to be the globally optimal tree. Finding the smallest optimal tree is computationally difficult, so practical algorithms rely on greedy recursive partitioning.

7.10 Gini impurity versus entropy

Entropy is another common classification criterion:

$$H_i = - \sum_{k=1}^K p_{i,k} \log_2(p_{i,k}),$$

where terms with $p_{i,k} = 0$ are omitted.

Entropy is zero for a pure node. It increases as the class distribution becomes more mixed.

```
p = np.linspace(0.001, 0.999, 500)

gini_curve = 1 - p**2 - (1 - p)**2
entropy_curve = -p * np.log2(p) - (1 - p) * np.log2(1 - p)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(p, gini_curve, label="Gini impurity")
ax.plot(p, entropy_curve, label="Entropy")
ax.set_xlabel("Proportion of class 1")
ax.set_ylabel("Impurity")
```

```
ax.legend()  
plt.show()
```

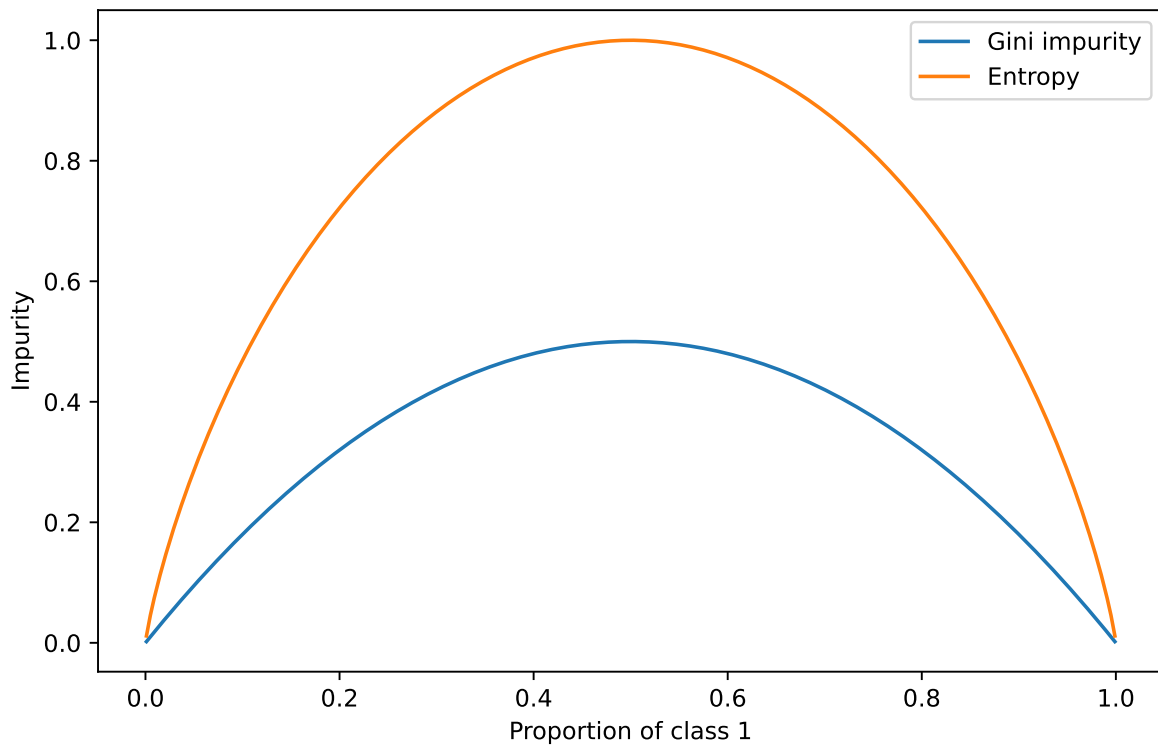


Figure 7.4: Gini impurity and entropy for a binary classification node.

In practice, Gini impurity and entropy often produce similar trees. Gini is somewhat faster to compute and is a reasonable default. Entropy sometimes creates slightly more balanced splits, but the difference is usually modest compared with the effects of regularization and validation.

```
gini_tree = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=3,  
    random_state=42  
)  
  
entropy_tree = DecisionTreeClassifier(  
    criterion="entropy",  
    max_depth=3,
```

```

    random_state=42
)

gini_tree.fit(X, y)
entropy_tree.fit(X, y)

gini_tree.get_n_leaves(), entropy_tree.get_n_leaves()

```

```
(np.int64(5), np.int64(5))
```

7.11 Why Decision Trees overfit

An unrestricted Decision Tree can continue partitioning until leaves are nearly pure. This frequently produces:

- very high training accuracy;
- low bias;
- high variance;
- unstable decision boundaries;
- poor performance on unseen data.

The following example uses a noisy moons dataset.

```

X_moons, y_moons = make_moons(
    n_samples=500,
    noise=0.30,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X_moons,
    y_moons,
    test_size=0.30,
    stratify=y_moons,
    random_state=42
)

```

```

unregularized_tree = DecisionTreeClassifier(
    random_state=42
)

unregularized_tree.fit(X_train, y_train)

fig, ax = plt.subplots(figsize=(8, 6))

DecisionBoundaryDisplay.from_estimator(
    unregularized_tree,
    X_train,
    response_method="predict",
    plot_method="pcolormesh",
    alpha=0.25,
    ax=ax
)

ax.scatter(
    X_train[y_train == 0, 0],
    X_train[y_train == 0, 1],
    marker="o",
    label="Class 0"
)

ax.scatter(
    X_train[y_train == 1, 0],
    X_train[y_train == 1, 1],
    marker="s",
    label="Class 1"
)

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.legend()
plt.show()

```

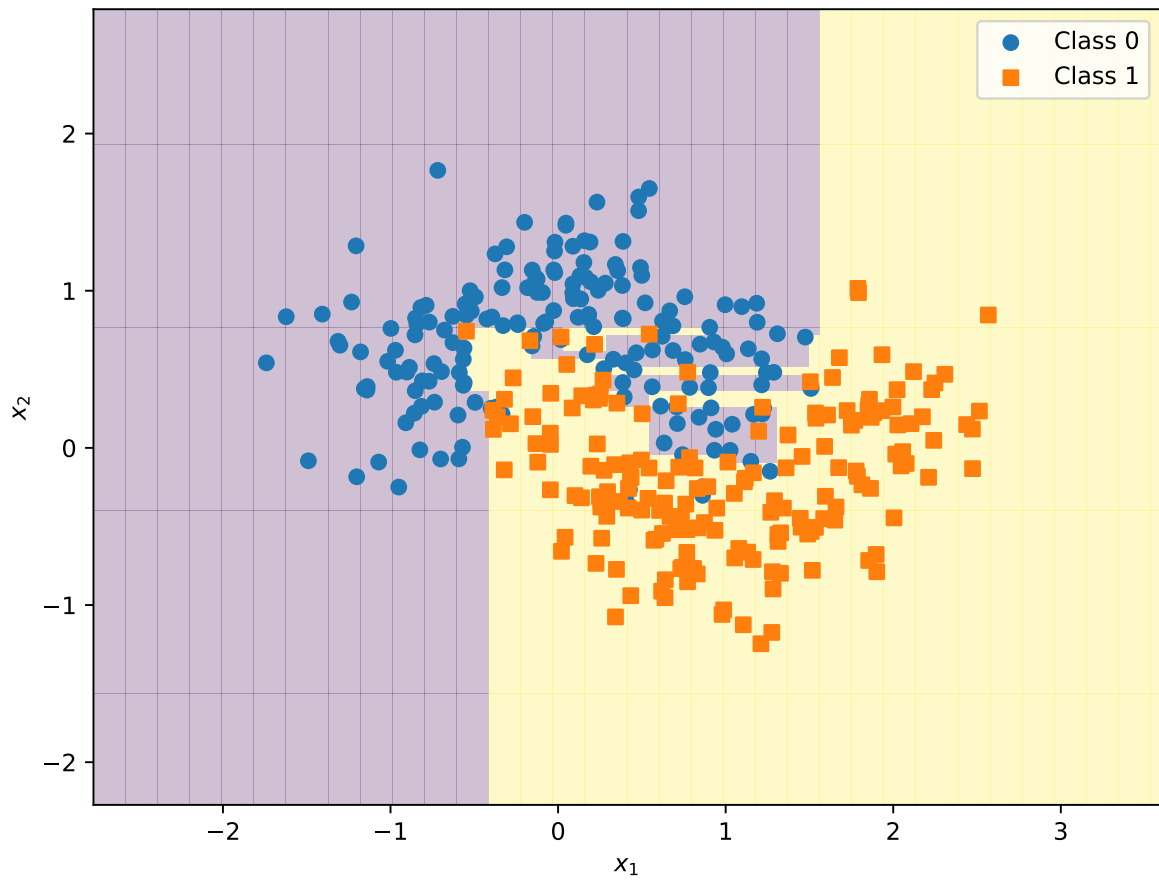


Figure 7.5: An unrestricted tree creates an irregular boundary on noisy data.

```
train_accuracy = accuracy_score(
    y_train,
    unregularized_tree.predict(X_train)
)

test_accuracy = accuracy_score(
    y_test,
    unregularized_tree.predict(X_test)
)

train_accuracy, test_accuracy
```

```
(1.0, 0.8666666666666667)
```


A large difference between training and test performance is evidence of overfitting.

7.12 Regularization hyperparameters

Decision Trees are nonparametric models: their complexity is not fixed before training. Regularization restricts the tree-growing process.

Hyperparameter	What it controls	Effect of stronger restriction
<code>max_depth</code>	Maximum number of tree levels	Produces a shallower tree
<code>min_samples_split</code>	Minimum observations required to split a node	Prevents splitting small nodes
<code>min_samples_leaf</code>	Minimum observations required in every leaf	Produces larger, more stable leaves
<code>min_weight_fraction_leaf</code>	Minimum weighted fraction required in a leaf	Useful with observation weights
<code>max_leaf_nodes</code>	Maximum number of terminal nodes	Directly limits tree size
<code>max_features</code>	Number of predictors considered for a split	Adds randomness and can reduce variance
<code>min_impurity_decrease</code>	Minimum required impurity reduction	Rejects weak splits
<code>ccp_alpha</code>	Cost-complexity pruning strength	Removes branches after growth

A regularized model might be:

```
regularized_tree = DecisionTreeClassifier(  
    max_depth=5,  
    min_samples_leaf=10,  
    random_state=42  
)  
  
regularized_tree.fit(X_train, y_train)
```

```
DecisionTreeClassifier(max_depth=5, min_samples_leaf=10, random_state=42)
```

```

fig, ax = plt.subplots(figsize=(8, 6))

DecisionBoundaryDisplay.from_estimator(
    regularized_tree,
    X_train,
    response_method="predict",
    plot_method="pcolormesh",
    alpha=0.25,
    ax=ax
)

ax.scatter(
    X_train[y_train == 0, 0],
    X_train[y_train == 0, 1],
    marker="o"
)

ax.scatter(
    X_train[y_train == 1, 0],
    X_train[y_train == 1, 1],
    marker="s"
)

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
plt.show()

```

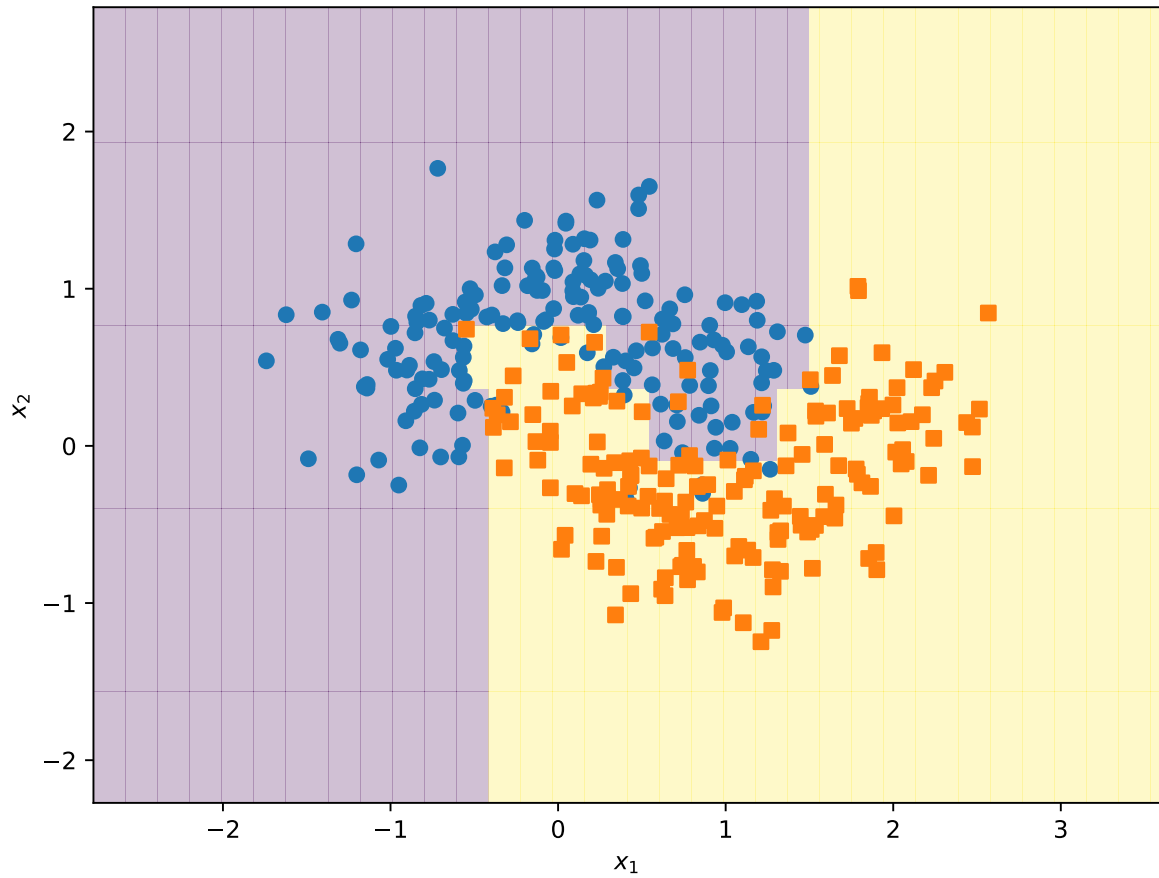


Figure 7.6: Regularization produces a smoother and more stable decision boundary.

```
regularized_train_accuracy = accuracy_score(
    y_train,
    regularized_tree.predict(X_train)
)

regularized_test_accuracy = accuracy_score(
    y_test,
    regularized_tree.predict(X_test)
)

regularized_train_accuracy, regularized_test_accuracy
```

(0.9457142857142857, 0.8933333333333333)

Regularization may reduce training accuracy while improving generalization.

7.13 Pre-pruning and post-pruning

Tree complexity can be controlled in two conceptually different ways.

7.13.1 Pre-pruning

Pre-pruning stops the tree while it is being grown. Hyperparameters such as `max_depth`, `min_samples_leaf`, and `min_impurity_decrease` are pre-pruning controls.

7.13.2 Post-pruning

Post-pruning first grows a larger tree and then removes weak branches.

Scikit-Learn implements **minimal cost-complexity pruning** through `ccp_alpha`. The objective balances impurity and the number of terminal nodes:

$$R_{\alpha}(T) = R(T) + \alpha|\widetilde{T}|,$$

where:

- $R(T)$ is the total leaf impurity;
- $|\widetilde{T}|$ is the number of leaves;
- α controls the penalty on tree size.

Larger values of `ccp_alpha` produce smaller trees.

```
full_tree = DecisionTreeClassifier(
    random_state=42
)

path = full_tree.cost_complexity_pruning_path(
    X_train,
    y_train
)

ccp_alphas = path.ccp_alphas
impurities = path.impurities
```

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(ccp_alphas[:-1], impurities[:-1], marker="o")
ax.set_xlabel("Effective alpha")
ax.set_ylabel("Total leaf impurity")
plt.show()
```

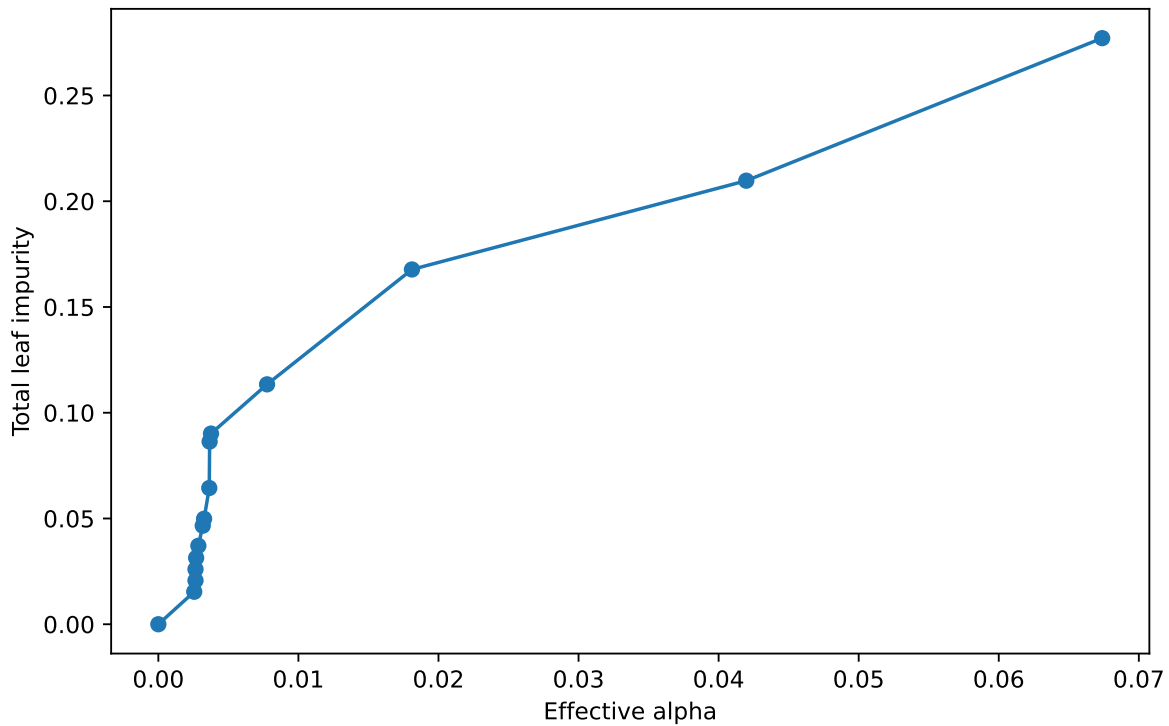


Figure 7.7: Cost-complexity pruning path for the training data.

```
pruned_trees = []

for alpha in ccp_alphas:
    model = DecisionTreeClassifier(
        random_state=42,
        ccp_alpha=alpha
    )
    model.fit(X_train, y_train)
    pruned_trees.append(model)

node_counts = [model.tree_.node_count for model in pruned_trees]
```

```
tree_depths = [model.tree_.max_depth for model in pruned_trees]
```

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(ccp_alphas, node_counts, marker="o", label="Number of nodes")
ax.plot(ccp_alphas, tree_depths, marker="s", label="Tree depth")
ax.set_xlabel("ccp_alpha")
ax.legend()
plt.show()
```

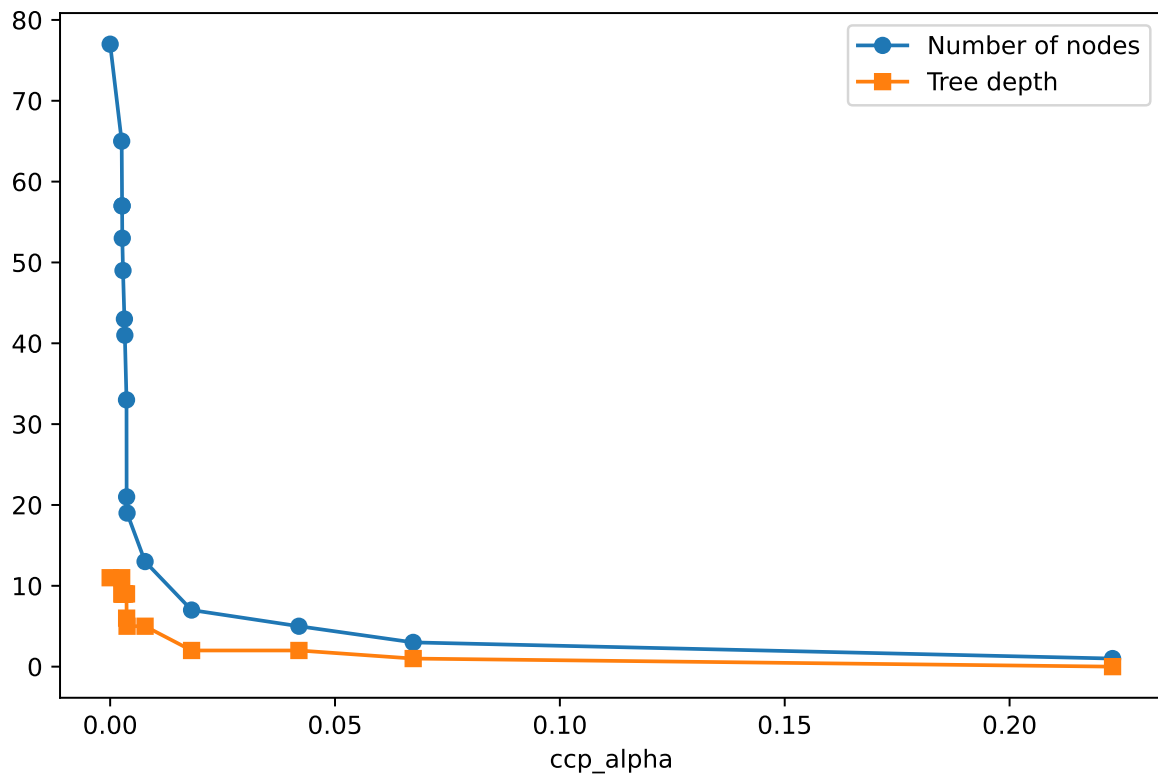


Figure 7.8: Tree size decreases as the pruning penalty increases.

The pruning parameter should be selected using validation or cross-validation, not the test set.

7.14 Feature importance

A fitted Decision Tree exposes impurity-based feature importances through `feature_importances_`.

```

importance_tree = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)

importance_tree.fit(iris.data, iris.target)

dict(
    zip(
        iris.feature_names,
        importance_tree.feature_importances_
    )
)

```

```

{'sepal length (cm)': np.float64(0.0),
 'sepal width (cm)': np.float64(0.01351351351351351),
 'petal length (cm)': np.float64(0.5581648224311269),
 'petal width (cm)': np.float64(0.42832166405535965)}

```

An impurity-based importance measures the accumulated weighted reduction in impurity produced by splits using each predictor.

Limitations of impurity-based importance

Impurity-based importance can favor continuous variables or predictors with many possible split points. It also does not indicate the direction of an effect. Permutation importance is often preferable when interpretation matters.

7.15 Regression trees

Decision Trees can also predict continuous outcomes. At each leaf, a regression tree usually predicts the mean target value of the training observations in that leaf.

We generate a noisy quadratic dataset:

```

rng = np.random.default_rng(42)

X_reg = 2 * rng.random((200, 1)) - 1

```

```

y_reg = (
    0.5 * X_reg[:, 0] ** 2
    + 0.1 * X_reg[:, 0]
    + rng.normal(0, 0.05, 200)
)

```

```

tree_reg_depth_2 = DecisionTreeRegressor(
    max_depth=2,
    random_state=42
)

```

```

tree_reg_depth_3 = DecisionTreeRegressor(
    max_depth=3,
    random_state=42
)

```

```

tree_reg_depth_2.fit(X_reg, y_reg)
tree_reg_depth_3.fit(X_reg, y_reg)

```

```
DecisionTreeRegressor(max_depth=3, random_state=42)
```

```

x_plot = np.linspace(-1, 1, 500).reshape(-1, 1)

fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))

for model, depth, ax in zip(
    [tree_reg_depth_2, tree_reg_depth_3],
    [2, 3],
    axes
):
    predictions = model.predict(x_plot)

    ax.scatter(X_reg[:, 0], y_reg, alpha=0.7)
    ax.plot(x_plot[:, 0], predictions)
    ax.set_title(f"max_depth = {depth}")
    ax.set_xlabel("$x_1$")
    ax.set_ylabel("$y$")

plt.tight_layout()
plt.show()

```

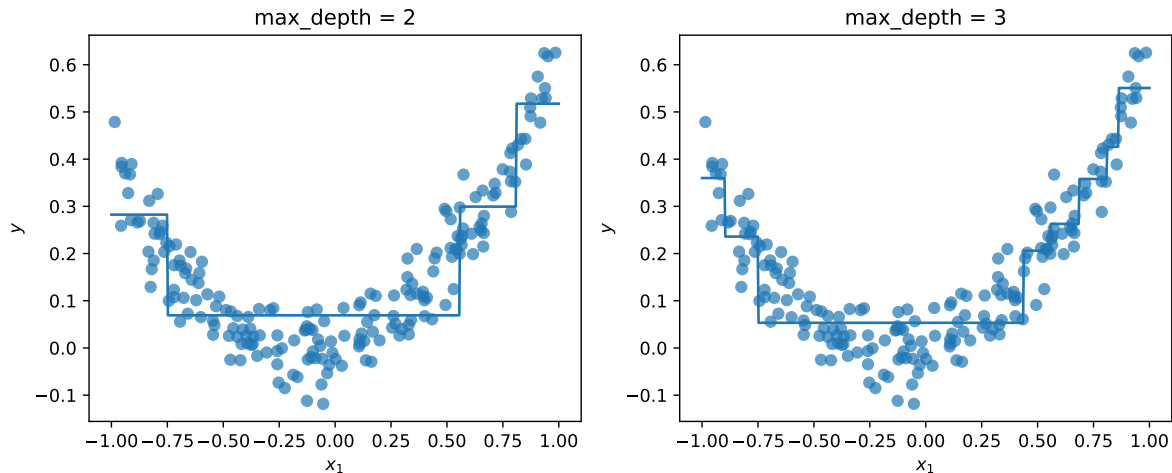



Figure 7.9: Regression trees produce piecewise-constant predictions.

A regression tree partitions the predictor space into regions and returns a constant prediction within each region. Increasing the tree depth creates more and narrower intervals.

7.16 Regression splitting criterion

For regression, CART selects the feature and threshold minimizing the weighted child-node error. With squared error, the split objective can be written as

$$J(j, t_j) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} .$$

For a leaf containing observations indexed by $\mathcal{L}$, the prediction is

$$\hat{y}_{\mathcal{L}} = \frac{1}{|\mathcal{L}|} \sum_{i \in \mathcal{L}} y_i .$$

The within-leaf mean squared error is

$$\text{MSE}_{\mathcal{L}} = \frac{1}{|\mathcal{L}|} \sum_{i \in \mathcal{L}} (y_i - \hat{y}_{\mathcal{L}})^2 .$$

7.17 Overfitting in regression trees

```
unrestricted_reg = DecisionTreeRegressor(  
    random_state=42  
)  
  
regularized_reg = DecisionTreeRegressor(  
    min_samples_leaf=10,  
    random_state=42  
)  
  
unrestricted_reg.fit(X_reg, y_reg)  
regularized_reg.fit(X_reg, y_reg)  
  
fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))  
  
for model, title, ax in zip(  
    [unrestricted_reg, regularized_reg],  
    ["Unrestricted tree", "min_samples_leaf = 10"],  
    axes  
):  
    predictions = model.predict(x_plot)  
  
    ax.scatter(X_reg[:, 0], y_reg, alpha=0.7)  
    ax.plot(x_plot[:, 0], predictions)  
    ax.set_title(title)  
    ax.set_xlabel("$x_1$")  
    ax.set_ylabel("$y$")  
  
plt.tight_layout()  
plt.show()
```

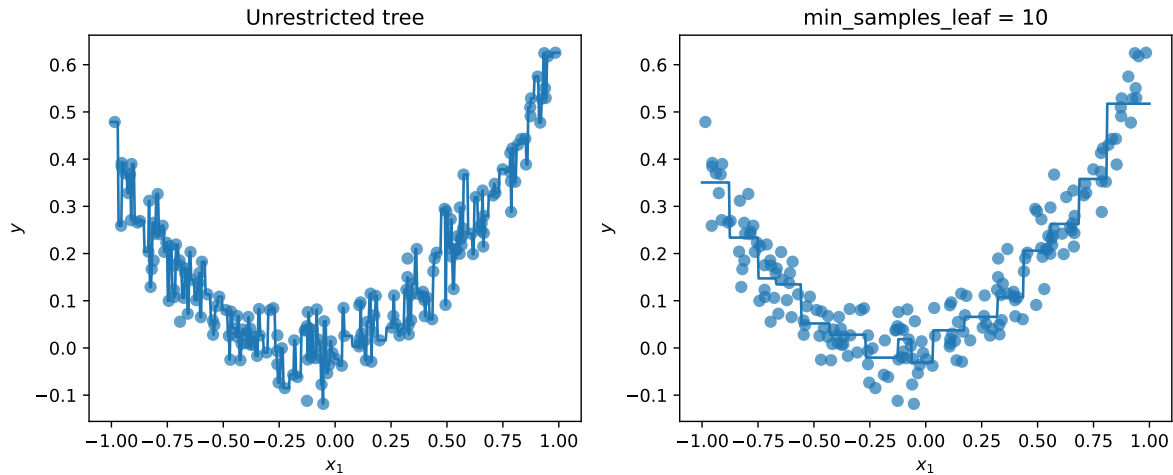


Figure 7.10: An unrestricted regression tree follows noise in the training data.

The unrestricted model creates many abrupt steps to match individual observations. Requiring larger leaves produces a smoother estimate.

7.18 Instability of Decision Trees

Decision Trees are high-variance estimators. Small changes in the training data may produce a different root split and a substantially different structure.

```
rng = np.random.default_rng(42)

indices_a = rng.choice(
    len(X_moons),
    size=len(X_moons),
    replace=True
)

indices_b = rng.choice(
    len(X_moons),
    size=len(X_moons),
    replace=True
)

tree_a = DecisionTreeClassifier(
    max_depth=5,
```

```

    random_state=1
)

tree_b = DecisionTreeClassifier(
    max_depth=5,
    random_state=2
)

tree_a.fit(
    X_moons[indices_a],
    y_moons[indices_a]
)

tree_b.fit(
    X_moons[indices_b],
    y_moons[indices_b]
)

fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))

for model, title, ax in zip(
    [tree_a, tree_b],
    ["Bootstrap sample A", "Bootstrap sample B"],
    axes
):
    DecisionBoundaryDisplay.from_estimator(
        model,
        X_moons,
        response_method="predict",
        plot_method="pcolormesh",
        alpha=0.25,
        ax=ax
    )

    ax.scatter(
        X_moons[y_moons == 0, 0],
        X_moons[y_moons == 0, 1],
        marker="o"
    )

    ax.scatter(
        X_moons[y_moons == 1, 0],

```

```

    X_moons[y_moons == 1, 1],
    marker="s"
)

ax.set_title(title)
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")

plt.tight_layout()
plt.show()

```

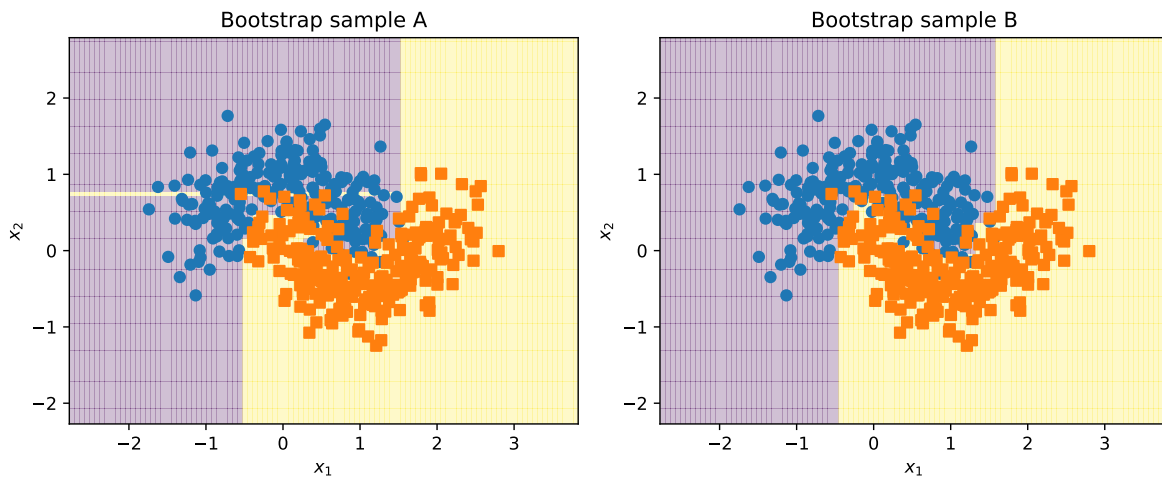


Figure 7.11: Two trees trained after a small change in the data may produce different partitions.

This instability motivates ensemble methods, which aggregate many different trees to reduce variance.

7.19 Decision Trees versus Extra Trees

An ordinary CART tree searches for the best threshold among candidate predictors at each node.

An Extra Tree introduces more randomness. It selects random thresholds for candidate predictors and then chooses the best among those randomized candidates.

```

decision_tree = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

extra_tree = ExtraTreeClassifier(
    max_depth=5,
    random_state=42
)

decision_tree.fit(X_train, y_train)
extra_tree.fit(X_train, y_train)

```

```

ExtraTreeClassifier(max_depth=5, random_state=42)

```

```

fig, axes = plt.subplots(1, 2, figsize=(9.5, 4))

for model, title, ax in zip(
    [decision_tree, extra_tree],
    ["Decision Tree", "Extra Tree"],
    axes
):
    DecisionBoundaryDisplay.from_estimator(
        model,
        X_train,
        response_method="predict",
        plot_method="pcolormesh",
        alpha=0.25,
        ax=ax
    )

    ax.scatter(
        X_train[y_train == 0, 0],
        X_train[y_train == 0, 1],
        marker="o"
    )

    ax.scatter(
        X_train[y_train == 1, 0],
        X_train[y_train == 1, 1],
        marker="s"
    )

```

```
)

ax.set_title(title)
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")

plt.tight_layout()
plt.show()
```

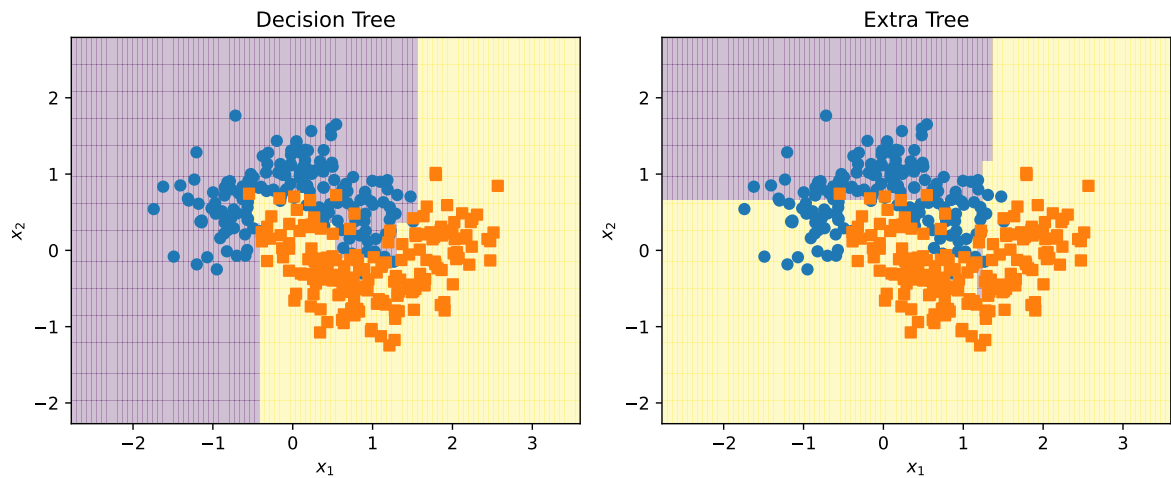


Figure 7.12: Decision boundary from a standard Decision Tree and a randomized Extra Tree.

Characteristic	Decision Tree	Extra Tree
Threshold selection	Searches candidate thresholds	Uses randomized thresholds
Randomness	Lower	Higher
Individual-tree bias	Often lower	Often higher
Individual-tree variance	Often higher	Often lower
Main use	Standalone interpretation or ensembles	Usually ensembles
Ensemble counterpart	Random Forest	Extra Trees ensemble

A single Extra Tree is not automatically better than a Decision Tree. Its main value appears when many randomized trees are aggregated.

i Extra Tree versus Extra Trees

`ExtraTreeClassifier` fits one randomized tree. `ExtraTreesClassifier` fits an ensemble of randomized trees. The variance-reduction advantage is primarily an ensemble effect.

7.20 Computational complexity

Training a balanced Decision Tree commonly requires approximately

$$O(mn \log m),$$

where:

- m is the number of training observations;
- n is the number of predictors.

Prediction is fast because an observation follows only one path from root to leaf. For a balanced tree, prediction is approximately

$$O(\log m).$$

However, an unbalanced tree may be much deeper and can have slower prediction.

7.21 Advantages of Decision Trees

Decision Trees have several important strengths:

- they are easy to visualize and explain;
- they naturally model nonlinear relationships;
- they capture interactions without explicit interaction terms;
- they require little numerical preprocessing;
- they work for classification and regression;
- they can handle multiclass problems directly;
- predictions are fast;
- they provide an interpretable sequence of decision rules.

7.22 Limitations of Decision Trees

Their main limitations include:

- high variance and instability;
- strong tendency to overfit;
- axis-aligned and piecewise-constant predictions;
- poor extrapolation in regression;
- possible bias in impurity-based feature importance;
- limited probability calibration;
- greedy optimization;
- often lower predictive performance than tree ensembles.

No reliable extrapolation

A regression tree predicts leaf averages. Outside the observed predictor range, it continues returning an existing leaf value rather than extending a trend.

7.23 Practical modeling workflow

A reliable workflow for Decision Trees is:

1. create separate training and test sets;
2. define a simple baseline tree;
3. tune structural parameters with cross-validation;
4. compare pre-pruning and cost-complexity pruning;
5. evaluate training and validation performance together;
6. inspect tree depth, leaf sizes, and class distributions;
7. assess probability calibration when probabilities matter;
8. reserve the test set for final evaluation;
9. compare the single tree with ensemble alternatives.

Example model:

```
final_tree = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=6,  
    min_samples_split=10,  
    min_samples_leaf=5,  
    random_state=42  
)
```

7.24 Common mistakes

⚠ Leaving the tree unrestricted

A fully grown tree often memorizes the training sample. Always evaluate regularization or pruning.

⚠ Selecting complexity from training accuracy

Training accuracy generally improves as the tree grows. Complexity must be selected using validation or cross-validation.

⚠ Treating feature importance as causality

A high impurity-based importance does not imply that a predictor causes the outcome.

⚠ Assuming stable interpretation

A visually simple tree may change substantially after a small change in the training sample.

⚠ Ignoring class imbalance

A tree trained on imbalanced classes may favor the majority class. Consider suitable metrics, `class_weight`, resampling, and leaf-level class counts.

7.25 Chapter summary

Decision Trees recursively partition the feature space using simple threshold rules. Classification trees predict classes and class proportions, while regression trees predict leaf averages.

CART grows binary trees greedily by selecting splits that minimize weighted impurity or prediction error. Gini impurity and entropy are common classification criteria, while squared error is frequently used for regression.

Unrestricted trees have low bias but high variance and often overfit. Their complexity can be controlled using depth limits, minimum node sizes, leaf-size restrictions, impurity thresholds, maximum leaf counts, and cost-complexity pruning.

Decision Trees are interpretable and require little preprocessing, but they are unstable and often less accurate than ensemble methods. Random Forests and Extra Trees reduce this

instability by combining many diverse trees.

7.26 Exercises

1. Fit Decision Trees with `max_depth` ranging from 1 to 10 on the moons dataset. Plot training and validation accuracy against depth.
2. Compare Gini impurity and entropy on the Iris dataset. Examine whether the resulting structures and predictions differ.
3. Fit a tree with several values of `min_samples_leaf`. Explain how this parameter changes the decision boundary.
4. Use `cost_complexity_pruning_path()` and cross-validation to select `ccp_alpha`.
5. Compare impurity-based feature importance with permutation importance.
6. Generate a nonlinear regression dataset and compare an unrestricted tree with trees regularized by `max_depth` and `min_samples_leaf`.
7. Demonstrate tree instability by repeatedly fitting models to bootstrap samples.
8. Compare `DecisionTreeClassifier`, `ExtraTreeClassifier`, and an ensemble of Extra Trees on the same dataset.
9. Explain why a regression tree cannot reliably extrapolate beyond the training range.
10. Investigate how `class_weight="balanced"` affects a Decision Tree on an imbalanced classification problem.

Part II

Part II — Neural Networks and Deep Learning

8 Logistic and Softmax Regression

8.1 Logistic Regression

Logistic Regression, also called *Logit Regression*, is commonly used to estimate the probability that an instance belongs to a particular class. For example, it can estimate the probability that an email is spam.

If the estimated probability is greater than or equal to 50%, the model predicts that the instance belongs to the **positive class**, usually labeled 1. Otherwise, it predicts that the instance belongs to the **negative class**, usually labeled 0.

Despite its name, Logistic Regression is therefore primarily used as a **binary classification algorithm**.

8.1.1 Estimating Probabilities

Logistic Regression begins by computing a weighted sum of the input features, including a bias term, just as Linear Regression does. However, instead of returning this score directly, it transforms it using the **logistic function**:

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \theta).$$

The logistic function, denoted by $\sigma(\cdot)$, is a sigmoid-shaped function defined as

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

It transforms any real-valued score into a number between 0 and 1, allowing the output to be interpreted as a probability.

Once the model estimates the probability

$$\hat{p} = h_{\theta}(\mathbf{x})$$

that an instance belongs to the positive class, it converts that probability into a class prediction using a threshold:

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5, \\ 1, & \text{if } \hat{p} \geq 0.5. \end{cases}$$

The value 0.5 is the default classification threshold, although a different threshold may be selected depending on the objectives and consequences of the classification problem.

8.1.2 Training Logistic Regression

Training Logistic Regression consists of finding the parameter vector θ that produces appropriate probabilities for the observed classes.

The objective is:

- For positive instances, where $y = 1$, the predicted probability $\hat{p}$ should be close to 1.
- For negative instances, where $y = 0$, the predicted probability $\hat{p}$ should be close to 0.

To achieve this objective, Logistic Regression minimizes a cost function known as **log loss** or **binary cross-entropy**.

Cost Function for a Single Training Instance

For a single observation, the cost is

$$c(\theta) = \begin{cases} -\log(\hat{p}), & \text{if } y = 1, \\ -\log(1 - \hat{p}), & \text{if } y = 0. \end{cases}$$

This definition has an intuitive interpretation:

- Correct and confident predictions produce a cost close to 0.
- Incorrect predictions receive a larger penalty.
- Incorrect predictions made with high confidence receive an especially large penalty.

Why Use the Logarithm?

The logarithmic function offers two important advantages:

- It strongly penalizes confident errors.
- It contributes to a convex objective function that can be optimized efficiently using gradient-based methods.

Cost Function for the Entire Training Set

For a dataset containing m training instances, the average cost is

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})].$$

Training therefore consists of solving the optimization problem

$$\min_{\theta} J(\theta).$$

The Logistic Regression cost function is convex. Consequently, gradient-based optimization does not become trapped in suboptimal local minima. Under appropriate optimization conditions, the algorithm converges toward a global minimizer.

The minimizer is not necessarily unique in every possible dataset—for example, redundant or perfectly collinear predictors may produce equivalent parameterizations—but the convexity of the objective remains an important computational advantage.

Gradient of the Cost Function

The partial derivative of the cost function with respect to parameter θ_j is

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m [\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}] x_j^{(i)}.$$

For each training instance, the optimization algorithm:

1. Computes the predicted probability.
2. Calculates the difference between the prediction and the target.
3. Multiplies this difference by the corresponding feature value.
4. Averages the result across the training instances.

The resulting gradient indicates how the parameters should change to reduce the cost.

Batch Gradient Descent uses all training observations at each optimization step. Stochastic Gradient Descent uses one observation at a time, while Mini-batch Gradient Descent uses a small group of observations.

Key Takeaways

- Logistic Regression estimates class probabilities before producing class labels.
- The logistic function maps a linear score to the interval $(0, 1)$.
- Log loss strongly penalizes confident incorrect predictions.

- The cost function is convex, so optimization is not affected by suboptimal local minima.
- Gradient-based algorithms can be used to estimate the model parameters efficiently.

8.1.3 Decision Boundaries

The Iris dataset provides a simple example of Logistic Regression. It contains sepal and petal measurements for 150 flowers belonging to three species:

- *Iris setosa*
- *Iris versicolor*
- *Iris virginica*

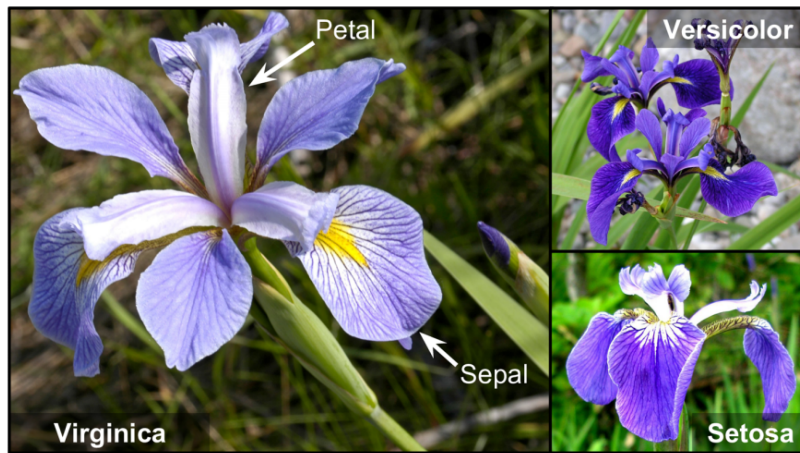


Figure 8.1: Flowers from the three Iris species.

The following block imports the required tools and loads the dataset as a pandas DataFrame. Loading it with `as_frame=True` allows the predictors to be selected using their descriptive column names.

The `load_iris()` function returns a dictionary-like `Bunch` object containing:

- the predictor matrix;
- the target variable;
- feature names;
- class names;
- metadata describing the dataset.

Listing 8.1

```
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

iris = datasets.load_iris(as_frame=True)

list(iris.keys())
```

8.1.3.1 Binary Classification Using One Feature

Suppose the objective is to identify *Iris virginica* flowers using only petal width.

The original three-class target is transformed into a binary target:

- **False:** the flower is not *Iris virginica*;
- **True:** the flower is *Iris virginica*.

The data is divided into training and test sets before fitting the model. The training set is used to estimate the parameters, while the test set is kept separate for later model evaluation.

Stratification preserves approximately the same class proportions in both subsets.

Listing 8.2

```
X = iris.data[["petal width (cm)"]].values
y = iris.target == 2

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

The Logistic Regression model is then fitted exclusively on the training data.

Once trained, the model can estimate class probabilities for any possible petal width.

The following code creates a sequence of petal-width values, obtains the corresponding probabilities, and identifies the point at which the probability of *Iris virginica* reaches 0.5.

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
```

```
y_proba = log_reg.predict_proba(X_new)
```

```
decision_boundary = X_new[  
    y_proba[:, 1] >= 0.5  
][0, 0]
```

```
plt.figure(figsize=(7, 3.5))
```

```
plt.plot(  
    X_new,  
    y_proba[:, 0],  
    "b--",  
    linewidth=2,  
    label="Not Iris virginica"  
)
```

```
plt.plot(  
    X_new,  
    y_proba[:, 1],  
    "g-",  
    linewidth=2,  
    label="Iris virginica"  
)
```

```
plt.plot(  
    [decision_boundary, decision_boundary],  
    [0, 1],  
    "k:",  
    linewidth=2,  
    label="Decision boundary"  
)
```

```
plt.arrow(  
    x=decision_boundary,  
    y=0.08,  
    dx=-0.3,  
    dy=0,  
    head_width=0.05,  
    head_length=0.1,  
    fc="b",  
    ec="b"  
)
```

```
plt.arrow(  
    x=decision_boundary,  
    y=0.92,  
    dx=0.3,  
    dy=0,  
    head_width=0.05,  
    head_length=0.1,  
    fc="g",
```

Listing 8.3

```
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

The two curves represent the estimated probabilities of the two classes. For every observation, these probabilities sum to 1.

As petal width increases, the estimated probability of *Iris virginica* also increases. The vertical dotted line marks the decision boundary, where both probabilities are approximately 0.5.

Therefore:

- Petal widths below the boundary are classified as **Not Iris virginica**.
- Petal widths above the boundary are classified as **Iris virginica**.

The squares and triangles represent the training observations used to estimate the model.

The numerical value of the decision boundary can be inspected directly:

Listing 8.4

```
decision_boundary
```

The exact value may vary slightly depending on the training subset. This is expected because the model is fitted using only the observations assigned to the training set.

Once the model has been trained, new flowers can be classified using the `predict()` method.

Listing 8.5

```
log_reg.predict([[1.7], [1.5]])
```

The first observation has a petal width of 1.7 cm, while the second has a petal width of 1.5 cm.

Their predictions depend on which side of the learned decision boundary they fall. Instead of assuming a fixed boundary value in the explanation, the predictions should be interpreted relative to the boundary estimated in the preceding block.

The underlying probabilities can be obtained using `predict_proba()`:

Each row contains the probabilities assigned to the two classes:

Listing 8.6

```
log_reg.predict_proba([[1.7], [1.5]])
```

- The first column corresponds to **Not Iris virginica**.
- The second column corresponds to **Iris virginica**.

The `predict()` method returns the class with the highest estimated probability.

8.1.3.2 Logistic Regression Using Two Features

The previous model uses only petal width. Although this simplifies visualization, it ignores potentially useful information contained in other predictors.

The following example uses both petal length and petal width. In a two-dimensional feature space, the decision boundary is no longer a single threshold. Instead, it becomes a line separating two regions of the plane.

A new training-test split is created because the predictor matrix has changed.

Listing 8.7

```
X = iris.data[
    ["petal length (cm)", "petal width (cm)"]
].values

y = iris.target == 2

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

The model is then fitted using the two predictors.

The parameter `C` controls the inverse strength of regularization. Larger values of `C` correspond to weaker regularization, while smaller values impose stronger regularization.

As shown in Figure 8.3, each point represents a training observation.

```

x0, x1 = np.meshgrid(
    np.linspace(2.9, 7, 500),
    np.linspace(0.8, 2.7, 200)
)

X_new = np.c_[x0.ravel(), x1.ravel()]

y_proba = log_reg_2d.predict_proba(X_new)
zz = y_proba[:, 1].reshape(x0.shape)

left_right = np.array([2.9, 7])

boundary = -(
    log_reg_2d.coef_[0, 0] * left_right
    + log_reg_2d.intercept_[0]
) / log_reg_2d.coef_[0, 1]

plt.figure(figsize=(7, 4))

plt.plot(
    X_train[y_train == 0, 0],
    X_train[y_train == 0, 1],
    "bs",
    label="Not Iris virginica"
)

plt.plot(
    X_train[y_train == 1, 0],
    X_train[y_train == 1, 1],
    "g^",
    label="Iris virginica"
)

contour = plt.contour(
    x0,
    x1,
    zz,
    cmap=plt.cm.brg
)

plt.clabel(
    contour,
    inline=True,
    fontsize=8
)

plt.plot(
    left_right,
    boundary,
    "k--",
    linewidth=2,
    label="Decision boundary"
)

```

Listing 8.8

```
log_reg_2d = LogisticRegression(  
    C=2,  
    random_state=42  
)  
  
log_reg_2d.fit(X_train, y_train)
```

The dashed line is the linear decision boundary. It separates the feature space into two regions:

- Flowers on one side are classified as **Not Iris virginica**.
- Flowers on the other side are classified as **Iris virginica**.

The contour lines represent different estimated probabilities of belonging to the *Iris virginica* class.

Near the decision boundary, the predicted probability is close to 0.5, which indicates greater uncertainty. As an observation moves farther from the boundary, the model assigns increasingly extreme probabilities to one of the two classes.

8.1.4 Softmax Regression

Logistic Regression can be generalized to support more than two classes directly. This generalization is known as **Softmax Regression** or **Multinomial Logistic Regression**.

Given an instance $\mathbf{x}$, the model first calculates a score for each class k :

$$s_k(\mathbf{x}) = \mathbf{x}^T \theta^{(k)}.$$

Each class has its own parameter vector $\theta^{(k)}$. These class-specific vectors can be stored together in a parameter matrix .

Once the score for each class has been computed, the Softmax function transforms the scores into class probabilities:

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}.$$

In this expression:

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is the vector containing all class scores.
- $\hat{p}_k$ is the estimated probability that instance $\mathbf{x}$ belongs to class k .

The Softmax probabilities satisfy

$$\sum_{k=1}^K \hat{p}_k = 1.$$

The predicted class is the one with the highest estimated probability:

$$\hat{y} = \arg \max_k \hat{p}_k = \arg \max_k s_k(\mathbf{x}) = \arg \max_k \left[\left(\theta^{(k)} \right)^T \mathbf{x} \right].$$

The arg max operator returns the value of k that maximizes the corresponding expression. In this case, it returns the class with the highest score and therefore the highest estimated probability.



Softmax Regression Predicts One Class

Softmax Regression predicts **one class per observation**. It is a multiclass classifier, not a multilabel or multioutput classifier.

It is therefore appropriate when the classes are mutually exclusive, such as different plant species or handwritten digits.

For example, a Softmax model can classify a flower as *Iris setosa*, *Iris versicolor*, or *Iris virginica*, but it cannot assign several of these species to the same flower.

Similarly, it is not directly suitable for a problem in which multiple labels may be correct simultaneously, such as identifying several people present in the same photograph.

8.1.4.1 Training Softmax Regression

Training Softmax Regression consists of finding a parameter matrix that assigns high probability to the correct class and low probabilities to the remaining classes.

The model minimizes the **categorical cross-entropy** cost function:

$$J() = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)}).$$

In this equation:

- m is the number of training instances.

- K is the number of classes.
- $y_k^{(i)} = 1$ when instance i belongs to class k .
- $y_k^{(i)} = 0$ when instance i does not belong to class k .
- $\hat{p}_k^{(i)}$ is the probability assigned by the model to class k for instance i .

Because the target classes are represented using one-hot encoding, only the logarithm corresponding to the correct class contributes to the cost for each observation.

When there are only two classes, this expression reduces to the binary cross-entropy cost used by binary Logistic Regression.

The gradient with respect to the parameter vector of class k is

$$\nabla_{\theta^{(k)}} J() = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}.$$

This gradient indicates how the parameters associated with each class should change to reduce the total cost.

An optimization algorithm computes the gradient for every class and iteratively updates the complete parameter matrix .

Why Cross-Entropy?

Cross-entropy penalizes a classifier when it assigns a low probability to the correct class. The penalty grows rapidly as the probability assigned to the correct class approaches zero. Consequently, the optimization process encourages the model to assign increasingly high probabilities to the observed target classes.

8.1.4.2 Multiclass Classification of Iris Flowers

The following example trains a Softmax Regression model to classify all three Iris species using petal length and petal width.

As in the binary examples, the observations are divided into training and test sets before fitting the model.

A compatible solver such as "lbfgs" can optimize the multinomial Logistic Regression objective directly.

In current Scikit-Learn versions, it is not necessary to specify the deprecated `multi_class="multinomial"` argument when using a compatible solver for a multiclass target.

Unlike binary Logistic Regression, Softmax Regression learns one set of parameters for each class.

Listing 8.9

```
X = iris.data[
    ["petal length (cm)", "petal width (cm)"]
].values

y = iris.target.values

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

Listing 8.10

```
softmax_reg = LogisticRegression(
    solver="lbfgs",
    C=10,
    random_state=42
)

softmax_reg.fit(X_train, y_train)
```

The learned intercepts and coefficients can be inspected directly:

The coefficient matrix contains one row per class and one column per predictor.

In this example:

- each row corresponds to one Iris species;
- the first column corresponds to petal length;
- the second column corresponds to petal width.

The model can then estimate the probabilities of all three species for a new observation:

The resulting row contains one probability for each class, following the order stored in

The final class prediction is the class with the highest estimated probability:

Listing 8.11

```
print("Intercepts:")
print(softmax_reg.intercept_)

print("\nCoefficients:")
print(softmax_reg.coef_)
```

Listing 8.12

```
softmax_reg.predict_proba([[5.0, 2.0]])
```

Although `X_test` and `y_test` are not used in this section, they remain isolated from the training process. They can therefore be used later to study model generalization without introducing information from the test data during parameter estimation.

Figure [Figure 8.4](#) illustrates how Softmax Regression partitions the feature space into three decision regions, one for each Iris species.

Each background color represents the class predicted by the classifier, while the contour lines indicate the estimated probability of the Iris versicolor class. Every point in the feature space belongs to exactly one decision region, and the predicted class is the one with the highest estimated probability.

Notice that the decision boundaries are still linear, but together they divide the predictor space into multiple regions corresponding to the three flower species. This is the multiclass extension of the linear decision boundary introduced earlier for binary Logistic Regression.

Logistic Regression and Softmax Regression are among the most widely used classification algorithms because they combine a solid probabilistic foundation with efficient optimization. Despite their simplicity, they often provide highly competitive performance and serve as the basis for many more advanced classification methods introduced in later chapters.

```

from matplotlib.colors import ListedColormap

custom_cmap = ListedColormap(
    ["#fafab0", "#9898ff", "#a0faa0"]
)

x0, x1 = np.meshgrid(
    np.linspace(0, 8, 500),
    np.linspace(0, 3.5, 200)
)

X_new = np.c_[x0.ravel(), x1.ravel()]

y_proba = softmax_reg.predict_proba(X_new)
y_predict = softmax_reg.predict(X_new)

zz = y_predict.reshape(x0.shape)
zz1 = y_proba[:, 1].reshape(x0.shape)

plt.figure(figsize=(8,4))

plt.plot(
    X_train[y_train == 2,0],
    X_train[y_train == 2,1],
    "g^",
    label="Iris virginica"
)

plt.plot(
    X_train[y_train == 1,0],
    X_train[y_train == 1,1],
    "bs",
    label="Iris versicolor"
)

plt.plot(
    X_train[y_train == 0,0],
    X_train[y_train == 0,1],
    "yo",
    label="Iris setosa"
)

plt.contourf(
    x0,
    x1,
    zz,
    cmap=custom_cmap,
    alpha=0.35
)

contour = plt.contour(
    x0,
    x1,

```

Listing 8.13

```
softmax_reg.classes_
```

Listing 8.14

```
print(softmax_reg.predict_proba([[5.0, 2.0]]))  
prediction = softmax_reg.predict([[5.0, 2.0]])  
  
iris.target_names[prediction]
```

Part III

Additional Material

9 Summary

In summary, this book has no content whatsoever.

References

Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* (USA) 27 (2): 97–111.
<https://doi.org/10.1093/comjnl/27.2.97>.